

THE LANGUAGE PASCALINE

Contents

1	Introduction	6
2	Summary of Pascaline extensions to Pascal	9
3	EBNF and syntax used in this document	13
4	Relationship to ISO 7185	15
4.1	Pascaline as a series of extentions	15
4.2	Additional reserved word-symbols	15
4.3	Lack of compliance with ISO 7185	15
5	Language extensions for pascaline	16
5.1	Word-symbols	16
5.2	Special symbols	16
5.3	Identifiers	16
5.4	Labels	16
5.5	Numeric constants	17
5.6	Constant expressions	18
5.7	Boolean integer operations	18
5.8	View parameters	19
5.9	Extended case statements	20
5.10	Variant record case ranges	22
5.11	Extended write/writeln statements	23
5.12	Array type shorthand	24
5.13	Container arrays	24
5.14	Parameterized Variables	28
5.15	Type converters/restrictors	29
5.16	Fixed objects	30
5.17	Extended file procedures and functions	30
5.18	Files of files	35
5.19	Added program header standard bindings	36
5.20	Redeclaration of forwarded procedures and functions	37
5.21	Unnamed function result	37
5.22	Extended Function results	39
5.23	Overloading of procedures and functions	40
5.24	Operator overloads	42
5.25	Static procedures and functions	43
5.26	Relaxation of declaration order	44
5.27	Exception handling	45
5.28	Assert Procedure	47
5.29	Extended range types	47
5.30	Character limit determination	49
5.31	Matrix mathematics	49
5.32	Modularity	51

5.32.1	Definition vs. implementation modules	54
5.32.2	Overrides.....	55
5.32.3	Parallel modules.....	56
5.32.4	Monitor signaling.....	59
5.33	Classes	62
5.33.1	Static objects.....	64
5.33.2	Dynamic objects	64
5.33.3	Class parameters	66
5.33.4	Inheritance	67
5.33.5	Inherited class parameters.....	68
5.33.6	Overrides for objects.....	69
5.33.7	Self referencing.....	70
5.33.8	Constructors and destructors.....	71
A	Annex A: Collected syntax	73
B	Annex B: Overview of standard libraries and modularity	83
B.1	Basic Language Support	83
B.1.1	extlib	83
B.1.2	strlib	83
B.2	Advanced User I/O and Presentation Management	83
B.3	Advanced device libraries	85
B.3.1	sndlib.....	85
B.3.2	netlib	85
B.4	Library procedure and function notation	85
C	Annex C: Extention Library.....	87
C.1	Filenames and Paths.....	87
C.2	Predefined paths.....	88
C.3	Time and Date.....	88
C.4	Directory Structures	90
C.5	File Attributes and Permissions	91
C.6	Environment Strings	91
C.7	Executing Other Programs.....	92
C.8	Error Return Code.....	93
C.9	Creating or Removing Paths	93
C.10	Option Character.....	93
C.11	Path Character	93
C.12	Exceptions.....	93
C.13	Functions and procedures in extlib	94
D	Annex D: String Library	101
D.1	Conventions	101
D.2	Words.....	101
D.3	Format Strings.....	101
D.4	Recycling.....	103
D.5	Procedures and functions in strlib.....	104

E	Annex E: Terminal Interface Library.....	115
E.1	Standard Output.....	115
E.2	Basic Cursor Positioning.....	115
E.3	Automatic Mode.....	116
E.4	Tabbing.....	116
E.5	Scrolling	116
E.6	Colors	116
E.7	Attributes.....	117
E.8	Multiple Surface Buffering	118
E.9	Advanced Input	118
E.10	Timers.....	121
E.11	Mouse	121
E.12	Joysticks	122
E.13	Function Keys	122
E.14	Terminal objects.....	122
E.15	Exceptions	124
E.16	Procedures, functions and methods in trmlib	124
F	Annex F: Graphical Interface Library	131
F.1	Terminal model	131
F.2	Graphics Coordinates	131
F.3	Character Drawing	131
F.4	String Sizes and Kerning.....	133
F.5	Justification	133
F.6	Effects.....	133
F.7	Tabs	133
F.8	Colors	133
F.9	Drawing Modes	134
F.10	Drawing Graphics	134
F.11	Figures.....	135
F.12	Predefined Pictures.....	135
F.13	Scrolling	136
F.14	Clipping.....	136
F.15	Mouse Graphical Position	136
F.16	Animation.....	136
F.17	Declarations.....	137
F.18	Graphical Terminal Object.....	Error! Bookmark not defined.
F.19	Procedures and functions in gralib	141
G	Annex G: Windows Management Library	148
G.1	Screen Appearance	148
G.2	Window Modes	148
G.3	Buffered Mode	148
G.4	Unbuffered Mode	149
G.5	Defacto transparency	150
G.6	Delayed Window Display	150
G.7	Window Frames	150
G.8	Multiple Windows.....	150

G.9	Parent/Child Windows	151
G.10	Moving and Sizing Windows.....	151
G.11	Z Ordering.....	152
G.12	Class Window Handling	152
G.13	Parallel Windows	152
G.14	Menus.....	153
G.15	Setting Menu Active	154
G.16	Setting Menu States	154
G.17	Standard Menus	154
G.18	Menu Sublisting.....	155
G.19	Advanced Windowing	156
G.20	Events.....	156
G.21	Procedure and Functions in manlib.....	160
H	Annex H: Widget Library	165
H.1	Tiles, Layers and Looks.....	165
H.2	Background colors and placement	165
H.3	Sizes	166
H.4	Logical Widget Identifiers	166
H.5	Killing, Selecting, Enabling and Getting Text to and from Widgets	166
H.6	Resizing and repositioning a widget	167
H.7	Types of widgets.....	167
H.8	Z ordering.....	167
H.9	Controls.....	167
H.10	Components	171
H.11	Dialogs.....	172
H.12	Events.....	174
H.13	Procedures and functions in widlib.....	178
I	Annex I: Sound Library	197
I.1	Ports	197
I.2	Channels and Instruments	199
I.3	Volume.....	205
I.4	Time and the Sequencer.....	205
I.5	Effects	206
I.6	Pitch Changes.....	207
I.7	Prerecorded MIDI	207
I.8	Waveform Files.....	207
I.9	Functions and Procedures in sndlib	208
J	Annex J: Networking Library	213
J.1	Functions and Procedures in netlib	213
K	Annex K: Character sets	215
K.1	ISO 8859-1 Character Set Encodings	215
K.2	ISO 10646 Character Set Encodings.....	215
K.3	Unicode text file I/O	215

K.4	Use of different character sets	216
L	Annex L: Character escapes	217
M	Annex M: Undefined program parameter binding	221
N	Standard exceptions.....	223

1 Introduction

"Standards are great. Everyone should have one of their own" - Anon.

Pascaline is a formal statement of a language that was created over the period 1993 to 2008 in a series of extensions to ISO 7185 Pascal. The name "Pascaline" was chosen both to show that the language is designed to be %100 compatible with the original language, and to continue the language Pascal's tribute to Blaise Pascal. The Pascaline being Pascal's calculator, Pascaline the language is "The machine Pascal built", or the "Machine that runs Pascal".

In 2008, there are several defacto standards for an extended Pascal, and one official one, the ISO 10206 standard. I have been dissatisfied with these existing extensions for the simple reason that instead of extending Pascal, they are more akin to redesigns of the language. In particular, the addition of the ability to coin pointer addresses to any variable and perform "type escapes" at will remove at a stroke all of the type security Wirth designed into Pascal. I think a good definition of what I mean by "redesign" would be the introduction of a feature that is designed to replace or duplicate a construct of original Pascal.

At the same time, I wanted Pascaline to achieve a level of completeness that the language never achieved in its original form. My goal was not to create an instructional language, but a complete and practical implementation that could address current problems in computing without further extensions or special support packages.

To design Pascaline I have enumerated a set of goals for the language design:

- To be completely upward and downward compatible with ISO 7185 standard Pascal.
- To be a "logical extension" of original Pascal. That is, to extend Pascal using the same working theories and means as the original language, and poses no element that does not interoperate completely with the original language.
- To provide a reasonable upgrade to the language capability, that can be implemented using using an existing standard compiler with minor effort compared to the original implementation of the compiler.
- To implement only features that could be implemented efficiently using existing computing hardware.

Pascaline was designed in a series of steps starting in 1993. For each feature, one or more proposals were made. Then, the proposals were evaluated, a winner chosen, and a test implementation in the compiler was made. Then, any adjustments required by the experience of actual use were performed. Every element in Pascaline is backed by a real implementation that is efficient and tested.

As for most modern languages, the major thrust of Pascaline is for extending the language via libraries, objects and code reuse. Pascaline will, and should, "obsolete itself" by allowing user written extensions to such a point that the major thrust of development with the language would become that of developing libraries of functions to cover new areas in computer applications.

This very ability to extend the language also forms the basis of a new problem in standard implementation that, although it has existed from the time Pascal was originally designed in the 1970s,

has become ever more pressing. That is the definition of standard libraries and platforms. Towards this end, the Pascaline standard, as in the standard for most of today's languages, is divided into the base standard and a "platform", consisting of a series of libraries that handle common I/O and support problems in a machine and system independent way.

These libraries are attached here as annexes. There is also a series of annexes covering issues such as character handling, string escapes and other "recommended practices" for Pascaline. The result should greatly aid the ability to write non-trivial Pascaline programs that are truly portable across machines, systems and implementations.

As I have offered in the past for Pascal, I extend the offer now, that I will evaluate Pascaline implementations for conformance to the Pascaline standard, no matter what the purpose of that implementation, public or private, profit or nonprofit. I only make the conditions that my access to the implementation be reasonable, that the authors provide me with a list of annexes that are complied with, and that an option exists in the implementation to enforce strict compliance with Pascaline regardless of any other extensions that might be present over and above Pascaline. This last is the same requirement that the ISO 7185 Pascal standard states.

This author further respectfully requests that the name "Pascaline" be applied only to an implementation that has been found to comply with the language specification here, and by the tests I provide free of charge or restriction. Further, unlike the ISO 7185 standard, I ask that no exceptions be allowed for the language (ISO 7185 5.1). A language may well comply with part of this specification, and be called a Pascal, or some other name. I only ask that it not be called "Pascaline" unless it can process the full language, aside from the appendices, without exception.

Scott A. Moore

July, 2008

2 Summary of Pascaline extensions to Pascal

Pascal defined a program as a series of nested blocks, one inside the other. The most basic block was the program block, which could contain any nested series of procedure and function blocks. Each block can contain a series of declarations containing labels, constants, types, variables, procedures and functions. Each block contains the code that executes the algorithms contained in the block.

The blocks of Pascal define a closed collection of these declarations with an interface that consists of a parameter list. In Pascaline terminology, this is a "top" interface, as the block communicates with the outside of the block via an interface at the "top" of the block:

```

+-----<> Parameter list
|
|
|-----|
| Block |
|-----|

```

Each block can also interface, informally, with declarations in the surrounding block. This is referred to in Pascaline terminology as the "side" interface:

```

|-----|
| Surrounding block |
| |-----| |
| | Block | ---< Variable declarations |
| | | ---< Procedures |
| | | ---< ... |
| |-----| |
|-----|

```

Each block can import declarations from the surrounding block.

Pascal envisioned a program as a tower of blocks resting one atop the other. This paradigm is a good one, but views the program as a monolithic whole. The provision of fixed types without the ability to extend them with change also contributes to the model of a program created as a static structure of code.

Pascaline's main thrust is to add extensibility to Pascal, and this is done by greatly augmenting the methods to create side blocks. Pascaline views programs as a series of adjoining tiles:

```

|-----| |-----|
| Block | ---<> Variable declarations <>--- | Block |
| Block | ---<> Procedures <>--- | Block |
| Block | ---<> ... <>--- | Block |
|-----| |-----|

```

In Pascaline, the program block is such an adjoining block, and adds several other block types that have this ability to export their declarations directly to other blocks. The primary of these is the module, which has all the powers of a program block, but adds the ability to specify both code that executes when the program starts, and code that executes when the program ends. In this way, modules appear as "service blocks" whose point is to provide constants, types, variables, procedures and functions to the program, along with a method both to set up such items as well as shut them down.

The program and module blocks form a group executing the same thread of execution for a program. Another module is the process, which defines a new thread of execution aside from the main program. Process modules cannot directly access program or module blocks, but the two can communicate via a monitor block, which automatically implements the multitasking primitives needed to coordinate such an exchange. A share block gives a way to define constants, types, procedures and functions that are usable by any task without the overhead of multitask coordination.

Pascaline also defines a new level of block that is an intermediate between a so-called "global" block such as a program, module, monitor, etc. Those blocks are static, with variables that are allocated for the duration of the program. This is a natural outermost block, since any program ultimately is rooted in such program constructs. The class block fits between the level of global blocks and procedure and function blocks. A class block has the ability to share its declarations via the "side", but also can have its variables created dynamically, or as part of the local variables in a block. Further, classes have the ability to be extended to any level. Any new class has the ability to be based on a previously defined class, and to have that class accepted as compatible with the base class.

Because classes define both declarations such as constants and types that have no allocation, as well as variables that do, the class must be instantiated either as part of the variable data, or via a reference. Any number of instances of a class may exist associated with such references. The instance of a class is called an object, and it is a true dynamically created set of the data associated with a class. The class contains the declaration of the format of the object it creates, and thus it is a "class" or "kind" for all of the objects created using it.

Because a class defines both a series of constants, types, variables, and also procedures and functions that can operate on those declarations, which are known as "methods", An object forms a true dynamic instance of a module. Classes complete the idea of "object orientation" which dictates that data exists as paired with the procedures or functions, referred to as "methods", needed to manipulate that data. Because classes that inherit from each other also have references that are compatible, classes can extend each other to any number of levels to implement program concepts.

To allow the static idea of parameter lists in Pascal to be extended, Pascaline implements the "overload" concept. Procedures, functions and methods can form "groups" under the same name that are differentiated by kind, type and number such that calls to such procedures and functions are sent to the instance that has the correct interface to operate on them. Built in expression operators can be overloaded, thus completing a full circle of data abstraction.

The concept of extendibility is further enhanced by the ability to override existing procedures, functions and methods. New modules and classes can override the previous meaning of them, and also extend them by performing new operations and calling the original definitions.

Pascaline has "container" types for arrays that do not specify an exact size. These types can be used to form a template to create such arrays of a runtime determined size at runtime. This allows procedures and functions to accept arbitrarily sized arrays to any dimension, and allows such arrays to be dynamically created as variables and pointer types as well.

The common case of an integer indexed array can be specified by a short form, which also underscores the idea of container types.

Pascaline introduces the break character, "_", for both identifiers and numbers. This aids readability for long identifiers and numbers.

Goto labels are freed from the restriction that they must appear as numbers, and can assume the same form as an identifier. Goto labels both retain their status as an interprocedure deep nested branching, but also allow for intermodular branching via a procedure or function call.

Pascaline implements the method of structured exception handling to handle deep nested returns. This allows code to be written that delivers exceptions to higher level code without needing knowledge of the surrounding code. This further enhances extendibility, and finally allows for complete replacement of "goto"s.

Constant expressions can be used wherever constants were used in Pascal. This makes it possible to use formulas for these constants instead of precalculated numbers.

Boolean operations on integers are permitted, and a new operator, "xor", is implemented for both boolean and integer operands.

For procedures, functions and methods, a new parameter "mode" is implemented, the "view" mode. This is identical to value parameter semantics, except that the parameter is protected from all modification. This makes certain compiler optimizations possible.

Case statements now have an "else" clause, and ranges of case constants are possible.

Case variant declarations can also use ranges of case constants.

Write/writeln can specify left justified fields, and a special mode allows the output of right padded strings in their natural length.

A new declaration exists, "fixed", which can be used anywhere a variable can, but cannot be modified. This allows the compile time specification of fixed tables, and ameliorates the need to create blocks of assignments at the start of a program.

Pascaline introduces a limited type conversion/restriction operation to convert between scalar types. This relieves the need to produce special handling to convert enumerated types to integer. It also introduces the ability to directly specify the precision needed within integer expressions, instead of always promoting such operations to the full size of integer. This allows more efficient numeric processing on small word size processors and arbitrary word length processors.

Pascaline standardizes a series of procedures and functions for files, such as binding to external file names, opening and closing a series of files, indexing within files, finding the length of a file, updating existing files, and appending to the end of such files, checking the existence of a file, and deleting and changing the name of a file.

A few new standard header parameters are introduced, including an error output, a list (or print) output, and a command line or file input.

The strict order of declarations from Pascal is relaxed in Pascaline. label, const, type, var and fixed declarations can occur in any order. This aids in the modular structure of Pascaline.

When a forwarded procedure, function or method appears as the actual declaration, the parameter list can be repeated. It is checked for congruence with the original. This allows such declarations to be created by cut and paste, and is more readable than the original method of having the actual declarations far from their forwarded declarations.

Procedures and functions can be declared as static, or non-recursive. This allows the creation of more efficient code on some processors.

Asserts are implemented, allowing the incorporation of runtime checks for code being debugged that can be removed without modifying the source.

Pascaline allows "subrange" types to be created that are larger than the natural Pascal range of an integer (-maxint..maxint). This allows an implementation to implement types that utilize double more precision while taking longer to perform them, so called "extended range" types. It also implements a set of predefined types that give an implementation defined set of unsigned and signed extended range types.

There is a predefined constant for the maximum character value in an implementation.

A way to specify a function result that obeys the rule of single entry/single exit with the result formed at the end. This is also required for operator overloads.

Besides the extensions to the base Pascal language, Pascaline defines a set of optional extension modules that define the Pascaline "platform". This is a nod to the fact that the set of support calls for an implementation make as much difference to portability for a program as the base language does.

3 EBNF and syntax used in this document

For the purpose of describing syntax elements of Pascaline, the EBNF or Extended Backus-Naur Format as used in the ISO 7185 standard is used. The syntax that appears here consists of the syntax elements from the ISO 7185 standard as modified for Pascaline use.

name = syntax-description .

Describes the syntax expansion of the syntax element by name of "name", which is terminated by '.'.

a | b

Either construct a or constructor b may appear, but not both.

{ a }

Construct a is repeated 0 or more times.

[a]

Construct a is repeated 0 or 1 times (it is optional).

'abc'

The characters "abc" appear literally.

(a b)

The elements a and b are grouped together.

The syntax expansions that appear in this document mirror the same by name in the ISO 7185 document. If a syntax element by name matches a name used in the document, and is different from the one contained there, then it represents a Pascaline extension to ISO 7185 Pascal, and replaces the original syntax definition.

Note the ISO 7185 use of ">" or "alternate" is not used here, because Pascaline does not use the "level 1" extensions of ISO 7185. The syntax for the level 1 extensions was removed.

Annex A contains a full syntax for Pascaline.

4 Relationship to ISO 7185

4.1 Pascaline as a series of extentions

The Pascaline standard accepts the entire language defined in the standard defined in ISO 7185 such that the set of features defined by Pascaline qualify as extentions under ISO 7185 3.2 “Extention”.

4.2 Additional reserved word-symbols

As allowed, the following particular spellings of identifiers are prohibited because of their use as word-symbols in the Pascaline standard:

forward	module	uses	private	external
view	fixed	process	monitor	share
class	is	overload	override	reference
joins	static	inherited	self	virtual
try	except	extends	on	result
operator	instance	start		

These word-symbols are all new word-symbols defined by Pascaline. The fact that certain identifiers are prohibited in Pascal may cause ISO 7185 compliant programs to fail to compile for this reason. To use a Pascaline implementation for such programs, one of two methods are used:

1. The ISO 7185 compliance switch is enabled (see ISO 7185 5.1 “Processors” note 2).
2. The identifiers in the program that overlap the above list are changed.

4.3 Lack of compliance with ISO 7185

ISO 7185 5.2 “Processors” allows an implementation to be considered compliant if it is accompanied by a list of the requirements for which it does not comply. Pascaline specifically does not allow such exceptions. A processor is not Pascaline compliant only if it has the following characteristics:

1. The complete requirements of ISO 7185 are followed, without exception.
2. The complete requirements of Pascaline, in this document, are followed without exception.

That is, the “list of requirements not complied with” from ISO 7185 is neither allowed as a starting point for Pascaline, nor carried forward into the language Pascaline.

Level 0 only

Pascaline is compliant with “level 0” Pascal only, as allowed for by the ISO 7185 standard (ISO 7185 5 “Compliance”). Conformant arrays are not specified in Pascaline, nor are they specified as an option under Pascaline. However, such conformant arrays could well be implemented as an extention to Pascaline.

5 Language extensions for pascaline

5.1 Word-symbols

word-symbol = 'and' | 'array' | 'begin' | 'case' | 'const' | 'div' | 'do' | 'downto' | 'else' | 'end' | 'file' | 'for' | 'function' | 'goto' | 'if' | 'in' | 'label' | 'mod' | 'nil' | 'not' | 'of' | 'or' | 'packed' | 'procedure' | 'program' | 'record' | 'repeat' | 'set' | 'then' | 'to' | 'type' | 'until' | 'var' | 'while' | 'with' | 'forward' | 'module' | 'uses' | 'private' | 'external' | 'view' | 'fixed' | 'process' | 'monitor' | 'share' | 'class' | 'is' | 'overload' | 'override' | 'reference' | 'joins' | 'static' | 'inherited' | 'self' | 'virtual' | 'try' | 'except' | 'extends' | 'on' | 'result' | 'operator' | 'instance' | 'start' .

Note that the directives "external" and "forward" in standard Pascal are promoted to word-symbols in Pascaline.

5.2 Special symbols

special-symbol = '+' | '-' | '*' | '/' | '=' | '<' | '>' | '[' | ']' | ':' | ';' | ':' | ';' | '^' | '(' | ')' | '<>' | '<=' | '>=' | ':=' | '..' | '(' | '.' | ')' | '@' | word-symbol .

5.3 Identifiers

Identifiers in Pascaline are identical to Pascal, with the addition of the break character '_'. An identifier can start with any of 'a'..'z' or '_', and continue with 'a'..'z', '_' and '0'..'9'. As in original Pascal, identifiers are not case sensitive.

identifier = letter | '_' { letter | digit | '_' }.

Example identifiers:

one_more_time

_last_time

5.4 Labels

Labels, used for goto purposes, can use the same format as identifiers under Pascaline. The original "apparent value" numeric labels of Pascal are accepted as well.

label = digit-sequence | identifier .

Example label:


```
program p;  
  
label exit;  
  
{ declarations }  
  
begin  
  
    goto exit  
  
    { statements to be skipped }  
  
    exit:  
  
end.
```

5.5 Numeric constants

unsigned-integer = decimal-integer | hex-integer | octal-integer | binary-integer .

decimal-integer = digit-sequence .

hex-integer = '\$' hex-digit-sequence .

octal-integer = '&' octal-digit-sequence .

binary-integer = '%' binary-digit-sequence .

digit-sequence = digit { digit | '_' } .

hex-digit-sequence = hex-digit { hex-digit }

octal-digit-sequence = octal-digit { octal-digit }

binary-digit-sequence = octal-digit { binary-digit }

digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' .

hex-digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' | 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | '_' .

octal-digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '_' .

binary-digit = '0' | '1' | '_' .

Both standard integer and real specifications are available. In addition, three "radix specifier" formats are available:

\$1234 - Specifies hexadecimal format

&1234 - Specifies octal format

%1010 - Specifies binary format

Each alternative format is specified with a "radix introduction" character. These formats can be specified anywhere the Pascal construct unsigned-constant is specified.

Pascaline accepts a "break character" within numeric constants. The "_" character can be used anywhere within a number:

123_456_789

It cannot be used as the first character of a number, which would make it an identifier.

The meaning of the number is considered without the break characters. Break characters can be used in any radix. They are useful for grouping digits so that the result is more readable:

12_334_222 { marked in thousands }

\$5566_1212 { marked in 16 bit sections }

5.6 Constant expressions

constant = [sign] constant-term { adding-operator constant-term } .

constant-term = constant-factor { multiplying-operator constant-factor } .

constant-factor = '(' constant ')' | 'not' constant-factor | character-string | constant-identifier | unsigned-integer .

multiplying-operator = '*' | '/' | 'div' | 'mod' | 'and' .

adding-operator = '+' | '-' | 'or' | 'xor' .

Whenever a constant appears in Pascal, Pascaline is able to accept a constant expression. Constant expressions have a syntax that is similar, but not identical to standard expressions in Pascal. However, constants can only operate on other constants, and cannot include variables.

The new operator **xor** is defined in the next section.

5.7 Boolean integer operations

expression = simple-expression [relational-operator simple-expression] .

simple-expression = [sign] term { adding-operator term } .

term = factor { multiplying-operator factor } .

factor = variable-access | unsigned-constant | function-designator | type-identifier '(' expression ')' | set-
constructor | '(' expression ')' | 'not' factor .

multiplying-operator = '*' | '/' | 'div' | 'mod' | 'and' .

adding-operator = '+' | '-' | 'or' | 'xor' .

Besides the use of and, or and not on Booleans in Pascal, Pascaline allows the use of and, or and not with integer operands. The result is the bitwise 'and', 'or' or 'not' of the bits in the integer.

In addition, Pascaline defines a new operator, **xor**, which has the same precedence as **and**. It gives the bitwise exclusive or of integers. It can also be used on boolean types, but in that case is equivalent to the operation `a <> b`.

Boolean operations on negative values are not defined, and may be treated as errors, caught at either compile time or run time, by the implementation.

5.8 [View parameters](#)

formal-parameter-list = '(' formal-parameter-section { ';' formal-parameter-section } ')' .

formal-parameter-section = value-parameter-specification | variable-parameter-specification | view-
parameter-specification | procedural-parameter-specification | functional-
parameter-specification .

value-parameter-specification = identifier-list ':' type-identifier .

variable-parameter-specification = 'var' identifier-list ':' type-identifier .

view-parameter-specification = 'view' identifier-list ':' type-identifier .

procedural-parameter-specification = procedure-heading .

functional-parameter-specification = function-heading .

A parameter to a procedure or function has a type and a "mode", that indicates the method of its passage.

Besides the Pascal parameter modes of var and value, Pascaline adds a mode introduced by **view**:

```
program p;  
  
type a = packed array [1..100] of char;  
  
procedure x(view b: a);  
  
begin  
    { use, but don't change, b }  
  
end;  
  
begin  
end.
```

View parameters have the same characteristics as value parameters, and could in fact be treated identically to value parameters. A view parameter cannot be modified or "threatened" in the procedure or function it belongs to. The meaning of "threatened" is the same as for "for" index variables of ISO 7185 (6.8.3.9), and means that the parameter cannot be assigned, used as an index in a for loop, or passed to another routine as a var parameter.

View parameters enable the underlying implementation to create more efficient code in many cases. If the above example was:

```
program p;  
  
type a = packed array [1..100] of char;  
  
procedure x(b: a);  
  
begin  
    { code to use array b }  
  
end;  
  
begin  
end.
```

The code would have to copy the actual parameter to b, which can be very inefficient for large arrays. The use of the view parameter allows the system to treat the parameter as a var parameter, but allow any expression to be passed as a value parameter would.

5.9 [Extended case statements](#)

case-statement = 'case' case-index 'of' case-list-element { ';' case-list-element } [';'] 'end' .

case-list-element = case-list-statement | case-list-default .

case-list-statement = case-constant-list ':' statement .

case-list-default = 'else' statement .

case-constant-list = case-constant-range { ',' case-constant-range } .

case-constant-range = case-constant ['..' case-constant] .

case-index = expression .

A case statement can feature an "else" clause at the end of all case selects for a given case statement:

```
program p(output) ;  
  
var y: integer;  
  
begin  
    { code that sets y }  
  
    case y of  
        1: write('one');  
        2: write('two')  
        else write('value unknown')  
  
    end  
  
end.
```

The statement indicated as "else" will be executed if the case select value does not match any of the values in the case list.

A case select value can appear as a range of values:

```
program x

var y: integer;

begin

    case x of

        1: write('one');
        2: write('two');
        3..5: write('three to five')

    end

end.
```

is shorthand for:

```
program p;

var y: integer;

begin

    { statements that set y }

    case y of

        1: write('one');
        2: write('two');
        3, 4, 5: write('three to five')

    end

end.
```

5.10 Variant record case ranges

The case constants on a variant record specification can appear as ranges:

record-type = 'record' field-list 'end' .

field-list = [(fixed-part [';' variant-part] | variant-part) [';']] .

variant-part = 'case' variant-selector 'of' variant { ';' variant } .

variant = case-constant-list ':' '(' field-list ')' .

case-constant-list = case-constant-range { ',' case-constant-range } .

case-constant-range = case-constant ['..' case-constant] .

```
program p;  
  
type sr = 1..5;  
  
var r: record  
  
    i: integer;  
    case s: sr of  
  
        1: (q: integer; b: boolean);  
        2: (z: real; t: array 10 of integer);  
        3..5: (o: set of char; j: integer)  
  
    end;  
  
begin { program p }  
end.
```

For each case range, the starting constant must be less than or equal to the ending constant.

5.11 Extended write/writeln statements

Pascaline extends the meaning of field width parameters in ISO 7185 6.9.3.1 to include negative or zero field widths. The appearance of a negative value as a field width has the same effect as the absolute value of the field width, except that the resulting character translation is set left justified within its field. I.e., the spaces that are specified to pad each output format to the field width are output after the contents of the field, not before.

```
write(f, i:-10);
```

Would output to file f the integer i in 10 spaces at least, with any padding to the right side of the number.

A zero field parameter is only applicable for string-type parameters. Usually, the field width for a string parameter means to output exactly that many characters of the string, truncating or padding the string as required. The appearance of a zero field width with a string indicates that the number of characters output will be defined as the number of characters from the first index of the string (1), to the last non-blank character of the string. If there are no non-blank characters in the string, then no characters are output at all.

```

program p;

var f: text;
    s: packed array 30 of char;

begin

    write(f, s:0);

end.

```

The zero field parameter extension provides a simple method to output so called "space padded" strings, which are Pascal strings with padding added to the right side to make them effectively fixed character arrays.

5.12 Array type shorthand

array-type = 'array' [dimension-specifier] 'of' component-type .

dimension-specifier = index-specifier | range-specifier .

index-specifier = '[' index-type { ',' index-type } ']' .

range-specifier = unsigned-integer { ',' unsigned-integer } .

Pascaline features a special declaration format for the most common case of integer indexed arrays:

```

type x = packed array 10 of char;
type y = array 20,10 of integer;

```

are equivalent to:

```

type x = packed array [1..10] of char;
type y = array [1..20,1..10] of integer;

```

Denoting an array index by size, instead of by an index specification, uses a different syntax, without the '[' ']' characters, to make it clear what kind of array specification is being used.

5.13 Container arrays

array-type = 'array' [demension-specifier] 'of' component-type .

The specification of only fixed arrays in original Pascal caused several practical issues with the language. Pascaline has the concept of "container arrays", which are array types specified without index specifications.

```

type a = array of integer;

```

Container arrays are a "template type" which cannot be used to directly specify a static type. For example:


```
program p;  
  
type a = array of integer;  
  
var b: a;  
  
begin  
end.
```

Would not be correct because type **a** does not contain enough information, namely the size of the array, to define the static variable **b**.

The power of container arrays is their ability to be bound to a fully defined static array at a time later than when the program is compiled. This can happen in two different ways:

1. When a parameter is accepted for a procedure or function.
2. As a result of the creation of an anonymous array in dynamic storage.
3. By use of a parameterized variable declaration (covered in the next section).

It is important to understand that Pascaline does not specify what are called "dynamic arrays", which is the ability to resize an array at runtime. Rather, Pascaline simply provides the means to create fixed arrays of any size at runtime and bind to them, or to create generate purpose procedures or functions that can bind to any size array at runtime. This distinction may seem to be unnecessary, but in practice it dramatically simplifies implementation of the concept.

Container types can have multiple index levels:

```
type a = array of array of array of boolean;
```

Container array types are compatible with other container and fixed array types that:

1. Have the same base types.
2. Have the same packed/unpacked status.
3. Have the same number of index levels.

Thus, the following types are compatible:

```
type a = packed array [1..100] of char;
```

and

```
type b = packed array of char;
```

```
type a = array [1..10, 1..100] of integer;
```

and

```
type b = array of array of integer;
```

A container array can be used for a procedure or function parameter:

```
type string = packed array of char;
```

```
procedure x(var s: string);
```

As with normal Pascal array parameters, if the parameter is passed by value, there can be considerable expense associated with copying the array to private location for the procedure or function.

When container arrays are used as parameters, they are simply serving as holders for an existing array. In the case of variable parameters, the array was created elsewhere in the code and the parameter simply points to it. In the case of value parameters, the array is created anew in a private location, and the existing array copied to it. In both cases, the information needed to complete the template for the array exists elsewhere.

Another way to allocate a container array is to use "new" to allocate it dynamically:

```
program p;
```

```
type ap = ^a;  
      a = packed array of char;
```

```
var b: ap;
```

```
begin
```

```
    new(b, 10);  
    b := 'hi there  ';
```

```
end.
```

Container arrays have the interesting property that they are free of index typing. Container arrays are considered to be compatible with other arrays (standard or container) that have the same number of base elements.

This means that the following assignments are possible:

```
program p;  
  
type a = packed array [20..120] of char;  
      b = array of char;  
      c = ^b;  
  
var q: a;  
     x: c;  
  
begin  
  
    new(x, 100);  
  
    x := q;  
  
    q := x;  
  
end.
```

It does not matter what the first or last element of the array is. When elements are copied from or to a compatible container, or when a container is used to reference a standard array, the elements are matched up from the first to the last without regard to the exact number of the index used. In the last example, `q[20]` and `x[1]` refer to the same element.

When a container array is indexed, the method of specifying its indices is always the integers 1 to `n`, where `n` is last element of the array. The last element of a container array, which happens to also be its length, can be found by the predefined function:

```
max(a, l);
```

where `a` is the container array, and `l` is the indexing level. For example:

```
program p;  
  
type a = array of array of array of integer;  
      b = ^a;  
  
var c: b;  
     i: integer;  
  
begin  
  
    new(c, 10, 20, 30);  
  
    i := max(c, 2);  
  
end.
```

Finds the second level index length of `b`, which is 20.

Specifying a level for **max** is optional. The function call:

```
max (a) ;
```

is equivalent to:

```
max (a, 1) ;
```

5.14 Parameterized Variables

Although containers cannot be directly used as the type of a variable, or part of a variable, they can be used for the type of a parameterized variable declaration:

variable-declaration = identifier-list ['(' expression { ',' expression ')' } ':' type-denoter .

variable-declaration-part = ['var' variable-declaration ';' { variable-declaration ';' }] .

```
program p;

type z = array of integer;

procedure q(i: integer; v: z);

var a(max(v)): array of integer;

begin
    for i := 1 to max(v) do a[i] := v[i]+1
end;

begin
end.
```

Variables which are declared using containers use a parameterized form that takes one or more expressions on the left side, which are used to fulfil the indicies that appear in the type of the variable on the right side.

The parameter expressions must be integer.

Each expression will be used to match an index in the type. If the type is formed from a nested declaration, the indicies are processed from the outermost declaration to innermost. If multiple dimension arrays exist, the indicies appear from major to minor.

Every index in the type must be matched. If there are more or fewer index expressions than there are indicies in the type, it is an error.

The expression used for an index parameter is not limited. Variables, parameters and functions can be used, but all of the elements used must be fully defined.

The evaluation of the index parameters are evaluated before the code in the block for which the variable is declared exists. The indices do not vary during the execution of the declaring block.

If a parameterized variable statement contains only constant parameters, the compiler may treat the variable as equivalent to a fixed type with the same parameters.

If the parameterized variable statement contains local variables, it is an error, since such variables are, by definition, undefined at the start of the block. Note that this does not apply to parameters, which are initialized externally to the procedure or function that contains the parameterized variable statement.

5.15 Type converters/restrictors

expression = simple-expression [relational-operator simple-expression] .

simple-expression = [sign] term { adding-operator term } .

term = factor { multiplying-operator factor } .

factor = variable-access | unsigned-constant | function-designator | type-identifier '(' expression ')' | set-constructor | '(' expression ')' | 'not' factor .

multiplying-operator = '*' | '/' | 'div' | 'mod' | 'and' .

adding-operator = '+' | '-' | 'or' | 'xor' .

Pascal defines the function "ord" to convert any scalar or character type to integer, and defines the function "chr" to convert integer to character. Pascaline extends this system to include any scalar type by use of a "type converter":

```
program p;

type a = (one, two, three);
      b = 20..30;

var x: a;
     y: integer;

begin

    x := a(1);
    y := b(y+1);

end.
```

The first assignment yields the value for "two" (Pascal enumerated types are defined as the integers from 0 to n). The second assignment would seem to have no function. However, the compiler can take this as a hint that instead of promoting the expression y+1 to the full size of an integer, that the operation can be performed in only the precision required for values within 20..30. This can enable more efficient processing of expressions on some machines.

When a type restrictor appears, the compiler can generate an error for values in the restricted value that cannot fit in the range of the target type. Type converters and restrictors never simply discard values.

5.16 Fixed objects

block = { declaration } ['private' declaration] statement-part [';' statement-part] .

declaration = label-declaration-part | constant-definition-part | type-definition-part | variable-declaration-part | fixed-declaration-part | procedure-declaration ';' | function-declaration ';' .

fixed-declaration-part = ['fixed' fixed-declaration ';' { fixed-declaration ';' }] .

fixed-declaration = identifier-list ':' type-denoter '=' value-constructor.

value-constructor = array-value-constructor | record-value-constructor | constant .

array-value-constructor = 'array' value-constructor { ',' value-constructor } 'end' .

record-value-constructor = 'record' value-constructor { ',' value-constructor } 'end' .

In addition to declaring variables, Pascaline can declare a program construct that appears anywhere a variable can, but is defined completely at compile time:

program p;

```
fixed a: integer = 1;
      b: array [1..10] of integer =
          array 5, 6, 8, 2, 3, 5, 9, 1, 12, 85 end;
      c: record a: integer; b: char end = record 1, 'a' end;
      d: array [1..5] of packed array [1..5] of char = array

          'tooth',
          'trap ',
          'same ',
          'fall ',
          'radio'

      end;
```

```
begin
end.
```

The declaration of a fixed is similar to a var declaration, but each variable is given a "value" part that contains a value constructor.

Fixed types can be used anywhere a variable type can, but a fixed type cannot be "threatened" in the sense of the "for" variable threat of ISO 7185 (6.8.3.9).

5.17 Extended file procedures and functions

Pascaline defines several additional functions dealing with files. Files that are not declared as external via the program header are normally given the property that they are anonymous and may be deleted at the end of the program. In Pascaline, such files can also be named using the function **assign**:

```
program p;  
  
var f: text;  
  
begin  
  
    assign(f, 'myfile');  
  
    { statements to use file f }  
  
end.
```

Assign takes any string (defined in original Pascal as a packed array of characters with a starting index of 1 and any length). The format of the file name contained in the string is entirely implementation dependent, but most implementations will honor, at minimum, the conventions that:

1. Leading and trailing spaces in the name are ignored, and removed if required.
2. That the characters in the set ['a'..'z', 'A'..'Z', '0'..'9', '_'] are valid in a filename.
3. The filename must begin with characters from the set ['a'..'z', 'A'..'Z', '_'].

Note that this is only a subset of implementation defined filename conventions. The particular Pascaline implementation will have its own additional requirements for the formatting of filenames.

The Pascaline standard library **extlib** will has higher level structuring of filenames that are upward compatible with the above. See Annex C.

The existence of a named file implies, but does not require, that the file is not deleted at the end of the program run. It further implies that applying reset to a named file will allow its preexisting contents to be accessed.

As in standard Pascal, applying reset or rewrite to a file causes the file to change from the "undefined" state to "read" for reset, and "write" for rewrite. It is an error to apply assign to a file that is not in the undefined state, that is, to rename it while it is active.

To allow the processing of multiple files by name, the procedure **close** exists:

```
program p;

var f: text;
    names: array 5 of packed array 10 of char = array

    'myfile    ',
    'thisfile  ',
    'thatfile..',
    'nextfile  ',
    'lastfile  '

    end;
    i: integer;

begin

    for i := 1 to 5 do begin

        assign(f, names[i]);

        { statements that use file f }

        close(f)

    end

end.
```

Close causes bond between the file variable and the named file itself to be broken, and the file state to be returned back to undefined. In this state, it can have **assign** applied again. In this way, a sequence of named files can be processed. **Close** implies that if the file is anonymous, it is deleted, and that the file returns to being anonymous unless assigned again.

Because in Pascaline, a named file is implied to persist beyond the end of a program, it is also implied that each file that exists in a read or write state has a **close** performed on it automatically either when the program ends, or when the file variable falls out of activation, or some combination of both.

Standard Pascal defines files as a series of elements which can be examined in turn. Pascaline defines the standard function **length** to give the number of elements within a file:

```
program p;

var f: file of integer;

begin

    assign(f, 'myfile');
    writeln('The length of the file is: ', length(f));
    close(f)

end.
```


The function length is equivalent to:

```
program p;  
  
type fi: file of integer;  
  
function length(var f: fi): lcardinal;  
  
var l: integer;  
  
begin  
    l := 0;  
    reset(f);  
    while not eof(f) do begin get(f); l := l+1 end;  
  
    length := l { return length }  
  
end;  
  
begin  
end.
```

Except that the built in Pascaline function does not change the location within the file. The length function cannot be applied to a file of type text, because the length of line endings is implementation defined, so the length of a text file is as well.

Each element of a Pascal file can be enumerated. In Pascaline, the elements of a file that is not of type text are numbered from 1 to n, where n is the last element of the file. The current location within the file is found with the built in function location:

```
program p(output);  
  
var f: file of integer;  
  
begin  
    writeln('The location in the file is: ', location(f));  
  
end.
```

The current location within a file can be set by the built in procedure "position":

```
program p;

var f: file of integer;

begin

    position(f, 10)

    { statements to use f }

end.
```

Position will work with any element from 1 to $n+1$, where n is the last element of the file. Position allows the location to be set one beyond the length of the file so that the file can be extended at the end.

As with length, there is no way to apply either location or position to a text file.

In standard Pascal, there is no way to update a preexisting file with new data. In Pascaline the procedure **update** can be used instead of **rewrite**:

```
program p;

var f: file of integer;

begin

    assign(f, 'myfile');
    update(f);

    { statements to use f }

end.
```

As for **rewrite**, **update** places the file in write mode. However, **update** does not clear the previous contents of the file. **Update** sets the location within the file at 1, just as **reset** does. If the common operation of updating a file at the end is wanted, the following code does this:

```
program p;

var f: file of integer;

begin

    assign(f, 'myfile');
    update(f);
    position(f, length(f)+1);

    { statements to use f }

end.
```

Update only works with non-text files. The built in procedure **append** performs the same action as above, but works with both text and non-text files:

```
program p(output);  
  
var f: text;  
  
begin  
  
    assign(f, 'myfile');  
    append(f);  
    writeln(f, 'hi there');  
  
end.
```

To determine if a file by name exists in the system, the function **exists** can be used:

```
if exists('thisfile') then { operate on 'thisfile' };
```

The **exists** function takes a variable reference to a Pascal string, and returns a boolean value that is true if the file exists.

The **exists** function can be used to determine beforehand if a **reset** will yield an error because of a non-existent file.

To delete a file by name, the **delete** procedure is used:

```
delete('oldfile');
```

Delete accepts a variable reference to a string, and causes the file to be removed from the system. It is an error if the file does not exist.

To change the name of an existing file, the **change()** procedure is used:

```
change('newname', 'oldname');
```

The file with the name of the second parameter is changed to be the first parameter. Both parameters are variable references to strings. It is an error if the second parameter does not exist in the system, and also if the first parameter exists before the **change** call. It is also an error if any differences exist in the names besides just their principal names. This rule forbids, for example, the changing of the location or system parameters of the file using the **change** call.

5.18 Files of files

ISO 7185 forbids files to contain any other files, either directly, or as part of a substructure (see ISO 7185 6.4.3.5 “File-types”). In Pascaline, this restriction is removed. File types can have as component-type either a file, or a structure containing a file at any level:

```

program p;

type textlist: file of text;

var fl: textlist;

procedure getfilelist(var l: textlist);
begin
    { get list of text files }

end;

begin
    getfilelst(fl);
    while not eof(fl) do begin { traverse files list }

        while not eof(fl^) do begin{ read file}

            if fl^^ > ' ' then { process non-space char };
            read(fl^, c) { next character }

        end;
        get(fl); { next text file }

    end

end.

```

Files of files are a convenient method to represent arbitrary lists of files. For example, a call to present a “user file dialog” which allows a user to select multiple files from a directory, could return the resulting list as a file of files.

5.19 Added program header standard bindings

Besides the standard Pascal textfile external bindings of **input** and **output**, there are three new external bindings defined by Pascaline:

error

This file can be used to output error messages or warnings to. It may be aliased to **output** on an implementation. The purpose of the **error** file is to provide a output file that will always go to the user's console, and cannot be redefined as routed to another display device.

list

This file is used to output for what is assumed to be printout from any attached hard copy device. The implementation may alias this file to the **output** file if such a device is not available, or if hard copy is not desired.

command

The **command** file is a text line or series of text line that deliver instructions to the running program. Most commonly, it carries a single line containing the remaining text of the command line that activated the running program, after the command name itself is discarded. However, it can contain any instruction line, and can even be directed to be an entire file containing such commands.

The exact format of the commands and parameters contained in file are implementation dependent. If the implementation has no concept of a command line, the file can be assigned to another file, or may appear empty.

Since on many systems a command line exists as a line of text in memory, not as part of a file system, it is understood that the implementation will simulate the access to such a line as a file, if that is required.

In this Pascaline specification, implementers are reminded of the requirements of ISO 7185 6.10 "Programs" wherein it is an error if a program parameter is acted on and cannot complete that action. This clause of ISO 7185 Pascal specifically forbids ignoring the program parameters.

Further, in Pascaline it is strongly suggested that the program parameters which are common text files be connected, or can be caused to connect, to external files by binding. See Annex M.

5.20 Redeclaration of forwarded procedures and functions

In Pascaline, the parameters of a forwarded procedure or function can be repeated. As in ISO 7185 Pascal, the names of the parameters are the same as the original names in the forwarded header. The new names, if they happen to be different in the repeated header, are ignored. However, the two parameter lists are checked for "congruency" using the same rules as procedure and function parameters in ISO 7185 6.6.3.6 "Parameter list congruity".

Repeating the parameters in a forwarded procedure or function both increases the self documenting nature of a program, by keeping the parameters close to the actual body of the procedure or function, as well as making it easier to create such headers, by simple block copy.

5.21 Unnamed function result

Pascaline has an alternate method to indicate a function result:

```
function-declaration = function-heading ';' directive | function-identification ';' function-block |
                      function-heading ';' function-block .
```

```
function-block = block .
```

```
block = { declaration } [ 'private' { declaration } ] statement-part [ ';' statement-part ] .
```

```
compound-statement = 'begin' statement-sequence [ 'result' expression ] 'end' .
```

At the end of the main block of a function, a result statement can appear:

```
program p;  
  
function y(a, b: integer): integer;  
  
begin  
  
    result a+b  
  
end;
```

The result statement is an alternative to the ISO 7185 arrangement of assigning the result to the name of a function that is active. It can only be used in a function, and it can only be used at the end of the function. It can only give the result of the enclosing function (in contrast to ISO 7185 function results). It cannot be used in conjunction with a ISO 7185 assignment to the name of the function appearing in either the block of the function whose result is to be set, nor any nested block. Thus, the following example is illegal:

```
{ illegal example }  
  
program p;  
  
function y(a, b: integer): integer;  
  
procedure q;  
  
begin  
  
    y := 1  
  
end;  
  
begin  
  
    result a+b  
  
end;
```

But this example is legal:

```
program p;  
  
function y(a, b: integer): integer;  
  
function q: integer;  
  
begin  
    y := a+b;  
  
    result 1  
  
end;  
  
begin  
    a := q  
  
end;
```

The assignment to the function result **y** is legal in **q**, even though **q** is using the result method, because the function **y** is using the ISO 7185 named function result method.

Unnamed function results give a clear way to indicate the function result, that obeys the single entry/single exit rule with the function result formed at the exit. They are also required in order to indicate the result of an operator overload function (See 4.23).

5.22 Extended Function results

Function results can be any type, including structured:

```
program p;  
  
type a: array 10 of integer;  
  
function y: a;  
  
var z: a;  
    i: integer;  
  
begin  
    for i = 1 to 10 do z[i] := i+10;  
  
    y := z  
  
end;  
  
begin  
end.
```

Such extended function results imply greater costs due to the need to copy the structure back to the caller.

Files cannot be returned as a function result.

5.23 Overloading of procedures and functions

procedure-heading = attribute 'procedure' identifier [formal-parameter-list].

function-heading = attribute 'function' identifier [formal-parameter-list] ':' result-type.

attribute = 'overload' | 'static' | 'virtual' | 'override' | 'operator' .

The word-symbol "overload" can be used to specify that one procedure or function forms an overload set with another:

```
program p;  
  
procedure x(y: integer);  
  
begin  
    { statements of to operate on parameter y }  
  
end;  
  
overload procedure x(c: char);  
  
begin  
    { statements to operate on parameter c }  
  
end;
```

Any number of procedures or functions with the same name can be joined this way into an overload group. When the procedure or function is called, the compiler determines which of the procedures or functions in the overload group is to be used to satisfy the call. Thus, both of these calls are possible with the definitions above:

```
x(1);  
x('c');
```

The compiler considers three things when determining which procedure or function of an overload group is meant:

1. If the called context is a procedure or function call.
2. The number of parameters.
3. The type compatibility of the parameters.

Overload groups are **not allowed** to be ambiguous. There is no "preference" or "priority" involved with overloads. If two overloads are ambiguous with respect to each other, then an error will result at the time of the definition of the conflicting overload.

Two calls are ambiguous with each other if they match for procedure/function status, have the same number of parameters, and the parameters are compatible between lists, using the ISO 7185 definition of compatibility according to 6.4.5 "Compatible types" or 6.4.6 "Assignment-compatibility" (where either parameter could be assignment compatible with the other), and also if one parameter is char while the other is a single character string. This means, for example, that having one parameter of type real, and the other of type integer, means those parameters are ambiguous with each other.

When parameter lists are compared, the mode of the parameter, or its **var**, **view** or value status is ignored.

Further, it is an error if two parameter lists "converge" with different modes. Two parameter lists are said to converge at any point in the list if that parameter and all parameters to the left of it are found to be compatible using the rules above. An example of an overload declaration that is invalid by this rule is

```
program p;

procedure a(i: integer; c: char; b: boolean);

begin
    { statements of first overload of a }

end;

overload procedure a(i: integer; var c: char; b: boolean; r: real);

begin
    { statements of second overload of a }

end;

begin
end.
```

Even though the two parameter lists would be distinct because they differ in number, the lists are convergent at parameter **c**, and parameter **c** has conflicting modes, the first being value, and the second being **var**.

The purpose of the convergence rule is to relieve compilers from having to store all the parameters in a procedure or function call before determining which routine of an overload group matches the call.

Overload groups must be completed within the same declaration block. Different modules, classes, procedures and functions cannot add to each other's overload groups.

5.24 Operator overloads

procedure-declaration = procedure-heading ';' directive | procedure-identification ';' procedure-block |
procedure-heading ';' procedure-block .

procedure-heading = attribute 'procedure' formal-identifier [formal-parameter-list] .

function-declaration = function-heading ';' directive | function-identification ';' function-block |
function-heading ';' function-block .

function-heading = attribute 'function' formal-identifier [formal-parameter-list] ':' result-type .

attribute = 'overload' | 'static' | 'virtual' | 'override' | 'operator' .

formal-identifier = identifier | 'not' | '+' | '-' | '*' | '/' | 'div' | 'mod' | 'and' | 'or' | '<' | '>' | '=' | '<=' |
'>=' | 'in' | 'is' | ':=' .

Besides overloading procedures and functions, it is also possible to overload expression operators:

```

program p;

type string = packed array of integer;
      pstring = ^string;

operator function +(a, b: string): pstring;

begin
    { code to concatenate strings a and b }
end;

operator procedure :=(a, b: string);

begin
    { code to assign string b to a }
end;

begin
end.

```

Where “+” and “:=” are the operators to overload. The following operator overloads are possible:

```

operator function not (a): r;
operator function +(a; b): r;
operator function -(a; b): r;
operator function *(a; b): r;
operator function /(a; b): r;
operator function div(a; b): r;
operator function mod(a; b): r;
operator function and(a; b): r;
operator function or(a; b): r;
operator function <(a; b): r;
operator function >(a, b): r;
operator function =(a, b): r;
operator function <=(a, b): r;
operator function >=(a, b): r;
operator function in(a, b): r;
operator function is(a, b): r;
operator procedure :=(a, b): r;

```

Note that each **a** and **b** above indicates **a** full parameter with type, and **r** indicates a return type.

The ‘-’ and ‘+’ operators can be both unary and binary, and they have different priorities. The exact type of operator is indicated by the number of parameters that appear. There is only one operator procedure, which is assignment (‘:=’).

The parameters to an operator procedure or function can be any type, but cannot be **var** parameters.

The results of a operator function cannot be established via a ISO 7185 by name function result assignment. Instead, the unnamed result form is used (see 4.20).

The existence of an operator overload does not change the priority of the operator.

Operator overloads follow the same rules as standard overloads. Specifically, operator overloads cannot be ambiguous with either each other, or the parameters of standard , built in operators.

The types of built in operators appears in ISO 7185 6.7.2 “Operators”.

The existence of an operator overload will not cause the built in operator to be altered in any way. An operator overload simply adds behavior where the built in operator would indicate a type error.

5.25 Static procedures and functions

procedure-heading = attribute 'procedure' identifier [formal-parameter-list].

function-heading = attribute 'function' identifier [formal-parameter-list] ':' result-type.

attribute = 'overload' | 'static' | 'virtual' | 'override' | 'operator' .

Pascaline defines a "static" attribute word-symbol that is used before any function or procedure to indicate that it is used statically.

```
program p;  
  
static procedure z;  
  
var x, y: integer;  
  
begin  
    { statements of z }  
  
end;  
  
static function y;  
  
    { statements of y }  
  
end;  
  
begin  
end.
```

The **static** word-symbol is used to indicate to the compiler that the procedure or function with that attribute will not recursively call itself.

On some processors with limited addressing resources, using static can result in smaller and faster executable programs, sometimes dramatically so. Compilers targeting more advanced processors typically ignore the **static** word-symbol.

In the case where the procedure or function is contained entirely within a module, and is not visible outside the module (it is within the private section), the compiler can automatically determine the static status of a procedure or function. However, if it is visible outside the module, the compiler must assume it may be recursively called. In this case, the static attribute is required to obtain the improved code.

The compiler may also detect that a **static** attributed procedure or function calls itself, either directly or indirectly via another procedure or function, and issue an error.

If a static attributed procedure or function recursively calls itself, the result is undefined, and programs should specifically not count on the resulting behavior.

5.26 Relaxation of declaration order

block = { declaration } statement-part .

declaration = label-declaration-part | constant-definition-part | type-definition-part | variable-declaration-part | fixed-declaration-part | procedure-declaration ';' | function-declaration ';' .

The declaration order in standard pascal of labels, constants, types, variables, then procedures and functions is relaxed. These constructs can appear in any order. However, the rule remains that each

identifier must be declared before its use, with the normal ISO 7185 Pascal exception for pointer declarations.

Relaxing declaration order in Pascaline is a more natural match for modular compilation.

5.27 Exception handling

try-statement = 'try' (statement-sequence | 'start') (except-series | except-unconditional) ['else' statement] .

except-series = except-clause { except-specifier } [except-unconditional] .

except-unconditional = 'except' statement .

except-specifier = 'on' exception-identifier 'except' statement .

The most common use for interprocedure gotos is to handle error returns easily from deep nested procedures. Pascaline provides an alternative structure. The **try** statement executes one or more statements with an "exception guard":

```

program p;

var myexception: exception;

begin

    try

        { code to be guarded }

    on myexception except { statement to execute on myexception }
    except { statement to be executed on any exception }
    else { statement to be executed if no exception occurs }

end.

```

If anywhere within the guarded execution the statement:

```
throw myexception;
```

or

```
throw;
```

is executed, any active procedures or nested statements are removed, and execution resumes with the statement after either the exception clause that matches the given exception, or the general exception statement. If no exception occurs, execution continues either with the **else** statement, if specified, or the next statement after the **try** statement.

The **try** statement may carry a specific list of exceptions, followed by an optional unconditional exception, or simply an unconditional exception. An unconditional exception matches any exception. A specific exception only matches the named exception.

When an unspecified exception is thrown, the result is as if an exception was thrown that does not appear anywhere in the program. It is the “master” exception that always exists in any program.

Try statements may nest. The result of executing "throw" is to return to the innermost try statement currently executing. If a **goto** is used to branch outside of the **try** statement, then it will be effectively removed from activation, and any **throw** statements will go to the next outer **try** statement. This means that a **goto** statement must be ready to unwind any **try** statements within its scope as required.

If an exception is thrown, and either there is no **try** statement active, or no **try** statement matches the exception indicated, then the result is an error. The result of unhandled exceptions is as if a **try** statement exists outside of the program that catches any exceptions that are not handled within it, and handle the exception by termination.

Exceptions are represented by a special variable defined at the system level called the “exception” type:

```
module throwit;  
  
var myexception: exception;  
  
procedure fault;  
  
begin  
    throw myexception  
  
end;  
  
begin { throwit }  
end. { throwit }
```

In order to throw an exception, both the thrower and the catcher must be able to see the exception variable. The exception to this rule is the unspecified exception, which is available program wide.

The exception handler itself can throw another exception, including an unspecified one. However, exceptions do not stack, nor is there any state, such as handled or unhandled, associated with exceptions.

Note:

Exceptions do not cross thread or process boundaries. A thrown exception will only return to the same thread, and only to an active outer block of that thread.

A special version of the try statement exists for module constructor sections (see 5.32 “Modularity”). It is common for modules, which only execute when the program starts up and shuts down, to register handlers for all of the exceptions that are thrown in that module. The reason is to allow the program

and other modules to handle the exceptions, but ultimately handle the exception in the originating module.

```
module m;

var FaultException: exception;

procedure doit;

begin

    throw FaultException

end;

begin { constructor }

    try start
    on FaultException except writeln('*** module fault occurred')

end; { constructor }

begin { destructor }
end. { destructor }
```

a **try start** statement can only be used in a module constructor block, and it must be the last statement in that block.

5.28 [Assert Procedure](#)

The **assert** procedure generates a runtime error if its expression is true:

```
assert-statement = 'assert' '(' expression [ character-string ] ')'
```

The expression can be any expression with a boolean result type. If the expression evaluates at runtime as true, then the program faults, otherwise it continues to run.

If a string constant is present after the exception boolean expression, it will serve to “annotate” the assert. Typically it contains the reason for the exception. It may be printed as part of any error message. Alternatively, it may have no effect.

The **assert** procedure enables placing a large number of consistency checks within a program being developed that can be automatically removed from the code by a compiler option.

In addition, an implementation can provide extended debugging information when the assert occurs, such as program location, registers, etc.

5.29 [Extended range types](#)

Standard Pascal defines the results of integer expressions to lie within the range -maxint..maxint. The size of an integer is by definition chosen to be the size that is most efficient and natural on the target

machine. Just as Pascaline defines operations to restrict results to numbers smaller than integer, it defines the ability to find results larger than a standard integer. If larger than integer variables and operations are specified, it is assumed that some penalty in terms of time, space or both will be added.

To specify an extended range value, a subrange is specified outside of the range `-maxint..maxint`. For example:

```
var li: 0..maxint*2;
```

Pascaline does not define the maximum length of such extended range types. They may be so large as to be unrepresentable.

When an extended range value appears in an expression, the range of the result will have a range according to the following rules:

1. If either type is signed, the result is a signed type.
2. If either type is extended range, the result is an extended range type.

Operations with extended types are NOT guaranteed to keep all the values of both operands. The implementation is only required to maintain at least the range of values in integer.

In addition, the programmer can specify the exact length of result at any point in the evaluation of an expression using range specifications. The only way to achieve an exact range of values throughout a calculation is to specify it.

Pascaline predefines three new types of integer:

```
cardinal
```

Cardinal types represent the positive integers only. The range of cardinal is:

```
0..maxcrd;
```

Where **maxcrd** is a predefined constant. **maxcrd** is the maximum unsigned value of a machine word on the target machine. It could also be equal to `maxint` on an implementation that does support extended cardinal types.

```
linteger
```

Long integer types represent an extended, signed integer type, defined as:

```
-maxlint..maxlint;
```

Where **maxlint** is a predefined constant. `maxlint` is typically the maximum value of a "double length" result on the target machine, that is, a value formed using two machine words. It can also be equal to `maxint` on an implementation that does not support extended linteger types.

```
lcardinal
```

Long cardinal types represent the positive integers only. The range of **lcardinal** is defined as:


```
0..maxlcrd;
```

Where **maxlcrd** is a predefined constant. maxlcrd is the maximum unsigned value of a "double length" result on the target machine, that is, a value formed using two machine words. It can also be equal to maxint on an implementation that does not support extended lcardinal types.

The interactions of binary operators on all integer based types is as follows:

TYPE A	TYPE B	RESULT TYPE
=====		
integer	integer	integer
cardinal	cardinal	cardinal
linteger	linteger	linteger
lcardinal	lcardinal	lcardinal
integer	cardinal	integer
linteger	lcardinal	linteger
linteger	cardinal	linteger
integer	lcardinal	linteger
linteger	integer	linteger
lcardinal	cardinal	lcardinal

5.30 [Character limit determination](#)

Besides the original **maxint** constant, Pascaline provides a constant for character set implementation limits:

```
maxchr
```

This character constant defines the maximum value of a character in the range:

```
chr(0) .. chr(maxchr);
```

maxchr is a numeric constant, because on most implementations it is an unprintable character, and has more meaning as a value than a character.

5.31 [Matrix mathematics](#)

One dimensional arrays of integers or reals and two dimensional arrays of integers or reals can participate in expression operations:

Operator	Function
=====	
+a	matrix affirmation
-a	Matrix negation
a+b	matrix Addition
a-b	matrix Subtraction
a*b	matrix Multiplication

```

program p;

type ai: array 10 of integer;

var ai1, ai2: ai;

begin

    ai1 := ai1+ai2;
    ai1 := ai1+1;

end.

```

Any single dimension array of integers or reals with an integer starting index of 1 qualifies as a vector. Any two dimensional array of integers or reals with both integer starting indexes of 1 qualifies as a matrix. The allowable operands are:

Operator	Operands
a+b	a or b may be matrix, integer or real.
a-b	a or b may be matrix, integer or real.
a*b	a or b may be matrix, integer or real.

For two vector or matrix operands to be compatible with each other, they must both be of the same type, vector or matrix and both integer or real, and have the same number of elements and dimensions. The result of a vector or matrix operation is a vector or matrix of the same type and size as the operand(s).

The operands of * (multiply) must be either single dimension or square 2 dimensional matrices. This means that 2 dimensional matrices must have an equal number of indices in x and y.

One of the operands of +, -, or * can be an integer or real when the other side is a matrix. The result of this is a vector or matrix with the given operation performed on each element of the matrix using the single real or integer. However, a single real can not be used with a integer matrix, as this would imply conversion of real to integer.

Matrices can also be assigned as a whole:

```

program p;

type ai: array 10 of integer;

var ai1: ai;

begin

    ai1 := 1

end.

```

Such a matrix assignment causes the expression to be assigned to each of the elements of the matrix. The assignment compatibility rules are the same as non-matrix assignment. If individual elements of the matrix are assignment compatible with the expression, then the entire matrix is also assignment compatible.

Matix expression operators do not perform a function that could not be added to Pascaline as a library function. Their primary reason for being here is that such operations can often be translated to efficient operations directly in the target hardware.

5.32 Modularity

module = module-heading ';' module-block '.' .

module-heading = module-type identifier ['(' module-parameter-list ')'] .

module-type = 'program' | 'module' | 'process' | 'monitor' | 'share' .

module-block = uses-declaration-part { module-declaration } ['private' { module-declaration }]
statement-part [';' statement-part] .

module-declaration = declaration | class-declaration .

uses-declaration-part = [link-specification module-name { ',' module-name }] .

link-specification = 'uses' | 'joins' .

module-name = identifier .

ISO 7185 Pascal expresses programs as a series of nested blocks, from programs to procedures and functions. Unfortunately, this model tends to require compilation as a single unit.

Pascaline defines a series of modules which have the same status and level as a program block, but exist in separate files. The basic module appears as:

```
module x(input, output);

uses mymodule;

var x: integer;

private

var y: integer;

begin
end.
```

A module appears very much like a program block. It exists in a separate file. It can have header parameters. It has a main block. It also has an "ending" block. Unlike the program block, Modules are not typically designed to execute for the duration of the program, but rather to run at some time before the program block. A module initializes its variables, then passes control on to either other modules, or

the program block. If the module has a termination block, this will be executed some time after the program block completes. This allows the module to perform clean up on termination, such as closing files, etc.

Modules allow a collection of resources to be created, such as labels, constants, types, variables, fixed, procedures, and functions. These are collected as a unit to serve the program. A program block can be thought of as a special case of a module that has no termination block.

The bonding of modules and programs into an executable unit, as well as the order in which the startup blocks and termination blocks are executed is implementation defined.

The use of a module by another module or program is specified by the **uses** or **joins** declaration:

```
uses mod1, mod2;
```

or

```
joins mod1, mod2;
```

The **uses** and **joins** specifiers must precede any other declaration within a program or module. There can be only one uses and one join specifier per program or module.

Both a **uses** and a **joins** specification cause the declarations contained within the specified module to be imported to the outer block declarations of the importing module or program. This importation is logically done as if the source file itself were actually read. However, the equivalent of this action may be performed instead by reading a preprocessed equivalent of the target program or module.

A **uses** or **joins** specification only imports that specific module. If the imported module itself imports further modules, which it may require to complete its declarations, any such import must occur without exposing that secondary module to the original importing module. This is a so-called "incidental import", and necessary. Such imports will be completed such that their declarations are available to the requesting module, but not the original importing module. This can create the situation that the importing module receives declarations of which it cannot access the component declarations within. If the importing module must access such components, it must specify the containing module in a **uses** or **joins** specification. This prevents "zipper effect", where a importing module gets an unintended series of submodules included.

Imported and importing modules can contain loops or mutually referring modules. However, the ISO 7185 Pascal rule remains that an identifier must be defined before its use, with the exception of pointers. Further, pointer declarations must complete within the same module. Import loops can create non-obvious definition problems:

```
module x;

uses y;

type q = integer;

begin
end.

module y;

uses x;

type r = q;

begin
end.
```

Will result in an undefined declaration error because module **x** has not completed the declaration of **q** at the time module **y** is processed.

Imports obey the rule that no module is read twice. This prevents the continuous processing of modules.

The appearance of a **uses** specification causes the identifiers within the imported module to be merged without qualification to the importing module or programs name space. The identifiers can be directly used, and an error will be flagged if there is a duplicate name between importing and imported modules.

The appearance of a **joins** specification causes the identifiers to be made available in "qualified" form. A qualification appears as follows:

qualified-identifier = module-name { '.' identifier }

For example:

mod1.alpha

Qualification prevents the collision of name spaces and is the preferred method of importing modules.

Notes:

1. Without a **uses** or **joins** specification, modules and programs will not conflict even if there are one or more names that are identical between them.
2. Because of the circular nature of imports, it is possible to refer to the components of a module before it has executed its start block. This is an error that may or may not be caught at runtime.
3. It is an error to goto a procedure or function that is not active. This is an error that may or may not be caught at runtime.

Pascaline provides a way to create declarations in a module that are not visible by importing modules or programs with the **private** specification:

```
module x;  
  
type alpha = array 10 of char;  
  
private  
  
var x: alpha;  
  
begin  
end.
```

The **private** word-symbol can appear anywhere between declarations. It indicates that all further declarations in a module cannot be exported. The appearance of the **private** word-symbol essentially divides the outer declarations of a module into a "public" section that appears first, and a private section that follows. The declarations for a module can thus be set up as a set of public declarations that define the external interface for the module, and a set of private declarations that perform the work of defining the functionality of the module.

The public and private areas cannot be mixed or changed in order. Specifically, it is not possible to create a public definition that is built with private declarations in Pascaline (so called "opaque" types). However, the converse is possible, private declarations based on public declarations.

Procedures and functions can be forwarded across from the public area to the private area. Thus, it is possible to create procedures and functions that use private declarations such as variables and other procedures and functions.

Pascaline does not enforce the structure of intermodule links, nor is this within the power of a language. However, the most reliable structure for modules is a tree with the program module at the top, and successive layers of service and support routines below that. This kind of structure necessarily excludes loops, the allowance of which was required to enable intermodule gotos to process exceptions. The modern means to do that is with exceptions.

Note that implementations may optionally flag module loops as errors.

5.32.1 Definition vs. implementation modules

Pascaline does not define a special definition as opposed to an implementation module. An implementation module, a module containing all of the source code required to define the contents of the module, also serves as its definition module.

If it is required to construct a module containing just the definitions within a module, and not the implementation source, the common method used is to "strip" the module to arrive at a defacto definition module. Stripping means to remove all definitions that are private, and to remove the contents of any public procedure or function. This will give a module that can be used to define the interface to the module for other modules, but cannot be used to build the module. It is typically used in conjunction with modules that have already been completely processed to object code in the target processor.

Such definition modules can be manually or automatically constructed by the implementation. Definition modules can be verified by compilation with sufficient options to ignore unreferenced definitions, and lack of function result assignments.

5.32.2 Overrides

procedure-heading = attribute 'procedure' identifier [formal-parameter-list].

function-heading = attribute 'function' identifier [formal-parameter-list] ':' result-type.

attribute = 'overload' | 'static' | 'virtual' | 'override' | 'operator' .

A function or procedure can be specified as capable of being overridden by higher level modules (or even the same module). In order to be overridable, the procedure or function must specify that capability using the **virtual** word-symbol:

```
module m;  
  
virtual procedure x;  
  
begin  
  
end;  
  
begin  
end.
```

When specifying a procedure or function as virtual, the user should realize that there is a small cost in program run time, program space, or both to handle the requirements of a virtual procedure or function.

To override the procedure or function, the word-symbol **override** is used:

```
module m;  
  
override procedure x;  
  
begin  
end;  
  
begin  
end.
```

In many cases, a procedure or function that overrides another will extend or change it by handling the new features of the procedure or function itself, but then send the unchanged part of the service back to the overridden procedure or function. This can be done via the **inherited** word-symbol:

```
module m;  
  
  override procedure x(q: integer);  
  
  begin  
    if q = 1 then inherited x(q) else { perform here }  
  
  end;  
  
begin  
end.
```

The **inherited** word-symbol specifies that the following procedure or function call is to be sent back to the original overridden version.

If multiple overrides of a function or procedure exist, they will nest. The procedure or function that is actually performed is the last override that was performed. The function or procedure that is performed when inherited is specified is the function or procedure that was current at the time of the override.

Only one override for the same procedure or function can exist in a module. The inherited version of a procedure or function can only be called from the module that overrides it.

The order in which overrides are executed is the same as the order of module or object initializations.

Override procedures and functions can only exist at the outer block of a module. They may not be nested.

The virtual procedure or function and its overrider must be congruous.

No overloads can exist to a **virtual** or **override** procedure or function.

5.32.3 Parallel modules

A module in Pascaline forms the basic structure of a program. The **program** block itself is actually a special instance of a module which has no ending block. Together, program modules and common modules form a group that runs the main task of the program.

The **process** module appears just as a **program** module. It has only a starting code block, and no ending code block. It can accept header parameters. The difference between a process and a program is that the process defines a new task within the program that runs in parallel with the main task, or any other process blocks.


```
process mythread(input, output);  
  
uses mymonitor;  
  
var x, y;  
  
procedure q;  
  
begin  
end;  
  
begin  
    while true do { process statements }  
  
end.
```

A process can run "forever", in which case it simply terminates when the main task terminates, or it can run for a time and then stop. A process stops when it exits from its startup block.

A process can have a **uses** statement, but it cannot use just any module. To do so would be to conflict with the main task run by the program module. No other can use a process module. A process can use its own globals or routines, and can use a **monitor**.

A monitor module appears just as normal, program callable module:

```
monitor m(output);

procedure locked; forward;

private

var x, y;

procedure locked;

begin

    x := 1

end;

begin

    { startup statements }

end;

begin

    { shutdown statements }

end.
```

The monitor module is a multitask "hard" module. Each of its publicly callable procedures and functions have a special locking function that is executed on entry to the routine, and unlocked on exit. The net effect is that only one task running under Pascaline is allowed within a monitor at one time. This prevents the data corruption that would otherwise occur in a multitasking system.

Monitors have other special requirements that prevent data corruption. A monitor cannot have global variable definitions. All global variables must be defined within the private section. Procedures and functions can be defined in the global section, which means that they are multitask locked procedures and functions, but to accomplish useful work, they must typically be forwarded into the private section.

Monitors have a further special restriction on global procedures and functions that none of the parameters can be, or have as a component, any pointer. This is required because otherwise a monitor could import or export a direct reference to a monitor variable, or pass a reference from one task to another, and bypass the monitor protection system.

Finally, monitors cannot use **var** or call by reference for any call to another, external procedure or function. This is, again, to protect the locking of a monitor. Monitors can have a public procedure or function with **var** parameters. This means that a monitor routine can access the variables of its calling task, but cannot export such references, nor export the monitors own data in a **var** passed variable.

Monitors can use other monitors, but cannot use a **process**, **program** or **module**. However there is a final module form that can be used by any other module, all of process, program, module and monitor, known as a **share** module:

```
share mylibrary(output);
```

```
procedure x;
```

```
begin  
end;
```

```
begin  
end.
```

Share modules cannot have any global variables at all. Because of this, it also has no startup or shutdown code blocks. There is nothing in a **share** to set up or to shut down.

The advantage to a share is that it can contain a series of support routines that can be accessed by any caller. A share acts as a monitor without any global data, but does not have the locking overhead of a monitor.

5.32.4 Monitor signaling

Monitors serve as more than just a library of procedures and functions callable by any task. Because a module has state in the form of globals, it can serve as a communications method between tasks. In Pascaline, the **monitor** is how multiple tasks coordinate their actions.

Given just the definition of monitors above, it would be possible for tasks to communicate by using status routines to flag conditions between them. However, each task would have to poll, or continually call a monitor function to find out that status. This wastes computer time that multitasking is supposed to use efficiently.

Instead, Pascaline defines a special variable called a **semaphore**, and a few special routines that can be used with them, the **signal**, **signalone**, and **wait** procedures.

```
monitor queue;

type byte = 0..255; { data type for queue }

procedure inqueue(b: byte); forward;
procedure outqueue(var b: byte); forward;

private

const maxque = 100; { maximum length of queue }

type queinx = 1..maxque; { pointers for queue }

var fifo:      array [queinx] of byte; { fifo for queue }
    inptr:     queinx;      { in pointer }
    outptr:    queinx;      { out pointer }
    notempty:  semaphore; { not empty signal }
    notfull:   semaphore; { not full signal }

{ queue pointer iterator }

function next(i: queinx): queinx;

begin
    if i = maxque then i := 1 { queue has wrapped }
    else i := i+1; { next location }

    next := i { return result }

end;

{ test queue is full }

function empty: boolean;

begin
    empty := inptr = outptr { pointers are equal }

end;

{ test queue is empty }

function full: boolean;

begin
    full := next(inptr) = outptr { next input location is out
                                  location }
```

```
end;

{ place byte in queue }

procedure inqueue(b: byte);

begin

    { if full, wait until a byte clears }
    while full do wait(notfull);

    { place input byte }
    fifo[inptr] := b;

    { set next input location }
    inptr := next(inptr);

    { signal queue is now not empty }
    signal(notempty)

end;

{ get byte from queue }

procedure outqueue(var b: byte);

begin

    { if empty, wait until a byte is available }
    while empty do wait(notempty);

    { get output byte }
    b := fifo[outptr];

    { set next output location }
    outptr := next(outptr);

    { signal queue is now not full }
    signal(notfull)

end;

begin { constructor }

    { set input = output, and queue is empty }
    inptr = 1;
    outptr = 1

end.
```

The **wait** procedure acts just as a loop to poll if the condition represented by the semaphore has occurred, but it is implemented efficiently under Pascaline. Typically, the calling task is put to sleep until the signal occurs, then it is woken up again to continue.

When a **signal** call is made, this causes any and all tasks waiting on the signal to be freed to run. If the signaling task knows that only one task can actually use the signal, it can use the **signalone** call instead. This will only free up a single task, and leave the rest to wait for the next signal.

Semaphores give tasks an easy method to flag a given event between them. They are most commonly used to pass data between tasks. A "data provider" task would place data into global structures in the monitor, then signal to any "data consumer" tasks that the monitor has data.

Semaphores can only be used within a monitor, which is the only place they have meaning. A **wait** call would not be useful if it left the monitor lock in effect, blocking any other task from signaling to it. The wait call releases the lock on the monitor that calls it until it is signaled and emerges from the wait. Then it reasserts the lock and continues.

Semaphores are implemented under Pascaline as "fair", or having the property that tasks that wait for a semaphore receive the signal on a "first come-first serve" basis. However, if the **signal** procedure is used to signal multiple tasks (as opposed to a **signalone** call), it is possible that one or more of the signaled tasks could find that the condition being awaited is no longer true, since another task has serviced it first. For this reason, wait is typically used in a loop that checks for the condition being true.

```
while boolean-expression do wait(sig);
```

5.33 Classes

```
class-declaration = 'class' identifier ' [ class-parameter-list ] ;' [ 'extends' class-name ';' ] block '.'.
```

```
class-parameter-list = '(' class-parameter { ';' class-parameter } ') '.
```

```
class-parameter = [ 'view' ] identifier-list ';' type-identifier .
```

```
class-name = qualified-identifier .
```

```
block = { declaration } [ 'private' { declaration } ] statement-part [ ';' statement-part ] .
```

```
declaration = label-declaration-part | constant-definition-part | type-definition-part | variable-  
              declaration-part | fixed-declaration-part | procedure-declaration ';' | function-declaration  
              ';' .
```

Modules in Pascaline provide a way to package types, variables, procedures, functions and other definitions together as a unit to be used by other parts of the program. Classes extend this further by providing a general description of a module that the program can create any number of instances of dynamically.

In the Pascaline model, modules are both a description of a class, and an instance of it.

Classes fit in the program hierarchy between modules and procedures/functions:

```
Modules
  |
  \ /
Classes
  |
  \ /
Procedures/functions
```

The class is a collection of types, variables and procedures called "members", and it has per-instance data.

A class is declared by the word-symbol "class", and it can appear anywhere in the declarations of a global module such as a program or module, etc.

```
program p(input, output);

uses mylib;

class myclass;

label 1, 2;

type a = char;

var c, d: integer;

procedure x;

begin

end;

private

procedure y;

begin

    { optional constructor }

end;

begin

    { optional destructor }

end.

begin { program }
end.
```

When a procedure or function appears in a class, it is referred to as a method. A class can refer to any of the declarations in the surrounding module. The methods of a class can refer to both the variables inside the class as well as the variables in the module that surrounds it.

As a module can, a class can use the **private** word-symbol to specify declarations within the class that are private.

A class does not create any actual variables, procedures and functions when it is defined. A class is a template for a type. To create an “object” from a class, it must be instantiated, which means to give the class a complete set of locations in memory. If that memory is static, the instance is static. If the memory is allocated dynamically, the instance is dynamic.

5.33.1 Static objects

To create a static instance, the class is used to form a type as follows:

```
type si = instance of c;
```

Which can be used to form static instances of the class:

```
var z: si;
```

The class **c** is not itself a type, nor can it be used as such. **si**, however, is a type. To use a class as a type, the compiler must know if a static or dynamic instance is meant.

A static instance is static even if it is a local of a procedure or function, or instantiated within another class. An instance does not become dynamic unless it is allocated dynamically via **new**.

5.33.2 Dynamic objects

To create a dynamic instance, the class is used to form a type as follows:

```
type di = reference to c;
```

Which can be used to form a dynamic reference to objects of the class:

```
var z: reference to c;
```

Where **c** is a class. This creates a special variable used to access the "object" that "instantiates" the class. The actual creation of the object is done by the standard procedures **new** and **dispose**:

```
new(z); { create new object, referred to by z }
```

```
dispose(z); { destroy existing object referred to by z }
```

The definitions for the object referenced by **z** can be accessed by the same notation as for a **joins** module:

```
z.c := 1; { assign value to object variable }  
z.x; { call object procedure x }
```


A reference variable can point to any object defined with a compatible class. It can also point to no object at all:

```
z := nil;
```

Just as a pointer variable can. Further, two references can be compared, or as-signed, just as pointers can:

```
program p;  
  
class c;  
  
begin  
end.  
  
var z, q: reference to c;  
  
begin  
  
    z := nil; q := z;  
  
    if z = nil then { perform actions on nil }  
    if q = z then { perform actions on q = z }  
  
end.
```

In fact, a reference behaves just as a pointer, but without the need to specify a dereference ("^"). Anytime a qualifier (".") follows a reference, it is understood that the object referenced is being accessed.

Just as for pointers, a reference can appear in a **with** statement:

```
program p;  
  
class c;  
  
begin  
end.  
  
var z: reference to c;  
  
begin  
  
    with z do { statements }  
  
end.
```

This effectively opens up the scope of the object within the code.

References are treated as pointers with respect to monitors. A reference cannot be exported by a monitor routine.

5.33.3 Class parameters

Classes accept parameters that are a subset of procedure and function parameters. Only value and **view** parameters are accepted in a class parameter list.

```
program p

class a(la: integer; b: char);

var n(la): array of integer;
    i:      integer;

procedure addup;

var i: integer;

begin

    a := 0;
    for i := 1 to la do a := a+n[i]+ord(b)

end;

begin { constructor }

    for i := 1 to la do n[i] := 0;

end.

begin
end.
```

Class parameters are special in that they are specified when the object formed from the class template is instantiated. The class parameters are evaluated, and kept as long as the object exists. The constructor, the destructor, and all of the methods of the class can access the class parameters.

```

program p

class a(la: integer; b: char);

.

var n(10, 'c'): instance of a;
    r: reference to a;

begin

    new(r, 20, 'g');

end.

```

Note that the actual parameter list for the class appears where it is instantiated. For static instances, this is the variable declaration for it. For dynamic instances, it is the **new** statement that creates the object.

Typically, class parameters perform a “geometry” change within the object. That is what makes a class parameter different from just applying a method to it after its constructor executes. A method could not specify the creation of the parameterized array **n** above. However, class parameters have many uses, and be used to specify any class configuration parameter.

5.33.4 Inheritance

class-declaration = 'class' identifier ' [class-parameter-list] ;' ['extends' class-name ';'] block '.' .

expression = simple-expression [relational-operator simple-expression] .

relational-operator = '=' | '<>' | '<' | '>' | '<=' | '>=' | 'in' | 'is' .

Classes can "inherit" the definitions contained in other classes:

```

program p;

class x;

begin
end.

class y;

extends x;

begin
end.

begin { program main block }
end.

```

The **extends** word-symbol is followed by the class that is inherited.

The meaning of this is that the new class receives all of the definitions present in the inherited class, and can then add its own. Objects have the special property that all references to a class are compatible with any class that is built by inheriting definitions from its reference class. Because class references can be freely assigned between compatible classes, this means that references are effectively compatible with the classes that it inherits, as well. This means that it is possible for some part of the definitions for a referenced class to be non-existent. Pascaline can detect and flag an error for an access to a missing definition in a referenced object either at compile time or runtime.

When a program needs to determine if the definitions it needs to access in an object exist can use a special expression operator:

```
is-operator = expression 'is' class-name
```

The **is** operator is true if the referenced object either is the indicated class, or is a class that inherits from that class. In either case, it means that all of the definitions from the class defined as the base of the reference exist.

5.33.5 Inherited class parameters

Derived classes can access all of the parameters that appear in the base class. They can also add new parameters:

```
program p;  
  
class x(i: integer);  
  
begin  
end.  
  
class y(b: char);  
  
extends x;  
  
begin  
end.  
  
begin { program main block }  
end.
```

Here, class **x** has one parameter, **i**, and class **y** has two parameters, **i** and **b**.

When the class is instantiated, the parameter list given in a parameterized variable or new construct must have all of the parameters given for all the base classes of a derived class, from the ultimate base class to the ultimate derived class, in left to right order:

```
program p;  
  
class x(i: integer);  
  
.  
  
class y(b: char);  
  
extends x;  
  
.  
  
var cr: reference to y;  
    ci(10, 'c'): instance of y  
  
begin { program main block }  
  
    new(cr, 20, 'u');  
  
end.
```

5.33.6 Overrides for objects

Just as static modules can, classes can override inherited procedures and functions.

```
program p;  
  
class x;  
  
virtual procedure y;  
  
begin  
  
end;  
  
begin { constructor for x }  
end.  
  
class z;  
  
extends x;  
  
override procedure y;  
  
begin  
end;  
  
begin { constructor for z }  
end.  
  
begin { main block for p }  
end.
```

The main difference for the use of overrides with objects is that the instances of the original object keep their original procedures and functions, and only derived classes have the procedure or function replaced. Static modules have the original procedure or function replaced for all callers.

Just as for a module, the original definition of a procedure or function can be accessed by using the **inherited** word-symbol:

```
inherited y; { call the original definition of member procedure y }
```

Only one override can exist for the same method in a object. The inherited version of a method can only be accessed within the object where the override exists.

5.33.7 Self referencing

Within the code that makes up the object, its procedures, functions and start and exit blocks, referring to definitions within the object is done as for any module, dynamic or static. However, an object frequently needs to access the reference that defines access to it, as well. This is most common in the case where the object exists in a list of objects, and is done via the word-symbol **self**:

```
program p;  
  
class list;  
  
var next: reference to list;  
  
procedure next;  
  
begin  
  
    if self <> nil do  
        self := next { next list item }  
  
end;  
  
begin { constructor for list }  
end.  
  
begin { main block }  
end.
```

self stands in for the reference that the object being operated on is based from.

5.33.8 Constructors and destructors

If a constructor or constructor and destructor exists for a class, they are executed when the object is created by **new**, or at the start of the object's defining block, and when the object is disposed of by **dispose**, or at the end of the object's defining block. The constructor is executed on a **new**, and the destructor is executed on "dispose". Just as for a static module, constructors give an opportunity to set up and tear down an object.

When an object has inherited classes, it has a set of constructors and destructors from all of the inherited classes. These constructors are executed in the order of inheritance on **new**, that is from base class to derived class. The destructors are executed in the opposite order on **dispose**. This gives each base class the ability to set itself up first, then when the inherited class has its constructor executed, the inherited class is fully initialized.

When an object is torn down, the derived class has the ability to deactivate itself before the class it is based on is deactivated.

A Annex A: Collected syntax

actual-parameter = expression | variable-access | procedure-identifier | function-identifier .

actual-parameter-list = '(' actual-parameter { ',' actual-parameter } ')' .

adding-operator = '+' | '-' | 'or' | 'xor' .

apostrophe-image = "'" .

array-type = 'array' [dimension-specifier] 'of' component-type .

array-value-constructor = 'array' value-constructor { ',' value-constructor } 'end' .

array-variable = variable-access .

assignment-statement = (variable-access | function-identifier) ':=' expression .

attribute = 'overload' | 'static' | 'virtual' | 'override' | 'operator' .

base-type = ordinal-type .

binary-digit = '0' | '1' | '_' .

binary-digit-sequence = octal-digit { binary-digit }

binary-integer = '%' binary-digit-sequence .

block = { declaration } ['private' { declaration }] statement-part [';' statement-part] .

Boolean-expression = expression .

buffer-variable = file-variable "'" .

case-constant = constant .

case-constant-range = case-constant ['..' case-constant] .

case-constant-list = case-constant-range { ',' case-constant-range } .

case-index = expression .

case-list-element = case-list-statement | case-list-default .

case-list-default = 'else' statement .

case-list-statement = case-constant-list ':' statement .

case-statement = 'case' case-index 'of' case-list-element { ';' case-list-element } [';'] 'end' .

character-string = "" string-element { string-element } "" .

class-declaration = 'class' identifier ' [class-parameter-list] ;' ['extends' class-name ';'] block '.'.

class-name = qualified-identifier .

class-parameter-list = '(' class-parameter { ';' class-parameter } ')' .

class-parameter = ['view'] identifier-list ';' type-identifier .

component-type = type-denoter .

component-variable = indexed-variable | field-designator .

compound-statement = 'begin' statement-sequence ['result' expression] 'end' .

conditional-statement = if-statement | case-statement | try-statement .

constant = [sign] constant-term { adding-operator constant-term } .

constant-definition = identifier '=' constant .

constant-definition-part = ['const' constant-definition ';' { constant-definition ';' }] .

constant-factor = '(' constant ')' | 'not' constant-factor | character-string | constant-identifier | unsigned-integer .

constant-identifier = qualified-identifier .

constant-term = constant-factor { multiplying-operator constant-factor } .

control-variable = entire-variable .

decimal-integer = digit-sequence .

declaration = label-declaration-part | constant-definition-part | type-definition-part | variable-declaration-part | fixed-declaration-part | procedure-declaration ';' | function-declaration ';' .

digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' .

digit-sequence = digit { digit | '_' } .

dimension-specifier = index-specifier | range-specifier .

directive = letter { letter | digit } .

domain-type = type-identifier .

else-part = 'else' statement .

empty-statement = .

entire-variable = variable-identifier | 'self' .

enumerated-type = '(' identifier-list ')' .

except-series = except-clause { except-specifier } [except-unconditional] .

except-specifier = 'on' exception-identifier 'except' statement .

except-unconditional = 'except' statement .

expression = simple-expression [relational-operator simple-expression] .

factor = variable-access | unsigned-constant | function-designator | type-identifier '(' expression ')' | set-constructor | '(' expression ')' | 'not' factor .

field-designator = record-variable '.' field-specifier | field-designator-identifier .

field-designator-identifier = qualified-identifier .

field-identifier = qualified-identifier .

field-list = [(fixed-part [';' variant-part] | variant-part) [';']] .

field-specifier = field-identifier .

file-type = 'file' 'of' component-type .

file-variable = variable-access .

final-value = expression .

fixed-declaration = identifier-list ':' type-denoter '=' value-constructor .

fixed-declaration-part = ['fixed' fixed-declaration ';' { fixed-declaration ';' }] .

fixed-part = record-section { ';' record-section } .

for-statement = 'for' control-variable ':' initial-value ('to' | 'downto') final-value 'do' statement .

formal-identifier = identifier | 'not' | '+' | '-' | '*' | '/' | 'div' | 'mod' | 'and' | 'or' | '<' | '>' | '=' | '<=' | '>=' | 'in' | 'is' | ':=' .

formal-parameter-list = '(' formal-parameter-section { ';' formal-parameter-section } ')' .

formal-parameter-section = value-parameter-specification | variable-parameter-specification | view-parameter-specification | procedural-parameter-specification | functional-parameter-specification .

fractional-part = digit-sequence .

function-block = block .

function-declaration = function-heading ';' directive | function-identification ';' function-block |
function-heading ';' function-block .

function-designator = function-identifier [actual-parameter-list] .

function-heading = attribute 'function' formal-identifier [formal-parameter-list] ':' result-type .

function-identification = attribute 'function' identifier .

function-identifier = qualified-identifier .

functional-parameter-specification = function-heading .

goto-statement = 'goto' label .

hex-digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9' | 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | '_' .

hex-digit-sequence = hex-digit { hex-digit } .

hex-integer = '\$' hex-digit-sequence .

identified-variable = pointer-variable "" .

identifier = letter | '_' { letter | digit | '_' } .

identifier-list = identifier { ',' identifier } .

if-statement = 'if' Boolean-expression 'then' statement [else-part] .

index-expression = expression .

index-specifier = '[' index-type { ',' index-type } ']' .

index-type = ordinal-type .

indexed-variable = array-variable '[' index-expression { ',' index-expression } ']' .

initial-value = expression.

instance-type = 'instance' 'of' class-name .

label = digit-sequence | identifier .

label-declaration-part = ['label' label { ',' label } ';'] .

letter = 'a' | 'b' | 'c' | 'd' | 'e' | 'f' | 'g' | 'h' | 'i' | 'j' | 'k' | 'l' | 'm' | 'n' | 'o' | 'p' | 'q' | 'r' | 's' | 't' | 'u' | 'v' | 'w' | 'x' | 'y' |
'z' | 'A' | 'B' | 'C' | 'D' | 'E' | 'F' | 'G' | 'H' | 'I' | 'J' | 'K' | 'L' | 'M' | 'N' | 'O' | 'P' | 'Q' | 'R' | 'S' | 'T' | 'U' | 'V' | 'W' |
'X' | 'Y' | 'Z' .

link-specification = 'uses' | 'joins' .

member-designator = expression ['..' expression] .

module = module-heading ';' module-block '.' .

module-block = uses-declaration-part { module-declaration } ['private' { module-declaration }]
statement-part [';' statement-part] .

module-declaration = declaration | class-declaration .

module-heading = module-type identifier ['(' module-parameter-list ')'] .

module-name = identifier .

module-parameter-list = identifier-list .

module-type = 'program' | 'module' | 'process' | 'monitor' | 'share' .

multiplying-operator = '*' | '/' | 'div' | 'mod' | 'and' .

new-ordinal-type = enumerated-type | subrange-type .

new-pointer-type = '^' domain-type .

new-structured-type = ['packed'] unpacked-structured-type .

new-type = new-ordinal-type | new-structured-type | new-pointer-type .

octal-digit = '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '_' .

octal-digit-sequence = octal-digit { octal-digit } .

octal-integer = '&' octal-digit-sequence .

ordinal-type = new-ordinal-type | ordinal-type-identifier .

ordinal-type-identifier = type-identifier .

pointer-type = new-pointer-type | pointer-type-identifier .

pointer-type-identifier = type-identifier .

pointer-variable = variable-access .

procedural-parameter-specification = procedure-heading .

procedure-and-function-declaration-part = { (procedure-declaration | function-declaration) ';' } .

procedure-block = block .

procedure-declaration = procedure-heading ';' directive | procedure-identification ';' procedure-block |
procedure-heading ';' procedure-block .

procedure-heading = attribute 'procedure' formal-identifier [formal-parameter-list] .

procedure-identification = attribute 'procedure' identifier .

procedure-identifier = qualified-identifier .

procedure-statement = procedure-identifier ([actual-parameter-list] | read-parameter-list | readln-parameter-list | write-parameter-list | writeln-parameter-list) .

qualified-identifier = identifier { '.' identifier } .

range-specifier = unsigned-integer { ',' unsigned-integer } .

read-parameter-list = '(' [file-variable ','] variable-access { ',' variable-access } ')' .

readln-parameter-list = ['(' (file-variable | variable-access) { ',' variable-access } ')'] .

real-type-identifier = type-identifier .

record-section = identifier-list ':' type-denoter .

record-type = 'record' field-list 'end' .

record-value-constructor = 'record' value-constructor { ',' value-constructor } 'end' .

record-variable = variable-access .

record-variable-list = record-variable { ',' record-variable } .

reference-type = 'reference' 'to' class-name .

relational-operator = '=' | '<>' | '<' | '>' | '<=' | '>=' | 'in' | 'is' .

repeat-statement = 'repeat' statement-sequence 'until' Boolean-expression .

repetitive-statement = repeat-statement | while-statement | for-statement .

result-type = simple-type-identifier | pointer-type-identifier .

scale-factor = [sign] digit-sequence .

set-constructor = '[' [member-designator { ',' member-designator }] ']' .

set-type = 'set' 'of' base-type .

sign = '+' | '-' .

signed-integer = [sign] unsigned-integer .

signed-number = signed-integer | signed-real .

signed-real = [sign] unsigned-real .

simple-expression = [sign] term { adding-operator term } .

simple-statement = empty-statement | assignment-statement | procedure-statement | goto-statement .

simple-type = ordinal-type | real-type-identifier .

simple-type-identifier = type-identifier .

special-symbol = '+' | '-' | '*' | '/' | '=' | '<' | '>' | '[' | ']' | ':' | ';' | '^' | '(' | ')' | '<>' | '<=' | '>=' | ':=' | '..' | '('
| '.' | '@' | word-symbol .

statement = [label ':'] (simple-statement | structured-statement) .

statement-part = compound-statement .

statement-sequence = statement { ';' statement } .

string-character = one-of-a-set-of-implementation-defined-characters .

string-element = apostrophe-image | string-character .

structured-statement = compound-statement | conditional-statement | repetitive-statement | with-statement .

structured-type = new-structured-type | structured-type-identifier .

structured-type-identifier = type-identifier .

subrange-type = constant '..' constant .

tag-field = identifier .

tag-type = ordinal-type-identifier .

term = factor { multiplying-operator factor } .

try-statement = 'try' (statement-sequence | 'start') (except-series | except-unconditional) ['else'
statement] .

type-definition = identifier '=' type-denoter .

type-definition-part = ['type' type-definition ';' { type-definition ';' }] .

type-denoter = type-identifier | new-type .

type-identifier = qualified-identifier .

unpacked-structured-type = array-type | record-type | set-type | file-type | reference-type | instance-type
.

unsigned-constant = unsigned-number | character-string | constant-identifier | 'nil' .

unsigned-integer = decimal-integer | hex-integer | octal-integer | binary-integer .

unsigned-number = unsigned-integer | unsigned-real .

unsigned-real = digit-sequence '.' fractional-part ['e' scale-factor] | digit-sequence 'e' scale-factor .

uses-declaration-part = [link-specification module-name { ',' module-name }] .

value-constructor = array-value-constructor | record-value-constructor | constant .

value-parameter-specification = identifier-list ':' type-identifier .

variable-access = entire-variable | component-variable | identified-variable | buffer-variable .

variable-declaration = identifier-list ['(' expression { ',' expression ')' } ':' type-denoter .

variable-declaration-part = ['var' variable-declaration ';' { variable-declaration ';' }] .

variable-identifier = qualified-identifier .

variable-parameter-specification = 'var' identifier-list ':' type-identifier .

variant = case-constant-list ':' '(' field-list ')' .

variant-part = 'case' variant-selector 'of' variant { ';' variant } .

variant-selector = [tag-field ':'] tag-type .

view-parameter-specification = 'view' identifier-list ':' type-identifier .

while-statement = 'while' Boolean-expression 'do' statement .

with-statement = 'with' record-variable-list 'do' statement .

word-symbol = 'and' | 'array' | 'begin' | 'case' | 'const' | 'div' | 'do' | 'downto' | 'else' | 'end' | 'file' | 'for' |
'function' | 'goto' | 'if' | 'in' | 'label' | 'mod' | 'nil' | 'not' | 'of' | 'or' | 'packed' | 'procedure' | 'program' | 'record'
| 'repeat' | 'set' | 'then' | 'to' | 'type' | 'until' | 'var' | 'while' | 'with' | 'forward' | 'module' | 'uses' | 'private' |
'external' | 'view' | 'fixed' | 'process' | 'monitor' | 'share' | 'class' | 'is' | 'overload' | 'override' | 'reference' |
'joins' | 'static' | 'inherited' | 'self' | 'virtual' | 'try' | 'except' | 'extends' | 'on' | 'result' | 'operator' | 'instance' |
'start' .

write-parameter = expression [':' expression [':' expression]] .

write-parameter-list = '(' [file-variable ','] write-parameter { ',' write-parameter } ')' .

writeln-parameter-list = ['(' (file-variable | write-parameter) { ',' write-parameter } ')'] .

B Annex B: Overview of standard libraries and modularity

In the annexes that follow, several standard libraries are introduced. The introduction of standard libraries into a standard is based on the following two ideas:

1. Pascaline has sufficient extendibility that major new applications can be covered with extension modules using the standard language, and not changes to the base language.
2. A typical Pascaline program is going to have library dependencies in addition to language dependencies.

The modules in the standard set are divided into two basic types:

1. Basic extensions used by programs regardless of extra devices or capabilities available.
2. Extensions that introduce new device capabilities.

The following extension libraries will be shown in this standard:

Annex C:	Extension library - Directory lists, file name handling, paths, environment, program execution, date and time, internationalization.
Annex D:	String library - String routines, alternate base I/O, formatting.
Annex E:	Terminal library - Output to text surface, advanced input.
Annex F:	Graphical library - Output to graphical surface.
Annex G:	Windowing library - Management of multiple windows.
Annex H:	Widget library - Buttons, lists, dialogs relevant to screen based programs.
Annex I:	Sound library - Midi and wave input and output.
Annex J:	network library - Network program access.

The library series in this version of the Pascaline standard was designed using a previous version of Pascaline that didn't have classes and objects. The calls contained here are, therefore, entirely procedural. Object based orientation is expected to be added in a later version of this standard.

B.1 Basic Language Support

B.1.1 extlib

extlib contains a series of operating system support extensions such as filename/path handling, directory listings, time and date, environment strings, executing other programs, and similar functions.

B.1.2 strlib

Implements common string handling routines, dynamic string management, numeric conversion to and from a string, advanced numeric formatting to and from both strings and files, string search and replace, and similar functions.

B.2 Advanced User I/O and Presentation Management

The advanced user I/O modules are a family that implement a series of advancing levels that match advancing levels in I/O capabilities, including:

Terminal character presentation

Gives the ability to place characters on a grid for a typical fixed size character display.

Graphical presentation

Gives the ability to draw figures, and place characters with proportional fonts.

Window management

Divides the display surface into a series of independent windows.

Widget placement and management

Gives a library of common controls, layered components and dialogs.

All of the supported display modes are upward compatible, and fixed size character presentation is always available in any graphic mode. The result is that there is two possible stacking levels of presentation:

```
Terminal  -> Single fixed display -> Windowed display -> Widgets
Graphical -> Single fixed display -> Windowed display -> Widgets
```

The terminal, graphical, window management and widget libraries are a set of libraries that implement a series of screen surface operating standards in an ascending sequence of capability:

```
serial I/O - I/O of single characters with line orientation and
|           blocking on input.
+--Terminal I/O - I/O to an X-Y fixed font surface with arbitrary
|           positioning and event driven I/O.
+--Graphical I/O - I/O to arbitrary pixel locations and multiple
|           figure types.
+-----Multiple windows - I/O to multiple windows.
+-----Widget - Catalog of predefined widgets and dialogs
```

These layers are all forward and backward compatible, so that any lower level can be used in any higher level. For example, a standard Pascal/Pascaline program that only inputs and outputs in terms of characters formatted in terms of lines can be run under any level all the way up to a multiple windowing environment with widgets without source change.

The primary method for performing this is the use of the "character grid", which shows where the cells of screen characters exist in a graphical system. The character grid is only relevant to characters as a figure, and not to lines or other figures, and output to the grid is the default. A program use arbitrary character placement by explicitly specifying it, and it can turn off the automatic character wrapping features of the grid.

Similarly, all text and graphical windows are buffered in a multiple windowing system, so that they need not be aware of the windows management unless they wish to.

Serial I/O is the name given here of the default method of Pascal/Pascaline which goes back to its origin, and includes the write, read, writeln, readln, page and similar built in procedures.

Terminal I/O is so named after the terminals that were used with computers for several decades, and for text mode still in use even in windowed operating systems.

From here, two distinct branches exist, one with, and one without graphics capability. For example, it is possible to have both multiple windows, widgets and dialogs all while staying within the capability of a character I/O only terminal. Therefore, there exists both a graphics version of the windows management library and the windows library, and one of each with character only ability.

The names of the libraries, as appears in a `uses` or `joins` statement, is as follows:

Name	Contents	Call model
trmlib	Terminal I/O	Terminal
gralib	Graphical I/O	Graphical
manlib	Managed windowing	Terminal and graphical
widlib	Widgets and dialogs	Terminal and graphical

The serial level does not require an explicit library, since that is the normal I/O method specified in Pascal/Pascaline.

The key to understanding the variation in libraries is that a **uses** or **joins** of a particular library does not necessarily lead to a completely different block of code. A system that only has graphical output modes will implement the terminal library by aliasing that to the graphical library. It is also common for a graphical or terminal mode library to include all of the window management and widget/dialog function in the same module. Thus, a **uses** or **joins** of the windows management libraries does not cause a specific library to be linked as much as flag that the program will be using these features.

B.3 Advanced device libraries

B.3.1 sndlib

sndlib gives the ability to drive a midi output, perform sequencing, and output wave files.

B.3.2 netlib

netlib allows access to a network using the standard Pascal file model.

B.4 Library procedure and function notation

Note that for purposes of brevity, BNF notation is used to show optional constructs, and the **view** and **overload** word-symbols have been omitted.

C Annex C: Extention Library

extlib contains common operating system related tasks, including directory access, time and date, files and paths, file attributes, environment strings, the local option character, and execution of external programs.

C.1 Filenames and Paths

A file specification is composed of a path, name and extension:

<path><name><ext>

The exact format of a file specification changes with the operating system. The routine **brknam** takes a file specification and breaks it down into its path, name and extension components. The routine **maknam** does the opposite, creating a composite name from the three components. When a file specification has no path, it means that it refers to the default path. When a file specification needs to be printed or saved in an absolute format, with the path always specified, the **fulnam** routine is used to "normalize" the file specification by filling out the complete path.

To parse file specifications from the user, the routine **filchar** returns a character set of all possible characters in a filename. This can be used to get a sequence of characters from a string or file that can constitute a file specification. Once the potential file specification has been loaded into a string, it can be checked for validity as a file specification by **validfile**, and as a path by **validpath**.

Filenames are based on the idea that there is a set of characters that may be used for filenames in a particular installation, and the characters outside that set are used to delimit between filenames. This characteristic of filenames allows a program to load the filename into a string, then check it's proper structure using subsequent calls. The program may remove certain characters from the set of filename characters to be able to parse command lines.

A common method is used to represent filenames in command lines and other text when the characters allowed in a filename are essentially unlimited is to quote the filename. Using this method, the filename appears as:

“myfile”

or

‘myfile’

This can work together with limited character set filenames by recognizing the leading quote. If both types of quotes are allowed, then the leading quote should match the trailing quote. Additionally, the program should be able to recognize a “quote image” of the forms:

“”	Double quote “
‘’	Single quote ‘
\”	Forced double quote “
\’	Force single quote ‘

A robust program would recognize all of these forms.

Note that **extlib** does not contain any method to parse filenames.

If a file specification contains wildcards, this can be determined by **wild**. **extlib** does not define what wildcards characters or specifiers are used, or their format. **wild** simply indicates that the filename contains a wildcard specification that may result in multiple files being indicated by the same file specification.

C.2 Predefined paths

To find common objects that a program needs, three predefined paths are provided:

Program path

Is the path that the program was executed from. This is used to find data that accompanied the program, and system wide option files. **getpgm** is used to read the path.

User Path

This is the path for the current user's home directory. This is used to store options that only apply to the current user. **getusr** is used to read the path.

Current Path

The default path is used to find options that apply only to the current file being worked on. Unlike the other path types, the current path can be both read and set. Setting the current path will set the default path used to finish incomplete file specifications. **getcur** is used to read the path, and **setcur** is used to set it.

Note that if the the system has no concept of a current path, this property is used to form full path names from default path names in **extlib**.

C.3 Time and Date

Time is kept in two different formats in **extlib**. The first is seconds time, and the second is "clock" time. Seconds time is literally a count of the number of seconds since a fixed reference time. Clock time is a free running clock that ticks every 100 Microseconds.

Seconds time is returned by the **time** function. It is in a format called "S2000", and it is the number of seconds relative to midnight on the morning of January 1, year 2000. This means that S2000 has a negative value for years before 2000, and a positive value after that. S2000 time is dependent on the representation of an **integer**. This results in the following limits according to bit size:

Bits	Farthest year into the past	Farthest year into the future
=====	=====	=====
32	1932	2068
64	-1608591645726	1608591649726

Note that these times don't account for leap years, which are less than a year of error in 32 bits in any case. 16 bits cannot reasonably be used to represent a time, since the seconds in a year exceed the format.

This represents a reasonable range of time for 32 bits, and by the year 2068 it is likely that the time will universally be 64 bits or better. Note that it does not matter how many bits are returned by time, so that transition will occur seamlessly. Because 64 bits is already in excess of what is needed for computer based timekeeping, it is likely that time in 64 or more bits will actually contain fewer bits, despite the **linteger** representation.

S2000 is self relative, meaning that times relative to the year 2000 are measured by a fixed number of seconds. S2000 only matched UTC time once, exactly at midnight on the morning of January 1, year 2000. All times after that or before that or after that are increasingly diverge from UTC. In particular, this means that the current time and date will not match current UTC.

There are two types of calculations that can be applied to UTC, fixed and dynamic. The fixed calculation typically finds UTC by applying leap year corrections, then a fixed table of leap second years. The fixed calculation will fall out of accuracy from the time that the system is released. It is impossible for it to be otherwise, since UTC is based on astronomical observation.

The dynamic calculation relies on receiving a current table of corrections from the network or other communications method. This is typically the same calculation as fixed time, but with a continually updated table of leap seconds.

When an S2000 time is not available, it is customary to set it to **-maxlint**, which makes it clear that the time was not set.

The time returned by **time** is in GMT or "universal" time. To convert to local time, the function **local** is used. It takes the given GMT S2000 time, and offsets it by the local time zone offset, and by daylight savings time, and returns the adjusted local time.

The time can be placed, in character format, to a string by **times**, and written to either an output file, or the standard output by **writetime**. The date can be placed, in character format, to a string by **dates**, and written to either an output file, or the standard output by **writedate**.

Clock time is typically derived from a free running counter kept in the host computer, and it is represented by **lcardinal** values. It may or may not be synchronised to the values returned by time. For this reason, **clock** should be treated as self-relative and not compared to **time** in any way.

Clock time is treated differently from **time**. Since it free runs, you must be prepared for it to "wrap", or suddenly start counting up from zero. This can be determined by if any stored time is greater, or later in time, than the current clock value. The function **elapsed** takes a reference time, and determines how much time has passed from that, including compensation for wraparound. The exact count at which the **clock** count wraps is system dependent.

The total amount of time that can be represented with **clock** is determined not only by the bit size of the **clock** return value, but also by the size of the counter the host computer maintains. All that is guaranteed is that clock will be able to keep a unique time for at least 24 hours.

The actual increment of time for each tick of the clock is determined by the host computer. If the host cannot time to 100 microsecond accuracy, then the **clock** time will increment in multiples > 1, or effectively a running approximation of the actual timer. For this reason, there may not be an exact time length between successive counts of the timer.

C.4 Directory Structures

The **list** procedure takes a file specification, including wildcards, and returns a linked list of all of the matching directory entries:

```
{ attributes }

attribute = (atexec,  { is an executable file type }
            atarc,   { has been archived since last modification }
            atsys,   { is a system special file }
            atdir,   { is a directory special file }
            atloop); { contains heriarchy loop }

attrset = set of attribute; { attributes in a set }

{ permissions }

permission = (pmread,  { may be read }
             pmwrite,  { may be written }
             pmexec,   { may be executed }
             pmdel,    { may be deleted }
             pmvis,    { may be seen in directory listings }
             pmcopy,   { may be copied }
             pmren);   { may be renamed/moved }

permset = set of permission; { permissions in a set }

{ standard directory format }

filptr = ^filrec; { pointer to file records }

filrec = record

    name:  pstring;  { name of file }
    size:  lcardinal; { size of file }
    alloc: lcardinal; { allocation of file }
    attr:  attrset;   { attributes }
    create: linteger; { time of creation }
    modify: linteger; { time of last modification }
    access: linteger; { time of last access }
    backup: linteger; { time of last backup }
    user:  permset;   { user permissions }
    group: permset;   { group permissions }
    other: permset;   { other permissions }
    next:  filptr     { next entry in list }

end;
```

Each directory file entry has the name of the file, along with a series of descriptive data for the file. These are divided into attributes and permissions. An attribute is a characteristic of the file, and

generally does not change. Permissions indicate what can be done with the file, and are divided into the user, group and other permissions.

Not all attributes nor all permissions are available on every operating system. If a particular permission or attribute is not implemented on a given operating system, then setting it will have no effect, and reading it will always return unset.

The size of the file is its size in bytes. The allocation is the total space it occupies on the storage medium, which may be different from its size for several reasons. The blocking may be such that the size is rounded up to the nearest block. The operating system may have the ability to reserve space for the file beyond what it is currently using, or may not release space back to the free space pool if the file is truncated.

The times of interesting events in the file's life are available, in "S2000" format (already discussed). If a particular time is not available, then it is set to **maxlint**.

The file structure is a collection of items that may be implemented on any given operating system. The way to prevent the need to decide what is and what is not implemented on a particular system is to focus on what is essential for all systems. For example, the size of a file is usually present, as well as the last modification time. The last modification time can be used to determine when to back up files, by comparing it to the date of the backup copy of the same file, or to the modification date of the archive containing the file. Similarly, a "make" style program can determine when to remake a file by looking at the modification time.

C.5 File Attributes and Permissions

The attributes of a file can be set by name with the **setatr** command. It takes a set of attributes and the filename, and sets all the given attributes on the file. The routine **resatr** resets attributes. The user permissions for a file are set and reset by **setuper** and **resuper**. The group permissions are set and reset by **setgper** and **resgper**. The other permissions for a file are set and reset by **setoper** and **resoper**.

C.6 Environment Strings

The environment is a collection of strings that is kept by the executive, and passed to programs when they are started. Each string has a name and a value, both of which are arbitrary strings. An environment string can be retrieved by name by **getenv**, and set by **setenv**. The entire environment string set can be retrieved at one time by the **allenv** routine, which uses a linked list to represent the environment strings:

```
{ environment strings }

envptr = ^envrec; { pointer to environment record }

envrec = packed record

    name: pstring; { name of string }
    data: pstring; { data in string }
    next: envptr { next entry in list }

end;
```

The standard format for environment string names is the same for Pascaline identifiers, i.e., a character in the sequence 'A'..'Z', 'a'..'z', '_', followed by any number of characters in the sequence 'A'..'Z', 'a'..'z', '_', '0'..'9'. The data in the string can be a series of any valid characters.

The reason for retrieving the entire environment is to pass it on to other programs in an **exec** statement.

Although most available operating systems implement the environment string concept, it has fallen out of favor in modern programs. Placing the needed configuration strings for a particular program into a place where the executive will use it both requires special calls, and places individual program data into a pool where it can be deleted or corrupted.

Some current systems use a repository concept where program data is kept in a central tree structured database. This is not covered in **extlib**, and would be covered in another library.

A better method is to use a file containing the configuration information for the program in its startup directory, then optionally another version in the user directory, and finally one in the current directory. This allows options that affect all runs on the current machine to be kept in one file, while the options for a particular user are kept in another file, and lastly the options in use in the current project in the current directory. This system has the advantage that it addresses concerns in a multiuser operating system, and allows each program to maintain its own startup data.

For systems that do not implement an environment string capability, implementations of Pascaline commonly keep a file in the user path area that contains the strings. This is then used just for that program, that is, the set of environment strings are kept just for the accessing program.

C.7 Executing Other Programs

An external program can be executed by **exec**, which takes the command line for the program, including the program name, and all of its parameters and other options on the same line after one or more spaces:

```
cmd parameter parameter... parameter
```

This is passed as a string to the **exec** routine. When programs are executed this way, the executing program does not need to await the finish of the program, nor can it find out if the program ran correctly. If this is required, the routine **execw** is used. **execw** will wait for the program to complete, then place the error return for the program in a variable. This variable will be 0 if the program ran correctly, or non-zero if it didn't. The exact numerical meaning of the error is up to the program executed.

If the environment is to be set for the executed program, the call **exece** can be used, which takes an environment list. This allows the environment to be retrieved from the current environment or created as new, modified or added to as needed, then passed to the executed program. **execew** does the same thing, but waits for the program to finish, and returns an error code.

If a command line concept does not exist on the target system, an implementation can pass it via another means, such as a file or environmental variable. Alternately, it could simply be ignored.

C.8 Error Return Code

Each program, when it completes, returns an error code. By convention, if the error code is 0, then no error occurred. If the code is not 0, then an error occurred, and the meaning of the number is defined by the application.

When a program under Pascaline exits, it returns a code that was set to 0 by default when the program started. The procedure **seterr** can be used to set a non-zero code, which will be returned when the program exits. Note that it does **not** cause the current program to exit, it simply sets the code that will be returned when it does. It does not matter how the program exits, from the main program block, or a **halt** statement or exception.

C.9 Creating or Removing Paths

A file path, or directory, is created by **makpth**, and removed by **rempth**. If the directory has files in it, then it cannot be removed until all the files (and directories) under it have been removed. This means by implication that each section of a path must be removed separately, and a tree structured delete would have to repeatedly remove the contents of one element of the path, then remove the path section itself, and so on.

C.10 Option Character

When parsing commands, the option character for the current operating system is found with **optchr**. **extlib** does not define the exact format of command line options. The option character is an aid to portability.

C.11 Path Character

The path character is used to separate path components, usually directories in a tree structured file system, within a path for the given system. It can be found with the **pthchr** function. **extlib** does not define the contents, structure or meaning of a path. The path division character is an aid to portability.

C.12 Exceptions

The following exceptions are generated in **extlib**:

NameTooLong	The filename passed was too long for an extlib internal buffer.
FileSizeTooLarge	The file(s) being processed were too large.
StringNil	The string passed was nil.
StringTooSmallForTime	The string result buffer was too small to hold the time.
StringTooSmallForDate	The string result buffer was too small to hold the date.
EnvironmentStringTooLarge	A system environment string was too large for an extlib buffer.
CommandStringEmpty	The command string passed was empty.
ProgramNotFound	External program not found.
CurrentPathTooLong	Current path is too long for extlib buffer.
FilenameStringEmpty	The filename passed was empty.
CannotDetermineProgramPath	The program path could not be determined.

Extlib establishes a series of exception handlers for each of the above exceptions during startup. Exceptions not handled by a client program of **extlib** will go back to **extlib**, then print a message specific to the error, then the general exception will be thrown.

Not all procedures and functions throw all exceptions. See each procedure or function description for a list of exceptions thrown. A client of **extlib** need only capture the exceptions occurring in the procedure or function that is called.

C.13 Functions and procedures in extlib

procedure list(fn: [p]string; var l: filptr);

Form a file list from the filename **fn**, and return in the file entry list **l**. The filename **fn** may contain wildcards, and may be fully pathed, or refer to the current directory. If no files are found, then the list pointer is returned **nil**.

Exceptions: FileSizeTooLarge, StringNil

procedure times(var s: string; t: linteger);
function times(t: linteger): pstring;

Place time from S2000 time **t** in string **s**.

Exceptions: StringTooSmallForTime

procedure dates(var s: string; t: linteger);
function dates(t: linteger): pstring;

Place date from S2000 time **t** in string **s**.

Exceptions: StringTooSmallForDate

procedure writetime([var f: text;] t: linteger);

Write time from S2000 time **t** to text file **f**, or by default, the standard output file.

procedure writedate([var f: text;] t: linteger);

Write date from S2000 time **t** to text file **f**, or by default, the standard output file.

function time: linteger;

Returns current S2000 time in GMT

function local(t: linteger): linteger;

Converts the given GMT S2000 time **t** to local time, using the time zone offset and daylight savings status in the host computer, and returns the result.

function clock: lcardinal;

Returns 100 microsecond, free running time.

function elapsed(r: lcardinal): lcardinal;

Given a stored **clock** time **r**, will check the current **clock** time and find the total number of 100 microsecond ticks since the given time, then return that. Accounts for timer wraparound.

function validfile(s: [p]string): boolean;

Parses and checks the file specification **s** for a valid filename on the current system. Returns true if valid.

Exceptions: StringNil

function validpath(s: [p]string): boolean;

Parses and checks the file specification **s** for a valid path on the current system. Returns true if valid, otherwise false.

Exceptions: StringNil

function wild(s: [p]string): boolean;

Checks if the file specification **s** contains wildcards. Returns true if so, otherwise false.

Exceptions: StringNil

procedure getenv(ls: string; var ds: string);

function getenv(ls: string): pstring;

Finds and returns an environment string. **ls** contains the name of the string to look up, **ds** or the return value contains the resulting string as found. If there is no environment string by that name, the return is either all blanks, or nil.

Exceptions: EnvironmentStringTooLarge

procedure setenv(sn: [p]string; sd: [p]string);

Finds and sets an environment string. **sn** contains the string name to set, and **sd** contains the contents to set it to. If there is no string by that name, it is created, otherwise the old string is replaced.

Exceptions: StringNil

procedure allenv(var el: envptr);

Returns a complete list of the strings in the environment to **el**.

```
procedure remenv(sn: [p]string);
```

Remove a string **sn** from the environment. The string is found by name, and removed from the environment. No error results if the string does not exist.

Exceptions: StringNil

```
procedure exec(cmd: [p]string);
```

Execute external program, with parameters, from the string **cmd**. Does not wait for the program to finish, and cannot detect if it finished with an error.

Exceptions: CommandStringEmpty, ProgramNotFound, StringNil

```
procedure exece(cmd: [p]string; el: envptr);
```

Execute external program, with parameters from the string **cmd** and full environment. The environment is passed as a list in **el**. Does not wait for the program to finish, and cannot detect if it finished with an error.

Exceptions: CommandStringEmpty, ProgramNotFound, StringNil

```
procedure execw(cmd: [p]string; var e: integer);
```

Execute external program, with parameters from the string **cmd**. Waits for the program to finish, and returns its error code in **e**. The error code is 0 for no error, otherwise the error is a code specified by the program executed.

Exceptions: CommandStringEmpty, ProgramNotFound, StringNil

```
procedure execew(cmd: [p]string; el: envptr; var e: integer);
```

Execute external program, with parameters from the string **cmd** and full environment. The environment is passed as a list in **el**. Waits for the program to finish, and returns its error code in **e**. The error code is 0 for no error, otherwise the error is a code specified by the program executed.

Exceptions: CommandStringEmpty, ProgramNotFound, StringNil

```
procedure getcur(var fn: string);  
function getcur: pstring;
```

Get current path to **fn** or the result. Returns the current directory path.

Exceptions: CurrentPathTooLong


```
procedure setcur(fn: [p]string);
```

Set current path from string **fn**. Sets the default path for all file specifications.

Exceptions: StringNil

```
procedure getpgm(var s: string);  
function getpgm: pstring;
```

Get the program path to **s** or returns it. Returns the program path, which is the path the program running was loaded from.

Exceptions: CannotDetermineProgramPath

```
procedure getusr(var fn: string);  
function getusr: pstring;
```

Get user path to **fn** or returns it. Return the user path, which is a path specific to each user.

```
procedure brknam(fn: [p]string; var p, n, e: [p]string);
```

Break down file specification. Breaks the file specification **fn** down into path **p**, name **n**, and extension **e**. Note that any one of the resulting components could be blank, if it does not exist in the name.

Exceptions: FilenameStringEmpty, StringNil

```
procedure maknam(var fn: string; view p, n, e: string);  
function maknam(p: [p]string; n: [p]string; e: [p]string): pstring;
```

Create file specification from components. Creates file specification **fn** from path **p**, name **n**, and extension **e**. Components may be blank, but the path and the name cannot both be blank.

Exceptions: NameTooLong, StringNil

```
procedure fulnam(var fn: string);  
function fulnam(fn: string): pstring;
```

Create full file specification from a partial file specification **fn**. Given a file specification with an incomplete path (either by using the default path, or mnemonic shortcuts for things like parent directory), creates a fully pathed name of standard form. This can "normalize" file specifications, for comparisons, and to store the complete path for the file. The fully pathed result is returned in **fn** or as the function result.

```
procedure setatr(fn: [p]string; a: attrset);
```

Set attributes. Given a file by name **fn**, the attributes in the set **a** are set true for the file.

Exceptions: StringNil

procedure resatr(fn: [p]string; a: attrset);

Reset attributes. Given a file by name **fn**, the attributes in the set **a** are set false for the file.

Exceptions: StringNil

procedure bakupd(fn: [p]string);

Set backup time current. Given a file by name **fn**, sets the backup time for the file as current. Backup programs should also reset the archive bit to show that backup has occurred.

Exceptions: StringNil

procedure setuper(fn: [p]string; p: permset);

Set user permissions. Given a file by name **fn**, the permissions in the set **p** are set true for the file.

Exceptions: StringNil

procedure resuper(fn: [p]string; p: permset);

Reset user permissions. Given a file by name **fn**, the permissions in the set **p** are set false for the file.

Exceptions: StringNil

procedure setgper(fn: [p]string; p: permset);

Set group permissions. Given a file by name **fn**, the permissions in the set **p** are set true for the file.

Exceptions: StringNil

procedure resgper(fn: [p]string; p: permset);

Reset group permissions. Given a file by name **fn**, the permissions in the set **p** are set false for the file.

Exceptions: StringNil

procedure setoper(fn: [p]string; p: permset);

Set other permissions. Given a file by name **fn**, the permissions in the set **p** are set true for the file.

Exceptions: StringNil

procedure resoper(fn: [p]string; p: permset);

Reset group permissions. Given a file by name **fn**, the permissions in the set **p** are set false for the file.

Exceptions: StringNil

procedure seterr(e: integer);

Set program return error **e**. The error code returned by the current program is set. This has no effect until the program exits.

procedure makpth(fn: [p]string);

Make path using **fn**. Creates a new path or directory. If the path already exists, it's an error.

Exceptions: StringNil

procedure rempth(fn: [p]string);

Remove path **fn**. Removes a path, or directory. The directory must exist, and must be empty of any files or other directories, or an error results.

Exceptions: StringNil

procedure filchr(var fc: chrset);

Returns the set of valid filename characters in **fc**.

function optchr: char;

Find the option character. Returns the character that is used to introduce options in command liens on the current system.

function pthchr: char;

Returns the character used to separate components in a path in the current system.

D [Annex D: String Library](#)

The string library implements various useful string functions. Strings are built out of arrays of characters in Pascal, and extended in Pascaline. **strlib** unifies two schemes. The first is the space padded right scheme familiar from standard Pascal. The second is dynamic strings.

Dynamic strings are almost the ideal string type. They are unlimited in length, they can be returned from a function. And because their storage is recycled, they are space efficient. They can also be fairly speed efficient by managing when and if they are recycled.

The declarations for padded or fixed, and dynamic strings appear as:

```
type string = packed array of char;  
    pstring = ^string;
```

The string functions are often overloaded so that they take both types of strings.

D.1 [Conventions](#)

In some cases, it is ambiguous whether a padded or dynamic string argument is meant. For example, the string compare facility does not know which type its operands are, and it makes a difference to the result, since length plays a part in string comparison. For these situations, a "p" (for padded) is appended to the name of the procedure or function.

Many functions and procedures must know if case matters. For example, string compare can be with case, or caseless. For these procedures and functions, there is appended a "c" to the ones that case does matter.

A few functions and procedures perform different actions on strings vs. characters. For the string versions, an "s" is appended to the name.

D.2 [Words](#)

Some of the functions and procedures treat the strings as a series of words. Words are a series of non-space characters surrounded by one or more spaces. For example:

```
‘ hi    there           George    ‘
```

Has three words, 'hi', 'there', and 'george'. Such words can be counted, indexed and extracted.

D.3 [Format Strings](#)

Some routines accept a "format string" to output numbers with. The format is an image of the output string the number is converted into. The string will contain a series of format characters. The entire format string is copied to the result string, but the special format characters are replaced with parts of the number to be converted.

The number “image” contained within the format string consists of from 0 to any number of leading spaces, followed by any number of trailing spaces. The spaces are significant, and will appear in the result.

The first thing the format routines do is match the format to the number for decimal point position. For example:

+999,999.999	Format characters
50.12	Actual number

Then, the meaning of each format character depends on if that character appears to the left or to the right of the decimal point. There can only be one decimal point in the format string, or it is an error.

Each format character operates on the digit that matches its position after decimal point alignment.

The format characters are:

9

Represents a digit. To the left of the decimal point, this is replaced with its matching digit from the number. If there is no digit in the number at that position, a space replaces the format character. To the right of the decimal point, if the digit position is non-zero, or there are non-zero digits to the right, the digit replaces the format character, otherwise space.

0

As "9" above, but "0" replaces the digit if no significant digit is found, instead of space when to the left of the decimal point. To the right of the decimal point, if the digit position is non-zero, then it replaces the format character, otherwise remains 0.

-

Represents the sign. To the left of the decimal point, if the number is negative, it is left alone. If the number is positive, it is replaced by a space. If the sign was already output, then it is replaced by space. It is an error if this format character is used to the right of the decimal point.

+

As "-", but "+" appears instead of space on a positive number.

\$, &, %

These characters are used to indicate if the number is hex (or USA dollars), octal or binary. To the left of the decimal point, if a significant digit can be matched to this position, that is output, otherwise, either the character, or a space is placed. The space is placed if any of the format characters '\$', '&', or '%' have already appeared. It is an error if this format character is used to the right of the decimal point.

,

To the left of the decimal point, if the comma appears to the right of any non-zero significant digits, it is output as is. Otherwise, it is either replaced by space. If a "\$", "&" or "%" character appears to the left of it, it will be replaced by that. It is an error if this format character is used to the right of the decimal point.

May only appear on real numbers. This format character is always printed, but specifies where the mantissa of a number appears, and where its fraction appears. The appearance of the decimal point will enable or disable the fraction. If present, fractional digits are output, otherwise, the fraction is discarded.

D.4 Recycling

By default, **strlib** leaves it up to the programmer to determine when dynamic strings should be recycled. To keep from losing space ("memory leaks"), the program must be careful to keep track of all dynamic strings created, and return them to free storage via `dispose`.

Strlib can use a nested blocking system to automatically dispose of strings for you. The **openstring** call begins a new string block, and **closestring** ends it. When a new block is opened, any dynamic strings allocated are recorded in the block. Then, each of them are disposed of when the block ends. Any number of block levels can exist. With no blocks in effect, the automatic recycling system is off, which is the state **strlib** starts in. When all blocks are closed, **strlib** reverts to this state. Because the dynamic strings are returned as a group, this system can be more efficient than returning the strings one at a time.

Exportstring to completely remove a given dynamic string from the automated recycling system. The **upstring** call takes a string, and moves it to the surrounding string block.

The **strlib** block system has nothing to do with blocks in Pascal. **strlib** blocks can cross blocks, functions and even whole modules of Pascal. The two are entirely unrelated.

D.5 Exceptions

The following exceptions are generated in **strlib**:

NoStringBlock	No string block is active.
OuterBlockFull	No room in outermost string block.
CurrentBlockFull	Current string block is full.
StringNil	String passed is nil.
StringDestinationOverflow	String was too large for destination.
IndexOutOfRange	String index out of range.
NegativeRepeatCount	Repeat count was negative.
WordIndexOutOfRange	Word array index was out of range.
StringReadOverflow	String was too large to read.
FormatTooLarge	Format is too large for destination
InvalidFieldSpecification	Field specified is invalid
NegativeValueNondecimal	Radix was negative
NumberOverflowsFormat	Number overflows space provided in format string.
NegativeNotPlaced	Negative sign not placed in format.
InvalidRealNumber	Invalid real number.
InvalidFractionSpecification	Invalid fraction specification.
InvalidRadix	Invalid radix
InvalidIntegerFormat	Invalid integer format.
NumberTooLarge	Number too large.
IntegerTooLarge	Integer too large.
InvalidRealFormat	Invalid real format.

Strlib establishes a series of exception handlers for each of the above exceptions during startup. Exceptions not handled by a client program of **strlib** will go back to **strlib**, then print a message specific to the error, then the general exception will be thrown.

Not all procedures and functions throw all exceptions. See each procedure or function description for a list of exceptions thrown. A client of **strlib** need only capture the exceptions occurring in the procedure or function that is called.

D.6 Procedures and functions in strlib

```
function lcase(c: char): char;
function lcase(s: pstring): pstring;
function lcases(s: string): pstring;
procedure lcases(s: string);
```

Finds the lower case version of a character **C** or string **S**. This is either returned, or converted in place.

Exceptions: StringNil


```
function ucase(c: char): char;  
function ucase(s: pstring): pstring;  
function ucases(s: string): pstring;  
procedure ucases(s: string);
```

Finds the upper case version of a character **c** or string **s**. This is either returned, or converted in place.

Exceptions: StringNil

```
procedure clears(var s: string);
```

Clears string **s** to all spaces. To set an entire string to another character value besides space, use **rep** below.

```
function len(s: [p]string): integer;
```

Finds the padded length of a string **s**. This is equivalent to the index of the last non-space character, or zero if there is none. Note that dynamic string lengths are found via the system function **max**.

Exceptions: StringNil

```
procedure copy(var d: string; s: [p]string);  
function copy(s: [p]string): pstring;  
procedure copy(var d: pstring; s: string);
```

Create a copy of the source **s** string in the destination **d** or returns it as a result. If the destination is a fixed string, then it will be padded on the right with spaces to fill it out. If the destination or return type is dynamic, then a new string will be created of the same length as the source, and the source copied to that. In the case of the last procedure, the source is taken to be a padded string, and the length is without padding. This procedure is used to copy from a padded string to a dynamic string.

Exceptions: StringNil, StringDestinationOverflow

```
procedure cat(var d: string; s: string);  
function cat(sa: [p]string; sb:[p]string): pstring;
```

Concatenate two strings **d** and **s**, or **sa** and **sb**. For the first procedure, the padded destination and source are concatenated into the padded destination. The function concatenates unpadded strings to a dynamic result.

When concatenating two padded strings, it is not possible using **cat** to concatenate with a space or spaces between strings. For this case, use a padded string **insert** instead, with the source string inserted after the desired number of spaces.

Exceptions: StringNil, StringDestinationOverflow

```
function comp[c](sa: [p]string; sb: [p]string): boolean;  
function comp[c]p(sa, sb:string): boolean;
```

Compare strings **sa** and **sb**. Returns true if the strings are equal. Compares case or caseless, and padded or fixed length. Strings are equal only if they have the same length and content.

Exceptions: StringNil

```
function gtr[c](sa: [p]string; sb: [p]string): boolean;  
function gtr[c]p(sa, sb:string): boolean;
```

Compare string **sb** is greater than string **sa**. Returns true if the second string is greater than the first. Compares case or caseless, and padded or fixed length. Strings are compared from left to right, until the first difference is found. Then, the second string is greater than the first if the ord of its character is greater than the first. In case one string is longer than the other, but otherwise equal, the shorter string is less than the longer string.

This function can be used to find less than, greater than or equal, and less than or equal by arranging the operands:

```
gtr(a, b)      a < b  
gtr(b, a)      a > b  
not gtr(a, b)   a >= b  
not gtr(b, a)   a <= b
```

Exceptions: StringNil

```
function index[c](sa: [p]string; sb: [p]string): integer;  
function index[c]p(sa, sb:string): integer;
```

Finds the incidence of the source string **sb** in the string **sa**. If the source string is found within the destination, the index of its first character is returned, otherwise 0. The comparison may be case or caseless. The padded versions are for when the source string is padded.

Exceptions: StringNil

```

procedure extract(var d: string; s: string; l, r: integer);
function extract(s: [p]string; l, r: integer): pstring;

```

Extract a substring. The source string **s** from the left index to the right index is extracted, and either placed in the destination **d**, or returned. It is an error if either index is out of range, but indexes $l > r$ simply result in a null string. If the result is too large for the destination, an error results. The procedure version is for padded strings.

Common equivalents using extract are:

<code>right(s, l)</code>	<code>extract(s, max(s)-l+1, max(s))</code>	Get right string
<code>left(s, l)</code>	<code>extract(s, 1, l)</code>	Get left string
<code>mid(s, l, r)</code>	<code>extract(s, l, l+r-1)</code>	Get mid string

Exceptions: StringNil, StringDestinationOverflow, IndexOutOfRangeException

```

procedure insert(var d: string; s: string; p: integer);
function insert(sa:[p]string; sb: [p]string; p: integer): pstring;

```

Inserts a substring from the source **s** into the destination **d**. The procedure version is for padded strings. If the source string is too long for the destination, an error results.

```

function rep(s: [p]string; r: integer): pstring;
procedure rep(var d: string; s:string; r: integer);

```

Repeats the source string **s** **r** times into the destination **d** or the return string. The procedure version is for padded strings. If the resulting string is too long for the destination, an error results.

```

function trim(s: [p]string): pstring;
procedure trim(var d: string; s: string);

```

Trim leading and trailing spaces from string **s**. and returns the result in string **d** or the result. The procedure version is for padded strings. For padded strings, only the leading spaces are affected.

```

function words(s: [p]string): integer;

```

Returns a count of the number of space delimited words in the string **s**.

```

function extwords(s: [p]string; l, r: integer): pstring;
procedure extwords(var d: string; s: string; l, r: integer);

```

Extracts the space delimited words from the string **s**, between the left to the right indices inclusive. Returns the result in **d** or as the result.

```
procedure reads([f: text;] var s: [p]string [; var ovf: boolean]);
```

Reads a line to string **s** from text file **f**. The versions of this procedure that have an **ovf** flag return it true if the input line overflowed the string. In this case, the string is returned truncated to the left. The non-**ovf** versions of the procedure **halt** with error on overflow.

If the input file is not specified, then the standard **input** file is used.

```
procedure ints(var s: string; i: integer [; fl: integer] | [fmt: string]);  
function ints(i: integer [; fl: integer] | [fmt: string]): pstring;
```

Convert integer **i** to string **s**. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" in the main text.

```
procedure reals(var s: string; r: real [; f: integer] | [; fl: integer; fr: integer] | [; fmt: string]);  
function reals(r: real [; f: integer] | [; fl: integer; fr: integer] | [; fmt: string]): pstring;
```

Convert real **r** to string **s** with fraction **f**. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" in the main text.

```
procedure reales(var s: string; r: real [; fl: integer]);  
function reales(r: real [; fl: integer]): pstring;
```

Convert real **r** to string **s** using "economy" format. The real is printed in the minimum number of characters possible. If the decimal position can be placed into the number, then it is, and the exponent is removed. All insignificant leading and trailing zeros are removed, and if there are no digits to the right or left of the decimal point, then that is removed.

If the decimal point cannot be placed into the number with less total characters than a full ISO 7185 floating point number format, then the standard floating point format is used.

The versions of this call with a field **fl** fit the result into the field by padding it out with blanks, either on the left for positive fields, or on the right for negative fields. If the number does not fit into the specified field, then all of the required parts of the number are printed. Note that if the ISO 7185 floating point number is chosen as the shortest length number, then the rules for fields are identical to that of normal real output formats.

```

procedure hexs(var s: string; w: lcardinal [; fl: integer]);
procedure hexs(var s: string; w: lcardinal; fmt: string);
function hexs(w: lcardinal [; fl: integer]): pstring;
function hexs(w: lcardinal; fmt: string): pstring;

```

Convert cardinal **w** to string **s**, using the hexadecimal radix system. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" in the main text.

```

procedure octs(var s: string; w: lcardinal [; fl: integer]);
procedure octs(var s: string; w: lcardinal; fmt: string);
function octs(w: lcardinal [; fl: integer]): pstring;
function octs(w: lcardinal; fmt: string): pstring;

```

Convert cardinal **w** to string **s**, using the octal radix system. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" above.

```

procedure bins(var s: string; w: lcardinal [; fl: integer]);
procedure bins(var s: string; w: lcardinal; fmt: string);
function bins(w: lcardinal [; fl: integer]): pstring;
function bins(w: lcardinal; fmt: string): pstring;

```

Convert cardinal **w** to string **s**, using the binary radix system. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" above.

```

procedure writed([var f: text;] i: integer; fmt: string);

```

Convert integer **i** to character format and output to file **f**, using the decimal radix system. Decimal numbers are well covered by the normal ISO 7185 standard write formats. However, these routines add image formatting to them.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" in the main text.

If the output file is not specified, then the standard output file is used.

```
procedure writer([var f: text;] r: real; fmt: string);
```

Convert real **r** to string **s**. Real numbers are well covered by the normal ISO 7185 standard write formats. However, these routines add image formatting to them.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" in the main text.

If the output file is not specified, then the standard output file is used.

```
procedure writere(var f: text; r: real; fl: integer);
procedure writere(r: real; fl: integer);
procedure writere(var f: text; r: real);
procedure writere(r: real);
```

Write real **r** to the "economy real" format to either the specified output file **f**, or to the standard output. The economy real format is explained in the **reales** routines descriptions.

The versions with a field **fl** fit the output to the field using standard Pascal rules, including negative fields.

If the output file is not specified, then the standard output file is used.

```
procedure writeh([var f: text;] w: lcardinal [; fl: integer]);
procedure writeh([var f: text;] w: lcardinal; fmt: string);
```

Write cardinal **w** to either the specified file **f**, or the standard output file, using the hexadecimal radix system. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" above.

```
procedure writeo([var f: text;] w: lcardinal [; fl: integer]);
procedure writeo([var f: text;] w: lcardinal; fmt: string);
```

Write cardinal **w** to either the specified file **f**, or the standard output file, using the octal radix system. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" above.

```
procedure writeb([var f: text;] w: lcardinal [; fl: integer]);  
procedure writeb([var f: text;] w: lcardinal; fmt: string);
```

Write cardinal **w** to either the specified file **f**, or the standard output file, using the binary radix system. The versions with a field **fl** format the number using the field using standard Pascal rules for output using a field. Negative (left justified) fields are also allowed.

The versions with a **fmt** string copy the format string into the result string, then replace each of the format characters with parts of the number. See "format strings" above.

```
function intv(s: [p]string [; var ovf: boolean]): integer;
```

Find value of string **s** with default decimal format. The number contained in the string, in valid Pascal signed number format, is parsed and the value returned. The number can be signed. Leading and trailing spaces are ignored, but extra characters past a valid number result in an error.

If the number is unsigned, and has a valid radix specifier, one of "\$", "&" or "%" prepended to it, then that radix overrides the default decimal radix. If this override effect is not desired, then the number should be checked for such radix specifications using index, and an error generated.

The versions of this call that feature the **ovf** flag do not produce an error if the number is too large to convert, but instead set the **ovf** flag, and then the result is undefined.

```
function hexv(s: [p]string [; var ovf: boolean]): lcardinal;
```

Find value of string **s** with default hexadecimal format. The number contained in the string, in valid Pascal signed number format, is parsed and the value returned. The number may not be signed. Leading and trailing spaces are ignored, but extra characters past a valid number result in an error.

If the number has a valid radix specifier, one of "\$", "&" or "%" prepended to it, then that radix overrides the default hexadecimal radix. If this override effect is not desired, then the number should be checked for such radix specifications using index, and an error generated.

The versions of this call that feature the **ovf** flag do not produce an error if the number is too large to convert, but instead set the **ovf** flag, and then the result is undefined (and probably is garbage).

```
function octv(s: [p]string [: var ovf: boolean]): lcardinal;
```

Find value of string **s** with default octal format. The number contained in the string, in valid Pascal signed number format, is parsed and the value returned. The number may not be signed. Leading and trailing spaces are ignored, but extra characters past a valid number result in an error.

If the number has a valid radix specifier, one of "\$", "&" or "%" prepended to it, then that radix overrides the default octal radix. If this override effect is not desired, then the number should be checked for such radix specifications using `index`, and an error generated.

The versions of this call that feature the **ovf** flag do not produce an error if the number is too large to convert, but instead set the **ovf** flag, and then the result is undefined (and probably is garbage).

```
function binv(s: [p]string [: var ovf: boolean]): lcardinal;
```

Find value of string **s** with default binary format. The number contained in the string, in valid Pascal signed number format, is parsed and the value returned. The number may not be signed. Leading and trailing spaces are ignored, but extra characters past a valid number result in an error.

If the number has a valid radix specifier, one of "\$", "&" or "%" prepended to it, then that radix overrides the default binary radix. If this override effect is not desired, then the number should be checked for such radix specifications using `index`, and an error generated.

The versions of this call that feature the **ovf** flag do not produce an error if the number is too large to convert, but instead set the **ovf** flag, and then the result is undefined (and probably is garbage).

```
function realv(s: [p]string): real;
```

Find value of real number string **s**. The string must contain a valid Pascal format real number. Leading and trailing spaces are ignored, but extra characters past a valid number result in an error.

```
procedure openstring;
```

Causes a new string block level to be created. All strings that were dynamically allocated within **strlib** will be recorded within the new block, and disposed of automatically on `closestring`. Note that there is a limit of 100 strings that can be allocated per block.

```
procedure closestring;
```

Removes the current string block level. Each string that was allocated and recorded in the current block is disposed of, and the surrounding string block, if it exists, is restored. All **strlib** dynamic allocations will then be recorded in that block.

It is an error if no block exists to close.


```
procedure exportstring(s: pstring);
```

Removes the indicated string **s** completely from the current string block. Only the current block is searched. It is only possible to export a string from the block it was created in. If the string does not exist in the current block, no error results.

```
procedure upstring(s: pstring);
```

Moves the indicated string **s** from the current block to the surrounding block. The string is removed from the current block, and recorded in the surrounding block, if it exists. If the string does not exist in the current block, or a surrounding block does not exist, no error results. If the surrounding block is full, an error results.

```
procedure subst[c][all](var s: string; m: string; r: string);  
function subst[c][all](s: [p]string; m: [p]string; r: [p]string): pstring;
```

Substitutes a string within another string. The source string **s** is searched for the match string **m**. If found, the match string is removed, and the replacement string **r** takes its place. The string lengths and positions are readjusted for any size difference between the match and replacement strings. The final string is either returned in the string **s**, or returned as the result.

Several variations exist. The matching process can be case sensitive, or not. In normal **subst** calls, only the first match is replaced. In the "all" variations, all matching substrings are replaced. Replacement strings are not searched in the "all" variations. The match process skips over the replacement string.

E [Annex E: Terminal Interface Library](#)

Standard ISO 7185 Pascal uses an I/O paradigm that is serial, or more correctly called "line oriented". Each line is built up in sections, then output to the I/O device with an appended "end of line". The end of line causes the current line to be completed, and the next line begins.

The next level of paradigm is the presentation of lines onto a 2d text surface. This can simply be an emulation of the serial only system or "virtual paper" using a screen that scrolls up from the bottom. The next step is to allow full addressing of the 2d surface and allowing text to be placed anywhere within that surface. To this is added colors, different text presentation modes, and finally advanced, multiple device input.

trmlib starts in **serlib** compatible mode, then allows the program to move to a full addressable surface without automatic scrolling. In advanced mode, **trmlib** emulates a infinite virtual surface where the terminal exists as a window clipped to the origin. This is the most consistent model of such a surface.

Because **trmlib** overrides the interface between the program and the operating system it runs on, all of the ISO 7185 defined serial I/O works compatibly to the **trmlib** presented surface. It is also modal, meaning that changes to character modes affect all further output to the terminal.

trmlib introduces the concept that several devices can be used for input at the same time. The normal user keyboard is supplemented by a mouse, timers and a joystick. These are all implemented via a general event concept, which unifies the input and removes the need to poll multiple devices.

trmlib gives a set of logical events for common control keys from the keyboard that allow the program to avoid direct recognition of implementation dependent key codes.

In addition to the default presentation surface, **trmlib** is capable of switching between multiple display surfaces. This capability has several uses.

E.1 [Standard Output](#)

To write data to the **trmlib** screen, ordinary Pascal write calls are used. A **write** statement places each character on the screen, then moves the cursor to the right, obeying any automatic line wrapping. If a **writeln** is called, then the cursor is returned to the left side of the screen, one line down. If the end of the line is reached, and automatic scrolling is enabled, then the screen will scroll upwards.

The **page** procedure, which causes a printer to move to the next (blank) page, is emulated by blanking the screen, then placing the cursor at the upper left handcorner of the screen.

E.2 [Basic Cursor Positioning](#)

The cursor is the point where text is entered onto the screen. Its usually marked with a blinking block or underline. To move the cursor one character up, down, left or right, the up, down, left and right procedures are used. To move the cursor anywhere on the screen, the **cursor** call is used. To find out where the cursor currently is, the functions **curx** and **cury** return the current x and y coordinates of it.

The character cells on the screen are labeled from 1 to N, where in x it's 1 to the left side of the screen, and N is the right side of the screen. In y it's 1 at the top of the screen, and N at the bottom of the screen.

The actual size of the screen is system specific, and can be found by **maxx** and **maxy**, which return the maximum index in x and y for the screen. When a terminal is emulated in a windowing system, it usually by default 25 lines of 80 characters each, because that was a very common size in terminals.

E.3 [Automatic Mode](#)

Automatic mode is the mode that causes **trmlib** to scroll upwards when the bottom of the screen is reached, and an **eoln** is written. Also, line wrap, or the wrapping of characters back to the left at the next line if text is written off the right hand side, is an automatic mode. The automatic mode is useful for emulating serial mode programs, but rapidly gets in the way for advanced programs.

Automatic mode can be turned off with **auto**. Turning **auto** off converts (or actually, reveals) the screen as a character surface that goes from -maxint to +maxint in both x and y, and has its origin at 1,1, and the screen is a "viewport" on this surface that extends from 1,1 to maxx, maxy. Text can be drawn anywhere, including off screen, but the characters outside of the 1,1 to maxx, maxy box will not be seen, and are "clipped out" of view. It can be determined if the cursor lies within the screen's bounds by **curbnd**.

The "virtual screen" that appears with **auto** off may seem strange at first, but it's a good match for today's windowing environments. Text can be drawn without worrying if it will cause the line to wrap, or the screen to scroll. And if a line of text happens to extend off the screen, that does not cause an error.

E.4 [Tabbing](#)

trmlib will keep track of tabs set in x, for any character position. When **trmlib** starts, the tabs are set to every 8th position on the screen, i.e., positions x= 9, 17, etc.

Outputting a tab character will cause the cursor to move to the next tab position on the line. If the cursor is at a tab position, then it will move to the next one. Tab positions can be set by **settab**, and cleared by **restab**. **clrtab** clears all set tabs.

E.5 [Scrolling](#)

The screen can be scrolled by the scroll routine. Actually, it implements true arbitrary scrolling. If the x value given is positive, then the screen data scrolls up. If the x value is negative, then the screen data scrolls down. Likewise, if the y value is positive, the screen data scrolls left, and if it's negative, the screen data scrolls right. If either x or y is 0, then there is no movement in that direction.

E.6 [Colors](#)

There are two colors to set for text. One is the foreground, and the other is the background. The foreground is the color within the character itself. The background is the space behind the character. The possible colors are chosen from the two sets of primary colors:

```
type color = (black, white, red, green, blue, cyan, yellow,
              magenta);
```

Characters are written in the currently set foreground and background colors. The foreground color is set by **fcolor**, and the background color by **bcolor**.

The current background color is also used to set the color of any blanked out areas caused by other commands. For example, the page procedure clears the screen to the background color. Scrolls, either programmer selected or automatic, use the background color for any uncovered areas that are blanked out.

For a program to be flexible, it should not rely on the fact that any more than 2 colors, black and white, are available.

E.7 Attributes

trmlib provides many attribute controls, each of which can be switched on or off individually.

reverse

Enables reverse video.

underline

Underlines each character.

superscript

Gives a smaller and higher character for subscripting.

subscript

Gives a smaller and lower character for superscripting.

italic

Prints in italic or slanted characters.

bold

Prints in extra dark, or bold characters.

Strikeout

Prints characters with a horizontal bar through them.

blink

Blinks each character on and off.

For programs to maintain portability, it is recommended that the programmer assume two things. First, that only a single attribute at one time can be set. This would mean that setting a second attribute after a first one is set, would cause the first attribute to be unset.

Second, don't assume that any one particular attribute is available. Many older terminals did not have more than one or two attributes. Superscript, subscript, italic and strikeout were rare in standalone

terminals. Superscript, subscript, italic and bold are rare today on graphical windowing systems because they alter the geometry of the characters such that it is difficult or impossible to emulate a character cell in such a system. blink is also rare in graphical systems because of the computational work required.

For this reason, there is a general mode, `standout`, that can be enabled or disabled just like an attribute. What `standout` does is enable an attribute the terminal does have, so that the program does not have to select a specific attribute.

E.8 Multiple Surface Buffering

trmlib implements up to 10 logical screens in buffers. This is a copy of the characters on the screen, saved in a buffer of the same size as the screen. Logical buffers have many uses. Multiple screens can be kept, and quickly switched to the user. A program, like an editor, that wants to save the screen as it was before it started, can switch away to a second screen to do its work, then switch back to the original screen to restore its contents. Buffers can also be used to accomplish animation.

The `select` procedure sets both the current screen to be displayed, and also the screen that updates are to go to. When **trmlib** starts, these are both set to the same screen, number 1. Any screen can be selected, from 1 to 10, and the display screen and the update screen can be different. If the update screen is not the one in display, then all of the write statements will place characters in the buffer, but not on screen. This "split mode" is typically used for animation.

E.9 Advanced Input

trmlib implements an advanced output model that is upward compatible with the old serial model. Similarly, an advanced input model is also implemented that meets the needs of newer systems.

Today's programs have to deal with multiple input devices, not just the keyboard. Devices include a mouse, a joystick, timers, and other future devices. The standard method on small computers was to poll for such devices, or accept interrupts from them. The difficulty with those solutions was that these methods did not fit well with modern multitasking systems.

One method would be to use a multithread per device. However, this complicates small programs unnecessarily. The common solution today is "events", and the "event loop". The event model takes advantage of the fact that user input devices don't generate a lot of high bandwidth data. Each of the devices attached to the input from the user have their data packaged as "messages", which are small records, which are the description of the event. These events are then placed in a queue, and the program pulls the events, one at a time, from the queue and acts on them.

The description of an event record is:

```
const maxtim = 10; { maximum number of timers available }
```

```
type
```

```
joyhan = 1..4; { joystick handles }
joynum = 0..4; { number of joysticks }
joybut = 1..4; { joystick buttons }
joybtn = 0..4; { joystick number of buttons }
joyaxn = 0..3; { joystick axes }
mounum = 0..4; { number of mice }
mouhan = 1..4; { mouse handles }
moubut = 1..4; { mouse buttons }
timhan = 1..maxtim; { timer handle }
funky  = 1..100; { function keys }
```

```
{ events }
```

```
evtcod = (etchar,      { ANSI character returned }
          etup,        { cursor up one line }
          etdown,      { down one line }
          etleft,      { left one character }
          etright,     { right one character }
          etleftw,     { left one word }
          etrightw,    { right one word }
          ethome,      { home of document }
          ethomes,     { home of screen }
          ethomel,     { home of line }
          etend,       { end of document }
          etends,      { end of screen }
          etendl,      { end of line }
          etscrl,      { scroll left one character }
          etscrr,      { scroll right one character }
          etscru,      { scroll up one line }
          etscrd,      { scroll down one line }
          etpagd,      { page down }
          etpagu,      { page up }
          ettab,       { tab }
          etenter,     { enter line }
          etinsert,    { insert block }
          etinsertl,   { insert line }
          etinsertt,   { insert toggle }
          etdel,       { delete block }
          etdell,      { delete line }
          etdelcf,     { delete character forward }
          etdelcb,     { delete character backward }
          etcopy,      { copy block }
          etcopyl,     { copy line }
          etcan,       { cancel current operation }
          etstop,      { stop current operation }
          etcont,      { continue current operation }
          etprint,     { print document }
```

```

    etprintb, { print block }
    etprints, { print screen }
    etfun,    { function key }
    etmenu,   { display menu }
    etmouba,  { mouse button assertion }
    etmoubd,  { mouse button deassertion }
    etmoumov, { mouse move }
    ettim,    { timer matures }
    etjoyba,  { joystick button assertion }
    etjoybd,  { joystick button deassertion }
    etjoymov, { joystick move }
    etterm);  { terminate program }

{ event record }

evtrec = record

    case etype: evtcod of { event type }

        { ANSI character returned }
        etchar: (char:          char);

        { timer handle that matured }
        ettim: (timnum:          timhan);
        etmoumov: (mmoun:          mouhan; { mouse number }
                  moupx, moupy: integer); { mouse movement }
        etmouba: (amoun:          mouhan; { mouse handle }
                  amoubn:          moubut); { button number }
        etmoubd: (dmoun:          mouhan; { mouse handle }
                  dmoubn:          moubut); { button number }
        etjoyba: (ajoyn:          joyhan; { joystick number }
                  ajoybn:          joybut); { button number }
        etjoybd: (djoyn:          joyhan; { joystick number }
                  djoybn:          joybut); { button number }
        etjoymov: (mjoyn:          joyhan; { joystick number }
                  joypx, joypy, joypz: integer); { joystick
                                                    coordinates }
        etfun: (fkey:          funky); { function key }
        etup, etdown, etleft, etright, etleftw, etrightw, ethome,
        ethomes, ethomel, etend, etends, etendl, etscrl, etscrr,
        etscru, etscrd, etpagd, etpagu, ettab, etenter, etinsert,
        etinsertl, etinsertt, etdel, etdell, etdelcf, etdelcb, etcopy,
        etcopyl, etcan, etstop, etcont, etprint, etprintb, etprints,
        etmenu, etterm: (); { normal events }

    { end }

end;

```

The next event for the program is retrieved via the **event** call. The basis of the event system is that events must be retrieved often enough that the input queue does not fill up. Thus, an "event model"

program will be centered around the event loop that gets the next event, acts on it, and returns to the top for more events.

E.10 Timers

Timers allow a **trmlib** program to perform periodic events, such as screen updates, and keeping track of time. From 1 to 10 timers are available, numbered 1..10. Each timer is given a time to measure, in 100 Microsecond counts (see **extlib** for more details). When the timer is done, it sends an event to the event queue.

Timers can either simply stop when their time is done, or they can automatically start timing their original set time again. This is the "recurrent" mode, and it's useful when an activity needs to be performed periodically for the life of the program. For example, updating the screen on an active program, or checking when to update a clock display.

Timers are set by timer. A timer can be stopped or "killed" by **killtimer**. Killing a timer that is not active will not generate an error. This allows a single run timer to be killed without worrying if it will time out during the call.

It is important to understand that due to the queuing nature of the event system, there is no guarantee about the accuracy of a timer. It can arrive later than its set time. If the program is late getting back to the event queue, or is busy with other events that occur before the timer, receipt of the timer event can be very late. All that receiving a timer event does is tell you that the time it measures has passed.

An example of what this means is the display of a clock, using the **extlib** time call. If we set a recurrent timer to go off on every second, the program should NOT simply advance the second on each timer event. Instead, the timer event should tell the program to read time to determine if the second has changed, and what value it currently has.

E.11 Mouse

The mouse gives a position x,y on the screen, as well as from 1 to 4 buttons on it. A mouse actually gives its position as relative movements in x and y, but the system converts this to a screen position for you. Its possible to have from 0 to 4 mice, and the function mouse returns the number of mice attached to the system.

Mice generate two events. First, when the mouse moves, it generates position changes via the **etmoumov** event. The program does not have to worry about where it is going. Each time the position changes, a new x, y position is posted as an event.

The second event generated by a mouse is mouse button asserts and deasserts, **etmouba** and **etmoubd**. An "assert" means a press of the button, and a "deassert" is the release of the button. Note that instead of an on/off status check as polled by the program, the assert and deassert events give exact notice of when the button changes state, and what it is changing to.

When there is more than one mouse per system, the default behavior should be to treat them as separate, but equal controls on the screen from the same user. When a mouse moves or changes button state, it gets control of the program. This matches the common use of multiple pointing devices, where two devices are alternated by one user. For example, a trackball and a mouse may be just two different input methods from one user.

Alternately, a second mouse can actually be a remote mouse over a network. This "collaborative computing" model allows two users to look at the same document, with separate mice.

E.12 Joysticks

From 0 to 4 joysticks may be supported. Each joystick can have from 0 to 3 "axes" of directions of travel. In addition, each joystick can have 0 to 4 buttons. The function `joystick` returns the number of joysticks in the system. The function `joyaxis` gives the number of axes on a given joystick. The function `joybutton` gives the number of buttons on a given joystick.

The messages **etjoyba** or joystick button assert, and **etjoybd** or joystick button deassert, give events for the assertion and deassertion of the buttons on a joystick. When any axis on a joystick moves, it generates a **etjoymov**, or joystick movement, event. This event gives the relative setting of each axis of the joystick.

Unlike a mouse, a joystick is entirely relative. Each axis is represented by an integer. If the axis is not implemented it always reads 0. If the axis is deflected left, up or in, depending on the type of axis, then it is negative. If it is right, down, or out, it is positive. The axis is determined in its amount of deflection. If it is in the middle, it is 0. If it is deflected to the maximum, it is **maxint**, with the sign giving the direction.

By convention, the axes on a joystick are:

1. Left/right, or slider.
2. Up/down
3. In/out

Joysticks are "rate limited" devices. This means that no matter how fast the joystick is "slewed", it will only produce an event about every 10 milliseconds at maximum rate. The reason for this is that humans cannot generally perceive events like movement of a pointer across the screen at a faster rate than this, so there is no point in updating faster than that, and flooding the input event queue.

E.13 Function Keys

The system may have from 0 to 100 function keys, which are keys whose function are determined by the program. The number of function keys implemented are found with `funkey`. Function key messages are sent by the message `etfun`.

E.14 Terminal objects

All of the procedures, functions and other declarations in **trmlib** are also available in a terminal object. of the form:

```
Class terminal;

var input, output: text; { input and output files for this screen }
    er: evtrec;           { event record }

{ procedures and functions from trmlib }

.
```

The complete catalog of procedures and functions from `trmlib` are available as methods in the terminal class. There are minor differences, which are detailed with the procedure and function descriptions in E.16.

When a terminal object is instantiated, it's input and output files are automatically linked to the standard **input** and **output** files of the program. Thus, where specifying the input and output files in **trmlib** procedures and functions is optional, in methods of the terminal object these files ***must not*** be specified.

The input and output files used by the terminal object are made available as class members `input` and `output`. This allows them to be used in non-method procedures (such as `read` and `write`).

A terminal object also contains an event record, so it is not necessary to specify an external event record in the **event** procedure.

A terminal object can be created as follows:

```
program p;

joins trmlib;

var ti: instance of trmlib.terminal;

begin

    ti.home; { send cursor to home position }
    writeln(ti.output, 'hello, terminal world');
    repeat { event loop }

        ti.event; { get next event }
        { process events }

    until ti.event = etterm { loop until program cancelled }

end.
```

Where **terminal** is the terminal object. Terminal objects can be instantiated statically or dynamically.

Terminal objects contain their own state for the following:

1. Location of cursor.
2. Current attributes and colors.
3. Buffer working/display status.
4. Tab stops.
5. Timer numbers.

However, each terminal object gets the complete state of the module **trmlib** when it is created.

Having each terminal object possess its own state means that each object can be writing to its own section of the screen in its own mode. Each object can even have it's own private screen buffer, but this must be specifically selected using the **select** procedure.

When multiple terminal objects exist, and all have the cursor visibility on, the cursor will remain at the last place it was specifically directed to. This can mean that the cursor will rapidly appear to swap back and forth between the active drawing terminal objects. This can be improved by only selecting one of the terminal objects as having a visible cursor. Typically, a the cursor should be placed to attract user attention to where text is being entered, or if none is being entered, it should be turned off.

E.15 Exceptions

The following exceptions are generated in **trmlib**:

TooManyFiles	The total number of open files possible was exceeded.
NoJoyStick	No joystick access was available.
NoTimer	No timer access was available.
ToManyTimers	The total number of timers available was exceeded.
CannotPerformSpecial	Cannot perform operation on specialfile.
FilenameEmpty	Filename specified was empty.
InvalidScreenNumber	Screen number specified was invalid.
InvalidHandle	An invalid handle was specified.
InvalidTab	An invalid tab position was specified.
CannotCreateScreenBuffer	Cannot create a buffer for the screen.
CannotQueryJoystick	Could not get information on joystick.
InvalidJoystickHandle	Invalid handle specified for joystick.
InvalidTimerHandle	Invalid handle specified for timer.

trmlib establishes a series of exception handlers for each of the above exceptions during startup. Exceptions not handled by a client program of **trmlib** will go back to **trmlib**, then print a message specific to the error, then the general exception will be thrown.

Not all procedures and functions throw all exceptions. See each procedure or function description for a list of exceptions thrown. A client of **trmlib** need only capture the exceptions occurring in the procedure or function that is called.

E.16 Procedures, functions and methods in trmlib

For all of the following calls, If the screen file **f** is not present, the default is the standard input or output file. If the procedure or function is a method, neither the input nor output screen file should be specified, since it is inherent in the object.

```
procedure cursor([var f: text;] x, y: integer);
```

Set cursor location for output surface file **f** in **x** and **y**.

```
function maxx([var f: text]): integer;
```

Find maximum screen location **x** in output surface file **f**.

```
function maxy([var f: text]): integer;
```

Find maximum screen location **y** in output surface file **f**.

```
procedure home[(var f: text)];
```

Send cursor to 1,1 location (upper left of screen) in output surface file **f**.

```
procedure del[(var f: text)];
```

Back up cursor by one character in output surface file **f**, and erase character at that location. If the cursor is at the left side of the screen, and automatic mode is on, the cursor will be moved up one line, and to the right of screen. If the cursor is at the top of the screen, extreme left, then the screen will be scrolled down one line, and the cursor moves to the right side of the screen.

```
procedure up[(var f: text)];
```

Move cursor up one line in output surface file **f**. If the cursor is already at the top line, and automatic mode is on, the screen will be scrolled down, and the cursor remains at the same position.

```
procedure down[(var f: text)];
```

Move cursor down one line in output surface file **f**. If the cursor is already at the bottom line, and automatic mode is on, the screen will be scrolled up, and the cursor remains at the same position.

```
procedure left[(var f: text)];
```

Back up cursor by one character in output surface file **f**. If the cursor is at the left side of the screen, and automatic mode is on, the cursor will be moved up one line, and to the right of screen. If the cursor is at the top of the screen, left, then the screen will be scrolled down one line, and the cursor moves to the right side of the screen.

```
procedure right[(var f: text)];
```

Move forward by one character in output surface file **f**. If the cursor is at the right side of the screen, and automatic mode is on, the cursor will be moved down one line, and to the left of screen. If the cursor is at the bottom of the screen, right, then the screen will be scrolled up one line, and the cursor moves to the left side of the screen.

```
procedure blink([var f: text;] e: boolean);
```

If **e** is true, causes all further characters written to the screen to appear in blinking text in output surface file **f**.

```
procedure reverse([var f: text;] e: boolean);
```

If **e** is true, causes all further characters written to the screen to appear in reverse text in output surface file **f**.

procedure underline([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in underlined text in output surface file **f**.

procedure superscript([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in superscript text in output surface file **f**.

procedure subscript([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in subscript text in output surface file **f**.

procedure italic([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in italic text in output surface file **f**.

procedure bold([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in bold text in output surface file **f**.

procedure strikeout([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in strikeout text in output surface file **f**.

procedure standout([var f: text;] e: boolean);

If **e** is true, causes all further characters written to the screen to appear in standout text in output surface file **f**. Standout is assigned to the first mode possible from the following order:

Reverse.
Underline.
Bold
Italic.
Strikeout.
Blink.

If none of those modes are available, standout is a no-op.

procedure fcolor([var f: text;] c: color);

Sets the foreground, or text color, to the color **c** in output surface file **f**.

```
procedure bcolor([var f: text;] c: color);
```

Sets the background, or space color, to the color **c** in output surface file **f**.

```
procedure auto([var f: text;] e: boolean);
```

Turns automatic mode on or off, according to **e** in output surface file **f**.

```
procedure curvis([var f: text;] e: boolean);
```

Turns cursor visibility on or off, according to **e** in output surface file **f**.

```
procedure scroll([var f: text;] x, y: integer);
```

Scroll in arbitrary directions. The screen is scrolled according to the differences in **x** and **y**. Uncovered areas on the screen appear in the current background color.

```
function curx([var f: text]): integer;
```

Find the current x location of the cursor in output surface file **f**.

```
function cury([var f: text]): integer;
```

Find the current y location of the cursor in output surface file **f**.

```
function curbnd([var f: text]): boolean;
```

Check cursor in bounds. Returns true if the cursor is currently within the bounds of the screen in output surface file **f**.

```
procedure select([var f: text;] u, d: integer);
```

Select buffer to update and display. Selects the active buffer for update **u**, and display **d** in output surface file **f**. The update buffer will receive the result of all writes. The display buffer will be shown on screen.

```
procedure event([var f: text;] var er: evtrec);
```

Get next event. Retrieves the next event from the input queue from the terminal input file **f** to event record **er**. If there is no event ready, the program will wait.

Note that specification of the event record is optional only for the terminal class method.

```
procedure timer([var f: text;] i: timhan; t: integer; r: boolean);
```

Set timer active in output surface file **f**. The timer **i** will be set to run for time **t**. If the repeat flag **r** is set, then the timer will automatically repeat when the time expires. If the timer is already in use, then it will cease its current timing, and perform the new time.

```
procedure killtimer([var f: text;] i: timhan);
```

Stop timer in output surface file **f**. Stops the timer **i**. If the timer is not active, no error is reported.

```
function mouse([var f: text]): mounum;
```

Returns the number of mice in output surface file **f**.

```
function mousebutton([var f: text;] m: mouhan): moubut;
```

Returns the number of buttons on a given mouse **m** in output surface file **f**.

```
function joystick([var f: text]): joynum;
```

Returns the number of joysticks in the system in output surface file **f**.

```
function joybutton([var f: text;] j: joyhan): joybtn;
```

Returns the number of buttons on the given joystick **j** in output surface file **f**.

```
function joyaxis([var f: text;] j: joyhan): joyaxn;
```

Returns the number of axes on the given joystick **j** in output surface file **f**.

```
procedure settab([var f: text;] t: integer);
```

Set new tab. Sets a new tab location at **t** in output surface file **f**.

```
procedure restab([var f: text;] t: integer);
```

Reset tab. Removes the tab at location **t** in output surface file **f**. If there is not a tab set there, it is not an error.

```
procedure clrtab([var f: text]);
```

Clear all tabs. All tabs are removed from the tabbing table in output surface file **f**.

```
function funkey([var f: text]): funky;
```

Returns the number of function keys available in output surface file **f**.

```
function frametimer([var f: text;] e: boolean);
```

Enables or disables the framing timer in output surface file **f**. If **e** is true, the frame timer is enabled, otherwise disabled. The frame timer gives frame timer events, which occur approximately on each refresh of the display screen. It may be tied to the refresh hardware, or may be simulated via a timer.


```
procedure autohold([var f: text;] e: boolean);
```

Sets the state of automatic hold in output surface file **f**. If **e** is true, autohold is enabled, otherwise disabled. Autohold determines if the window will exit immediately if the program self terminates. If an exit was not ordered via the user interface, the display is held until it is.

```
procedure wrtstr([var f: text;] s: string);
```

Writes the string **s** directly to the output surface file **f**. No control character interpretation is done. This procedure is used to perform efficient writes to the display surface without per-character overhead.

It is an error to call this routine when auto is enabled.

F Annex F: Graphical Interface Library

The graphical library **gralib** extends the **trmlib** model by adding graphical output procedures.

Since it is completely upward compatible with **trmlib**, and standard ISO 7185 serial output modes, any program from standard Pascal, or **trmlib** compliant Pascaline will run under **gralib**. Even in the most advanced modes of **gralib**, ordinary Pascal write statements can still be used to output text, so all of the output formatting procedures you are used to will still work.

gralib will handle any graphics task. Besides drawing features, it supports double page animation. Combined with the sound library **sndlib**, full graphical games are possible.

F.1 Terminal model

gralib emulates **trmlib** by setting up a "character grid" across the pixel based screen. Each "cell" on the grid matches the pixel height and width of a character. The character font is set to a fixed font by default when **gralib** starts. This gives every character in the font the same height and width. Finally, the text drawing mode is set such that both the foreground (the inside of the characters) and the background (the space behind the characters) are drawn, overwriting any content of each character cell as it is written.

This mode can be kept, while drawing other figures on the same text surface. Alternately, the cursor can be set to any arbitrary pixel on the screen, and text written anywhere, down to the pixel. Also, the font can be changed to a proportional one for a more pleasing look. Because automatic mode relies on characters being neatly placed on the grid, it must be turned off before the cursor leaves the grid or becomes a non-fixed font.

F.2 Graphics Coordinates

The total size of the graphics screen is found by **maxxg** and **maxyg**, which return the maximum pixel index in *x* and *y*. The pixel coordinates on the screen are from 1,1 to **maxxg(f)**, **maxyg(f)**. The cursor can be set to any pixel position by **cursorg(f, x, y)**. The current location of the cursor in pixel terms is found by **curxg** and **curyg**.

F.3 Character Drawing

There can be any number of fonts available on the system, including both fixed, and proportional fonts that vary in the width of characters. The number of fonts in the system can be found with the **font** function. Fonts are chosen by logical number with **font(f, c)**, where *c* is the font code, 1 to **font**. Of course, you don't know what font each code stands for. There are two methods. The first is the standard font codes. Standard fonts are numbers for commonly used fonts in the system. These are just commonly available fonts the program may need.

Font 1: Terminal Font

This is the default font set up by **gralib** when it starts. It's a fixed font. It also cannot be superscripted, subscripted, bold or italic, because these modes change the size of the font.

Font 2: Book Font

This font has serifs, and is good for general purpose text such as what this paragraph is written in. This is the most common proportional font.

Font 3: Sign Font

This font has no serifs (sans serif), and is best for headings, titles and similar uses. It gets its name from the road and other signs it is used on. It's a proportional font.

Font 4: Technical Font

The technical font is a fixed font that is guaranteed to be able to scale to any arbitrary size. This font is used to label drawings and engineering documents. Before the technology existed to create arbitrary fonts in any point size, the technical font was done with stroked vector graphics. Now, it is more likely to be equivalent to the sign font.

Beyond the standard fonts, the name of an installed font can be found by `font- nam(f, fc, fns)`, where `fc` is the font code, 1 to font, and `fns` returns the descriptive string for the font, like "Helvetica" or similar. It isn't really intended that the program try to match font name strings against font strings from this call. Rather, the application should use the standard fonts by default, then present the system fonts, by name to the user, and let them choose one of them.

The size of each character is set by `fontsiz(f, n)`, where `n` is the height of the font in pixels. The reason character sizes are set by their height is because proportional fonts vary in the width of the character. The height never varies. The current height of the font is found by `chrsizy`. Its width is found with `chrsizx`. Of course, we just got through saying that fonts have no standard width! When a proportional font is active, `chrsizx` returns the width of a space in that font, which is always as wide or wider than the widest character in that font.

Besides the basic font, extra space can be added between lines (known as "leading" in typography, for the lead strips used between type lines) with `chrspcy(f, n)`. `n` is the number of pixels of extra space to add between lines. Extra space between characters is added with `chrspcx(f, n)`, where `n` is the number of pixels of extra space to add between characters.

The graphical cursor is placed at the upper left of the character box for the next character to be drawn. Often, we need to know exactly where the character will rest if drawn on a line. The offset from the top of the character box to the baseline of the text is found with the function `baseline`.

In typography, fonts and characters are measured by points, of which there are 28.35 points per centimeter. Point measurement implies that you know the exact size of objects drawn on the screen. This can be determined by the functions `dpmx` and `dpmy`, which return the "dots per meter" or pixels in one meter for both `x` and `y`. The reason it can be two different measures for the two different axes is that the display may not have square pixels or a 1:1 aspect ratio.

To find a given point size in terms of the height needed for the character, it is found by:

$$\text{dpmy}/2835 * \text{point size}$$

The screen aspect ratio can also be found from these calls, it is:

$$\text{dpmx}/\text{dpmy}$$

F.4 String Sizes and Kerning

When writing text to the screen in proportional font, we often need to know exactly how much space the string will take up on the screen, down to the pixel. This amount of space can change not only because of the variable width of proportional fonts, but also because of an effect called "kerning". When a string of characters is draw together, the system can apply "kerning" to characters in the string. Kerning means to fit the individual letters together, like puzzle pieces, to come up with a tighter spacing than is normally possible. For example, "A" and "V" together as "AV" can typically be kerned together to take less space because they can overlap. To find the exact number of pixels in x that a string will occupy, the function `strsiz(f, s)` is used, where s is the string. The size of the string in y does not change, and can be found with `chrsizy`. To find the exact position, in pixels offset from the beginning of the string in x, of a given character, use `chrpos(f, s, n)`, where s is the string, and n is the index of the character you want the x offset of.

F.5 Justification

Justification is the spreading of spacing through a string of characters to fit a given space. You can find out if the string will fit into the space with `strsiz(f, s)`, and checking if the resulting pixels required are less than or equal to the space they will occupy as justified. The character string is written in justified mode with `writejust(f, s, n)`, where s is the string to write, and n is the number of pixels to fit it in. If the number of pixels allowed for is not enough, the string will be larger than the requested number. To find the offset, in x, of a given justified character, use `justpos(f, s, p, n)`, where s is the string, p is the offset of the character you are interested in, and n is the total number of pixels to fit the string in, as in `writejust`.

F.6 Effects

`gralib` expands the effects in `trmlib` a bit. For smaller character baselines, use `condensed(f, b)`. For larger character baselines, use `extended(f, b)`.

In addition to normal bold, there are also `light(f, b)`, `xlight(f, b)`, and `xbold(f, b)` effects. for lighter than normal, extra light, and extra bold modes.

Characters can have an embossed look with `hollow(f, b)` and `raised(f, b)`. `hollow` makes the character look sunken, and `raised` makes it look as if coming off the page.

F.7 Tabs

`gralib` extends the character level of tabbing in `trmlib` with procedures that can set tabs on an individual pixel. `settabg(f, x)` sets a tab at the pixel x. `restabg(f, x)` resets the tab at pixel x. The `trmlib` procedure `clrtab(f)` clears all tabs, including pixel level tabs.

F.8 Colors

The simple eight colors from `trmlib` are still available, but there are two new calls that let you get access to the full range of colors an advanced graphic system provides. `fcolorg(f, r, g, b)` sets the foreground color from values of red r, green g, and blue b. Similarly, `bcolorg(f, r, g, b)` sets the background color from rgb values. The values of the colors are ratioed. This means that instead of an absolute number, the possible colors are ratioed from 0 to `maxint`, where 0 is dark, and `maxint` is saturated color. Color ratios allow the true color range implemented by the system to be hidden.

F.9 Drawing Modes

When colors, background or foreground, are drawn on the screen, they can be in a number of mixing modes. Mixing modes govern how the new color is laid over the old. The modes are mutually exclusive, so the setting of a new mix mode for a given color deactivates the old mode. If the old color is simply to be overwritten, then `fover(f)` or `bover(f)` is appropriate. If the new color is to be xor'ed with the old color, then `fxor(f)` or `bxor(f)` is used. If the new color is to be ignored, leaving the old color underneath intact, use `finvis(f)` or `binvis(f)`.

There might not seem to be a use for an "invisible" color, but there are actually several uses. First, if the background is set invisible, the text can be overlaid on another pattern. In fact, this is the most common drawing mode, and most programmers will prefer to turn the background off and lay the backgrounds themselves. Similarly, leaving the background on, then setting the foreground invisible can be used to "stencil" letters with arbitrary patterns inside the letters.

Xor mode is good for several things. First, if a series of figures, say rectangles, are to be laid, but the intersections between them are to be left visible, xor is the right mode. Second, xor can be used to place, and then remove a pattern, even a complex one, easily. This is used to allow the user to place figures by dragging them with the mouse to a new location. It can be used to draw "rubber band" boxes around selections.

Xor mode has two rules of interest:

1. Any two colors, even the same colors, xored together, will give a third color, with the exception of black.
2. Having xored a drawing into the viewplane, xoring the same color and drawing into the viewplane again will restore the old drawing.

Those rules actually make xor good for a lot of tricks and effects. However, there is a warning to go with the fun of xor. Xor doesn't like inaccuracy. Xoring something back off the screen has to be done the same way it was put on, with the same parameters. Also, the mode of drawing in `gralib` is not very friendly to xor mode. Drawing a rectangle, for example, will result in "corner errors", because the rectangles are built from lines, and lines start and end at the coordinates of the box corners. Because one line starts at the same point another ends, they will xor mix together. The result is a recognizable point of off-color on each corner.

The best way to use xor is to xor a single figure onto the screen, then xor it back off. If complex figures are to be xored on and off the screen, be prepared to run the xor backwards, that is, from the last figure drawn back to the first.

F.10 Drawing Graphics

A graphics element that is not a character is referred to as a "figure". What we have tried to do is provide a small toolset, that does not include figures that you could reasonably construct from the lower level figures. For example, there is no circle figure in `gralib`, because that is simply a special case of ellipse.

A parameter that applies to almost all figures is the width of lines. The width of a line usually defaults to 1, but may be more if a single line is unusable on the current display. This can easily happen on a very high resolution display. The line width can be set by the procedure `linewidth(f, n)`, where `n` is the

number of pixels for the line to use. There is no limit on the width of a line, and in fact, lines are a defacto way to draw arbitrary angle filled rectangles.

When the line width is set to an even number of pixels, an effect called "even line uncertainty" exists. If you draw a line between two points, and the line width is say, 2, one of the 2 pixels is going to be on the line, and the other could be to either side of it. It literally depends on how the math happens to round off.

To prevent this, its recommended that the line width always be set to an odd number.

F.11 Figures

The fundamental figure in graphics is the line. A line is drawn, in the current linewidth, by `line(f, x1, y1, x2, y2)`. A rectangle is drawn with `rect(f, x1, y1, x2, y2)`, whose borders have the current linewidth. A filled rectangle is drawn with `frect(f, x1, y1, x2, y2)`, whose interior is the foreground color. An ellipse is drawn with `ellipse(f, x1, y1, x2, y2)`. The x and y parameters define a rectangle that contains the figure. The procedure `fellipse(f, x1, y1, x2, y2)` draws a filled ellipse.

The procedure `arc(f, x1, y1, x2, y2, rs, re)` draws an arc line around the ellipse formed by the rectangle formed by the x and y parameters. The start and end points of the arc are described by a special ratio notation that gives the angle. The angles in a 360 degree circle are described by a number from 0 to `maxint`. 0 is the 0 degree, or top center, of the ellipse. The angles around the circle clockwise then go from 0 to `maxint`, at which time a full 360 degrees have been traversed. For example, `maxint div 2` is 180 degrees, `maxint div 4` is 90 degrees, etc. The parameter `rs` gives the angle where the arc starts. The parameter `re` gives the angle where the arc ends. Arcs can be specified to cross `maxint` back to zero, or use negative degrees, or any combination.

The procedure `farc(f, x1, y1, x2, y2, rs, re)` draws a filled arc. The procedure `fcord(f, x1, y1, x2, y2, rs, re)` draws a filled cord (a line bisecting the circle).

Rectangles with rounded corners can be drawn with `rrect(f, x1, y1, x2, y2, xw, yw)`. The x and y parameters describe the bounding box, The `xw` and `yw` parameters describe the size of the ellipses that are placed in the corners to round the edges of the box. To draw a filled rounded rectangle, use `frect(f, x1, y1, x2, y2, xw, yw)`.

The general purpose shape `ftriangle(f, x1, y2, x2, y2, x3, y3)` draws a filled triangle. Parting with convention, `gralib` does not give complex polygon procedures. Rather, you can build up polygons from triangles, and in any case, a high speed drawing engine in hardware would accept triangles only, so the lower level software would have to break up the polygon for you.

Single pixels can be set with `setpixel(f, x, y)`.

F.12 Predefined Pictures

A picture, or a bitmap, is defined outside the program by a drawing application. Its format is typically operating system specific. `gralib` considers pictures to be a cached resource. A picture is loaded from a file by `loadpict(f, p, fs)`. The string `fs` indicates the file name for the picture file. `p` is a logical picture number, from 1 to `n`, and indicates how you want to refer to the picture while its loaded into memory.

A logical picture is drawn onto the screen with `picture(f, p, x1, y1, x2, y2)`. The parameter `p` indicates the logical picture to draw. The `x` and `y` parameters indicate the box that the picture is to be drawn into. `gralib` will scale or stretch the picture as needed to make the picture fit into the space given.

In order to determine the parameters of a picture, such as native size and aspect, the functions `pictsize(f, p)` and `pictsizey(f, p)` are used. These give the native size of the picture in `x` and `y`, and the aspect ratio of the picture is then found with `x/y`.

F.13 Scrolling

Like `trmlib`, `gralib` can scroll in arbitrary directions. It can also scroll down to the pixel, using the call `scrollg(f, x, y)`. The parameters work the same way as the character position parameters of `scroll`, except that pixels are specified instead of characters.

F.14 Clipping

Clipping is completely automatic in `gralib`. Any figure drawn is clipped to the edges of the screen. If a figure is drawn entirely outside the screen bounds, it is completely clipped out.

F.15 Mouse Graphical Position

A new event, `etmoumovg`, exists that gives mouse movements in pixels, not just characters. The old `etmoumov` still occurs, and carries the character grid message. The `etmoumovg` message happens when the mouse moves a pixel, and the `etmoumov` message happens when the mouse moves a whole character cell. If you don't need the `etmoumov` message, you simply ignore it.

F.16 Animation

In `trmlib`, the `select` call was introduced, that switches between multiple screen buffers. This call is tailor made for double buffer animation, and yes, it works in `gralib` the same way. Double buffer animation works by having two screen buffers. You draw to one of them, and display the other one. When you are done drawing a "frame", i.e., have completed the picture in the buffer you are currently drawing, you swap the drawing and display buffers, and then you start drawing on what used to be the display buffer, clearing it if necessary.

What double buffering does is, remove any of the ongoing drawing operations from the user. Although computers do this quite fast, seeing drawing operations in progress tends to produce annoying flashing effects, sparkles and other effects during the drawing. In addition, although the worst interactions with the painting of the graphics card memory to the CRT screen have disappeared, you still get odd effects from the fact that the CRT is being swept across the image you are drawing.

`gralib` supports more than double buffering (triple, quad or better). However, the advantages of this typically diminish as the amount of data being managed grows without compensating gain in drawing speeds.

To get a really first class, rock solid display, the best thing to do is flip your buffers when the CRT display enters its retrace period. Syncing with the CRT eliminates the effects that occur when you are trying to redraw the screen while the CRT beam is painting across it. The retrace period is when the beam has finished the last lines on the screen, and will head back to the top of the screen. This gives a short time while the CRT is not doing anything, so if the buffer swap can occur within the retrace, no

effects will appear on the screen at all. Note that all of this applies to LCD screens as well, since they also scan.

The solution is the `etframe` message. `etframe` is sent to you when the CRT enters refresh. If the system does not allow notification for retrace, the `etframe` message is generated by a timer that keeps either the screen refresh rate, or 30 cycles per second if that cannot be determined.

F.17 Declarations

The declarations for `gralib` are very similar to `trmlib`, with the addition of the mouse move graphical event, and the standard font codes.

```
const
```

```
maxtim = 10; { maximum number of timers available }
maxbuf = 10; { maximum number of buffers available }
font_term = 1; { terminal font }
font_book = 2; { book font }
font_sign = 3; { sign font }
font_tech = 4; { technical font (vector font) }
```

```
type
```

```
{ colors displayable in text mode }
color = (black, white, red, green, blue, cyan, yellow, magenta);
joyhan = 1..4; { joystick handles }
joynum = 0..4; { number of joysticks }
joybut = 1..4; { joystick buttons }
joybtn = 0..4; { joystick number of buttons }
joyaxn = 0..3; { joystick axes }
mounum = 0..4; { number of mice }
mouhan = 1..4; { mouse handles }
moubut = 1..4; { mouse buttons }
timhan = 1..maxtim; { timer handle }
funky  = 1..100; { function keys }
```

```
{ events }
```

```
evtcod = (etchar,      { ANSI character returned }
          etup,        { cursor up one line }
          etdown,      { down one line }
          etleft,      { left one character }
          etright,     { right one character }
          etleftw,     { left one word }
          etrightw,    { right one word }
          ethome,      { home of document }
          ethomes,     { home of screen }
          ethomel,     { home of line }
          etend,       { end of document }
          etends,      { end of screen }
          etendl,      { end of line }
          etscrll,     { scroll left one character }
          etscrr,      { scroll right one character }
          etscru,      { scroll up one line }
          etscrd,      { scroll down one line }
          etpagd,      { page down }
          etpagu,      { page up }
          ettab,       { tab }
          etenter,     { enter line }
          etinsert,    { insert block }
          etinsertl,   { insert line }
          etinserttt,  { insert toggle }
          etdel,       { delete block }
```

```

    etdell,      { delete line }
    etdelcf,    { delete character forward }
    etdelcb,    { delete character backward }
    etcopy,     { copy block }
    etcopyl,    { copy line }
    etcan,      { cancel current operation }
    etstop,     { stop current operation }
    etcont,     { continue current operation }
    etprint,    { print document }
    etprintb,   { print block }
    etprints,   { print screen }
    etfun,      { function key }
    etmenu,     { display menu }
    etmouba,    { mouse button assertion }
    etmoubd,    { mouse button deassertion }
    etmoumov,   { mouse move }
    ettim,      { timer matures }
    etjoyba,    { joystick button assertion }
    etjoybd,    { joystick button deassertion }
    etjoymov,   { joystick move }
    etterm,     { terminate program }
    etmoumovg,  { mouse move graphical }
    etframe);   { frame sync }

{ event record }

evtrec = record

    case etype: evtcod of { event type }

        { ANSI character returned }
        etchar: (char:          char);
        { timer handle that matured }
        ettim:  (timnum:          timhan);
        etmoumov: (mmoun:          mouhan; { mouse number }
                  moupx, moupy:      integer); { mouse movement }
        etmouba: (amoun:          mouhan; { mouse handle }
                  amoubn:         moubut); { button number }
        etmoubd: (dmoun:          mouhan; { mouse handle }
                  dmoubn:         moubut); { button number }
        etjoyba: (ajoyn:          joyhan; { joystick number }
                  ajoybn:         joybut); { button number }
        etjoybd: (djoyn:          joyhan; { joystick number }
                  djoybn:         joybut); { button number }
        etjoymov: (mjoyn:          joyhan; { joystick number }
                   joypx, joypy, joypz: integer); { joystick
                                                         coordinates }
        etfun:   (fkey:          funky); { function key }
        etmoumovg: (mmoung:       mouhan; { mouse number }
                   moupxg, moupyg: integer); { mouse movement }
        etup, etdown, etleft, etright, etleftw, etrightw, ethome,

```

```
ethomes, ethomel, etend, etends, etendl, etscr1, etscrr,
etscru, etscrd, etpagd, etpagu, ettab, etenter, etinsert,
etinsertl, etinsertt, etdel, etdell, etdelcf, etdelcb, etcopy,
etcopyl, etcan, etstop, etcont, etprint, etprintb, etprints,
etmenu, etterm, etframe: ( ); { normal events }
```

```
{ end }
```

```
end;
```

F.18 Graphical Terminal Objects

All of the procedures, functions and other declarations in **gralib** are also available in a graphical terminal object. of the form:

```
Class Graphicalterminal;

var input, output: text; { input and output files for this screen }
    er: evtrec;           { event record }

{ procedures and functions from gralib }
```

.
The complete catalog of procedures and functions from **gralib** are available as methods in the **graphicalterminal** class. There are minor differences, which are detailed with the procedure and function descriptions in E.16.

When a **graphicalterminal** object is instantiated, it's input and output files are automatically linked to the standard **input** and **output** files of the program. Thus, where specifying the input and output files in **trmlib** procedures and functions is optional, in methods of the terminal object these files **must not** be specified.

The input and output files used by the terminal object are made available as class members **input** and **output**. This allows them to be used in non-method procedures (such as **read** and **write**).

A terminal object also contains an event record, so it is not necessary to specify an external event record in the **event** procedure.

A **graphicalterminal** object can be created as follows:

```
program p;

joins gralib;

var ti: instance of gralib.graphicalterminal;

begin

    ti.home; { send cursor to home position }
    writeln(ti.output, 'hello, terminal world');
    repeat { event loop }

        ti.event; { get next event }
        { process events }

    until ti.event = etterm { loop until program cancelled }

end.
```

Where **terminal** is the terminal object. Terminal objects can be instantiated statically or dynamically.

Terminal objects contain their own state for the following:

6. Location of cursor.
7. Current attributes and colors.
8. Current font and size.
9. Buffer working/display status.
10. Tab stops.
11. Timer numbers.
12. Cached pictures.

However, each terminal object gets the complete state of the module **gralib** when it is created.

Having each terminal object possess its own state means that each object can be writing to its own section of the screen in its own mode. Each object can even have it's own private screen buffer, but this must be specifically selected using the **select** procedure.

When multiple terminal objects exist, and all have the cursor visibility on, the cursor will remain at the last place it was specifically directed to. This can mean that the cursor will rapidly appear to swap back and forth between the active drawing terminal objects. This can be improved by only selecting one of the terminal objects as having a visible cursor. Typically, a the cursor should be placed to attract user attention to where text is being entered, or if none is being entered, it should be turned off.

Note that **gralib** also exposes a terminal object that is identical to the **trmlib terminal** object. It features the same calls as **GraphicalTerminal**, but does not allow access to the advanced features in **GraphicalTerminal**.

F.19 Procedures and functions in gralib

See **trmlib** for the basic text functions. These are all implemented in **gralib**.

For all of the following calls, If the screen file **f** is not present, the default is the standard output file. If the procedure or function is a method, neither the input nor output screen file should be specified, since it is inherent in the object.

function maxxg[(var f: text)]: integer;

Returns the maximum pixel index **x** in output surface file **f**.

function maxyg[(var f: text)]: integer;

Returns the maximum pixel index **y** in output surface file **f**.

function curxg[(var f: text)]: integer;

Returns the current location of the cursor, in pixel units, for **x** in output surface file **f**.

function curyg[(var f: text)]: integer;

Returns the current location of the cursor, in pixel units, for **y** in output surface file **f**.

procedure line[(var f: text;] x1, y1, x2, y2: integer);

Draw line. Draws a line between the point **x1,y1** to **x2,y2**, using the current line width, color and mode in output surface file **f**.

procedure linewidth[(var f: text;] w: integer);

Set line width. Sets the line drawing width at **w** pixels wide in output surface file **f**. Use of an odd number of pixels is recommended.

procedure rect[(var f: text;] x1, y1, x2, y2: integer);

Draw rectangle. Draws a rectangle whose opposite corners are **x1,y1** and **x2,y2**. Uses the current line width, color and mode in output surface file **f**.

procedure frect[(var f: text;] x1, y1, x2, y2: integer);

Draw filled rectangle. Draws a solid rectangle, whose opposite corners are **x1,y1** and **x2,y2**. Uses the current color and mode.

procedure rrect[(var f: text;] x1, y1, x2, y2, xs, ys: integer);

Draw rounded rectangle. Draws a rectangle with rounded corners. The opposite corners of the bounding box are specified as **x1,y1** to **x2,y2**. The ellipses that specify the rounded corners are **xs** and **ys**, which specify the width and height of the ellipse. Uses the current line width, color and mode.

procedure frrect[(var f: text;] x1, y1, x2, y2, xs, ys: integer);

Draw filled rounded rectangle. Draws a rectangle with rounded corners. The opposite corners of the bounding box are specified as **x1,y1** to **x2,y2**. The ellipses that specify the rounded

corners are **xs** and **ys**, which specify the width and height of the ellipse. Uses the current color and mode.

```
procedure ellipse([var f: text;] x1, y1, x2, y2: integer);
```

Draw an ellipse. Draws an ellipse bonded by a box whose opposite corners are **x1,y1** to **x2,y2**. Uses the current line width, color and mode.

```
procedure fellipse([var f: text;] x1, y1, x2, y2: integer);
```

Draw a filled ellipse. Draws a solid ellipse bonded by a box whose opposite corners are **x1,y1** to **x2,y2**. Uses the current color and mode.

```
procedure arc([var f: text;] x1, y1, x2, y2, sa, ea: integer);
```

Draw an arc. Draws an arc on an ellipse, whose bounding box is **x1,y1** to **x2,y2**. The arc starts on the ellipse from the angle **sa**, to the angle **ea**. The angles are given in 360 degree to maxint ratio form. Uses the current color and mode.

```
procedure farc([var f: text;] x1, y1, x2, y2, sa, ea: integer);
```

Draw a filled arc. Draws a filled arc on an ellipse, whose bounding box is **x1,y1** to **x2,y2**. The arc starts on the ellipse from the angle **sa**, to the angle **ea**. The angles are given in 360 degree to maxint ratio form. Uses the current color and mode.

```
procedure fchord([var f: text;] x1, y1, x2, y2, sa, ea: integer);
```

Draw a filled cord. Draws a filled cord on an ellipse, whose bounding box is **x1,y1** to **x2,y2**. The cord starts on the ellipse from the angle **sa**, to the angle **ea**. The angles are given in 360 degree to maxint ratio form. Uses the current color and mode.

```
procedure ftriangle([var f: text;] x1, y1, x2, y2, x3, y3: integer);
```

Draw a filled triangle. Draws a filled triangle given the three points **x1,y1**, **x2,y2**, and **x3,y3**. Uses the current color and mode.

```
procedure cursorg([var f: text;] x, y: integer);
```

Position the cursor graphically. Moves the cursor to the pixel position **x** and **y**.

```
function baseline([var f: text]): integer;
```

Find baseline of current font. Finds the baseline, or line on which all characters of the current font sit upon, in terms of offset from the character bounding box origin in **y**.

```
procedure setpixel([var f: text;] x, y: integer);
```

Set single pixel. Sets a single pixel at the point **x,y**, using the current color and mode.

```
procedure fover[(var f: text)];
```

Set overwrite mode foreground. Sets all new drawing to overwrite old colors on the foreground.

```
procedure bovver[(var f: text)];
```

Set overwrite mode background. Sets all new drawing to overwrite old colors on the background.

```
procedure finvis[(var f: text)];
```

Set invisible mode foreground. Sets all new drawing to discard colors on the foreground.

```
procedure binvis[(var f: text)];
```

Set invisible mode background. Sets all new drawing to discard colors on the background.

```
procedure fxor[(var f: text)];
```

Set xor mode foreground. Sets all new drawing to xor old colors with new colors on the foreground.

```
procedure bxor[(var f: text)];
```

Set xor mode background. Sets all new drawing to xor old colors with new colors on the background.

```
function chrsize[(var f: text)]: integer;
```

Find character size in x. Returns the x size, in pixels, of the characters in the current font. If the font is proportional, its x size will vary per character. The size will then be the space character, which is guaranteed to be the widest character in the font.

```
function chrsizey[(var f: text)]: integer;
```

Find character size in y. Returns the y size, in pixels, of the characters in the current font.

```
function fonts[(var f: text)]: integer;
```

Find number of fonts. Returns the number of fonts installed on the system.

```
procedure font([(var f: text;] fc: integer);
```

Select logical font. Selects a font by logical number **fc**, where **fc** is 1..fonts.

```
procedure fontnam([(var f: text;] fc: integer; var fns: string);
```

Find name of logical font. Returns the name of the logical font **fc** in the string **fns**.


```
procedure fontsize([var f: text;] s: integer);
```

Set font size. Sets the height of the current font, in pixels.

```
procedure chrspcy([var f: text;] s: integer);
```

Set character y spacing. Sets the line to line spacing to *s*, in pixels.

```
procedure chrspcx([var f: text;] s: integer);
```

Set character x spacing. Sets the character to character spacing *s*, in pixels.

```
function dpmx([var f: text]): integer;
```

Find dots per meter *x*. Finds the dots per meter in *x* of the current display device.

```
function dpmy([var f: text]): integer;
```

Find dots per meter *y*. Finds the dots per meter in *y* of the current display device.

```
function strsiz([var f: text;] s: string): integer;
```

Find pixel size of string. Finds the total *x* size of the string *s*, in pixels. Accounts for all sizes and spacing.

```
function chrpos([var f: text;] s: string; p: integer): integer;
```

Find pixel offset of character. Finds the pixel offset from the start of a string *s* in terms of *x* pixels, to the character by the index *p*.

```
procedure writejust([var f: text;] s: string; n: integer);
```

Write string justified. Writes the given string *s* into the number of *x* pixels *n*. If the space is more than is required, the extra space will be distributed between the characters. If the space given is insufficient for the characters to be drawn, it will be drawn in the minimum amount of space for the entire string.

```
function justpos([var f: text;] s: string; p, n: integer): integer;
```

Find justified pixel off set of character. Finds the pixel offset from the start of a string *s*, in terms of *x* pixels, to the character by the index *p*, for a string justified to fit into *n* pixels. The rules of justification are the same as for `writejust`.

```
procedure condensed([var f: text;] e: boolean);
```

Set condensed mode. Sets the current font to occupy a shorter baseline than normal. Note that this effect may not be implemented.

procedure extended([var f: text;] e: boolean);

Set extended mode. Sets the current font to occupy a longer baseline than normal. Note that this effect may not be implemented.

procedure xlight([var f: text;] e: boolean);

Set extra light mode. Sets the current font to extra light printing. Note that this effect may not be implemented.

procedure light([var f: text;] e: boolean);

Set light mode. Sets the current font to light printing. Note that this effect may not be implemented.

procedure xbold([var f: text;] e: boolean);

Set extra bold mode. Sets the current font to extra bold printing. Note that this effect may not be implemented.

procedure hollow([var f: text;] e: boolean);

Set hollow mode. Sets the current font to hollow, or sunken look, printing. Note that this effect may not be implemented.

procedure raised([var f: text;] e: boolean);.

Set raised mode. Sets the current font to raised, or relief look, printing. Note that this effect may not be implemented.

procedure settabg([var f: text;] t: integer);

Set graphical tab. Sets a tab to the pixel **t**.

procedure restabg([var f: text;] t: integer);

Reset graphical tab. The tab at pixel **t** is removed. If no tab is set at **t**, no error will result.

procedure fcolorg([var f: text;] r, g, b: integer);

Set foreground color graphical. The foreground color is set to the red **r**, green **g** and blue **b** color. The colors are in 0..maxint ratios, with 0 = black, and maxint = saturated. The nearest color to the one given is found and set active as the foreground color. The exact color that results will depend on the total number of colors implemented in the system, and could well be black and white in a system so equipped.

procedure bcolorg([var f: text;] r, g, b: integer);.

Set background color graphical. The background color is set to the red **r**, green **g** and blue **b** color. The colors are in 0..maxint ratios, with 0 = black, and maxint = saturated. The nearest color to the one given is found and set active as the foreground color. The exact color that

results will depend on the total number of colors implemented in the system, and could well be black and white in a system so equipped.

```
procedure loadpict([var f: text;] p: integer; fn: string);
```

Load picture. Loads the picture from the filename **fn** to logical picture number **p**, which is 1..n. The file must be in a format that the system understands, and is converted to an in memory format that is optimal for the system, such as a direct match for the graphics device in use.

```
function pictsizx([var f: text;] p: integer): integer;
```

Find picture size x. Returns the size, in pixels of x, of the logical picture **p**.

```
function pictsizy([var f: text;] p: integer): integer;
```

Find picture size y. Returns the size, in pixels of y, of the logical picture **p**.

```
procedure picture([var f: text;] p: integer; x1, y1, x2, y2: integer);
```

Draw picture. The logical picture **p** is drawn into the bounding box formed by **x1,y1** to **x2,y2**. The picture is stretched or compressed as required to do this. The method used to stretch or compress may depend on how much computing power is available on the system. There is no attention paid to aspect ratio.

```
procedure delpict([var f: text;] p: integer);
```

Remove logical picture from use. The logical picture **p** is removed from the picture queue, and will no longer take up memory space.

```
procedure scrollg([var f: text;] x, y: integer);
```

Scroll in arbitrary directions. The screen is scrolled according to the differences in **x** and **y**, which are in pixels. Uncovered areas on the screen appear in the current background color.

G [Annex G: Windows Management Library](#)

manlib is completely upward compatible with **trmlib** and **gralib**. Given a single fixed screen, **manlib** subdivides the screen into virtual windows, which can be set to any size or position. A **trmlib** compatible program sees its window as a "virtual screen", and does not know that some or all of the contents of that screen may be hidden. A **manlib** aware program can participate in the benefits of a windowed environment.

G.1 [Screen Appearance](#)

The idea of windows management is to take a program that thinks it is talking to an ordinary terminal, and allow the presentation of multiple such windows within a single screen.

By default, **manlib** emulates a terminal interface for each window that is identical to the behavior of a full screen window from the programs' point of view. A standard size screen is implemented for the program of 80 characters by 25 lines (even if the real display has a different size). **manlib** accepts program writes, and places that information in a buffer that looks like the ideal screen. Then, the contents of that buffer are placed on the screen in various arrangements to complete the desktop for the user.

G.2 [Window Modes](#)

A window can be any actual size on the display. A window can also be off the display entirely, so that it accepts changes to its buffer, but does not display those changes. In addition, a window can be active, inactive, or overlapped, or even completely covered by other windows on the display.

G.3 [Buffered Mode](#)

A window comes up by default in the "buffered" mode. In buffered mode, the program has a "virtual display surface" that it writes to. **manlib** maintains this surface in memory, and maintains the actual onscreen window as a scrolled window on that. The program is unconcerned with what part of its screen is being displayed, or even if it is displayed at all. The user operates the window using the frame controls.

Buffered mode is designed to allow the program to be unconcerned with the management of the display. However, the program can set the size of the virtual display using **sizbuf[g]**. When a buffer is resized, its contents is cleared to the current background color.

The buffer is a display surface that the buffer handler keeps for the program. The buffered mode allows a program to "think" it has a window of size N, but the appearance of the window on the actual display is a window on that virtual display that is maintained by the buffer handler. Because from the programs point of view, the window size does not change, and is always the complete window, the resize events are performed transparently to the program. Similarly, the minimize and maximize events are handled for the program (see G.4). Discovery is when a window or part of a window is uncovered on the display.

When in buffered mode, the window manager is may use scroll bars to display within a screen view that is smaller than the buffer.

Although buffered mode automatically handles window status events (covered in G.4), these events are still sent through. The program normally ignores them. However, on a very fast system, this allows

a program to keep buffered mode active, but still respond to window management requests such as resize. The meaning of these events is given next.

G.4 Unbuffered Mode

The program can leave buffered mode by executing **buffer**. Unbuffered mode has no display buffer, and no default actions for events. Instead, the onscreen appearance of the window is set dynamically by the program.

When unbuffered mode is entered, the buffer for the window is discarded, and it becomes entirely up to the program to manage its own window by watching events. The size of the window no longer reflects the buffer, but instead is the size of the onscreen window. In contrast to buffered mode, this can change many times as the window is resized under user control.

Buffered mode can be reentered at any time. The contents of the buffer are cleared to spaces.

When running unbuffered, it is assumed that the program will manage the update of the on-screen display surface itself. When running unbuffered, the program handles events that are normally handled automatically.

The **etredraw** event contains the limits of an update rectangle that shows what part of the client area needs to be redrawn on the screen. The program can ignore the update rectangle, and simply redraw the entire screen, or it may only redraw the figures that overlap the update rectangle. The latter is usually more time efficient. **etredraw** gives its update rectangle in character coordinates. There is also a pixel version of that, **etredrawg**, which uses pixel coordinates. On a graphical system, both messages will be sent, and they duplicate each other. The character coordinate **etredraw** will contain a rectangle of characters that at least covers the pixels in a graphical system. Since **etredraw** and **etredrawg** duplicate each other, only one of them should be obeyed, and the other ignored.

The **etresize** event indicates that the onscreen window has changed its size. Its purpose is to let the program update any calculations based on the size of the window. This event can be ignored, used to update stored **maxx[g]** and **maxy[g]** values, or it could even cause the entire arrangement of the screen to be reformed.

etresize is not normally used to redraw areas of the screen. If a resize event causes new screen area to be uncovered, then one or more **etredraw** events will also be sent. There is no way to determine exactly which redraw operations correspond to the resize event, so some redundancy is possible with applications that repack their window's contents on a resize.

The **etmin** event indicates that the window has been minimized. When a window is minimized, it will not be displayed, and won't receive **etredraw** events. On the desktop, the window is represented by a small icon. An alternative form of minimization is for the window to receive an **etmin** message, followed by a **etresize** message, and continued **etredraw** messages. This is a hint for the window to display vital information in a much smaller format that fits into the minimized icon.

The **etmax** event indicates that the window has been maximized, or covers the entire desktop. Typically, this requires no action on the part of the program.

The **etnorm** event indicates a normal window mode has been resumed from **etmin** or **etmax**.

G.5 Defacto transparency

Using unbuffered mode, it is possible to achieve window transparency on most systems. Transparency means that some parts of the window will show through to the display surface underneath the window. This can be used to implement advanced effects like non-rectangular windows, and it is the basis for widgets (which appear in **widlib**). For defacto transparency to work, the drawing order of windows must be tightly controlled.

G.6 Delayed Window Display

When a window is created, it is not displayed until an actual write operation is made to it. This allows the window to be changed in configuration by the program before it is displayed, and prevents the window being rapidly changed on screen as that happens. For example, the program might want to take the window out of buffered mode, change its size, remove the frame, or add menus.

G.7 Window Frames

Under most window systems, each window has a "window frame", which has various window management functions. These include:

- Move bars to resize and move the window.
- A title bar that indicates what program is running in the window.
- A "minimize" button.
- A "maximize" button.
- A "menu" bar.

When a window is in buffered mode, none of the frame features are under the control of the program, but instead are managed by the buffering software.

The appearance, or even the existence, of any of these items can be customized by the program. The frame can be enabled or disabled by **frame**. The title can be set by **title**. The title, minimize and maximize buttons are considered part of the "system bar", which can be enabled or disabled by **sysbar**. The resize bars are enabled or disabled by **sizable**.

The frame control functions **frame**, **title**, **sysbar** and **sizable** are optional within an implementation. The system may not allow an application to, for example, disable its sizing bars, or it may not have a set of system controls on the window. The program must be ready for these commands not to have an effect. For example, after the sizeable function is set false, it should still be ready to receive a new size. If, for example, the size of the graphic to be presented is a fixed size, it would be filled out or clipped to fit the resulting window.

G.8 Multiple Windows

If the output file appears in the program header, then that is output as the main window by default. A window can also be explicitly opened by **openwin**. When a new window is opened, it is placed on the desktop with the other windows. The text files **inw** and **outw** specify the input and output files attached to the window. The input file receives all user input and events from the window, and all of the write and other drawing operations are sent to the output file. The **id** is a number, from 1 to N, that specifies the logical window. The logical window number 1 is reserved for the standard input and output pair, even if it is not specifically opened, since it can be opened by the program at any time.

The output side of a window must always be unique, but the input side can be shared. Multiple windows can be attached to a single input file, and that input file will receive all of the input and events from all attached windows. The logical window number is returned with each event, so that the source of the input or event can be determined.

To completely decouple separate windows, another thread is used to run the separate window. This can be done, for example, to create a widget library that does not depend on the event handler of the calling module.

Windows can be closed using the standard Pascaline **close** call. Only the output side of a window is used to close that window. The input side is automatically closed when there is no longer a window that references it.

G.9 Parent/Child Windows

Windows systems are tree structured, and each window is a child of a parent window, with the exception of a "root" or master window, which is usually the window that contains the entire desktop. So far, we have been creating windows that live side by side on the desktop, and have the desktop as their parents. However, it is possible to nest windows within each other.

A window is opened as a child by **openwin**, which is the same **openwin** call with a parent specified. The parent file **par** is the text file that is attached to the output window of the parent. To be a child a window means to move as one within it. It maintains its position within the parent, even as the parent itself moves. It always is in front of the Z ordering for the parent, but behind any other siblings of the parent. It is minimized, maximized and closed with the parent.

All windows have their position given in relative terms to the parent's client area origin. Any position operation is also relative to the parent.

The common use of the parent is to create "sibling" windows, or windows equivalent to the default program window in status, that are positioned independently within the parent. A program starts as the child of an external parent window. This could be an actual program, such as a program acting as a manager, or it could be the desktop root. There is no difference between these for the program.

G.10 Moving and Sizing Windows

Moving a window is done with the **setpos[g]** procedure. The position is given in parent coordinates. When a window is on the desktop, it is necessary to find the size of the desktop, which is found with **scnsiz[g]**. When the desktop size is returned for character sizes, this is calculated using the current character sizes for the window. This is for comparative purposes only. The desktop may use a completely different set of characters and sizes.

Sizing a window is done with **setsiz** procedure. This sets the size of the entire window, and so does not indicate the resulting size of its client area. To find out what window size is needed for a given client area, before actually sizing the window, the function **winclient[g]**. It returns the window size needed to contain that client.

The mode set declaration is:

```
{ windows mode sets }
winmod = (wmframe,    { frame on/off }
          wmsize,     { size bars on/off }
          wmsysbar); { system bar on/off }
winmodset = set of winmod;
```

To find the current size of a window, in parent terms, the procedure **getsiz[g]** is used. This procedure is useful when you need to find the size of a window on the desktop.

G.11 Z Ordering

The Z ordering, or back to front ordering of windows, is determined by default to be newest created windows to the front, oldest to the back. This order can be modified by the users when they select windows towards the back to come to the front. The program can also reorder windows at will by sending them to the front or back. Note that the Z ordering is always relative within a parent.

To send a window to the front, the call **front** is used. To send a window to the back, the call **back** is used. To achieve a specific order within a set of windows, they should be send to the front in the opposite order from which they are to appear, ie, the backmost first, and the frontmost last. Alternately, the windows can be sent to the back in the order they appear.

There are other reasons besides reordering windows that these functions might be usefull. When an error in encountered, it is common to send a window containing the error message to the front of the parent Z order, and to send the entire window to the front. Placing an active display or a background pattern in a frameless maximized window can create wallpaper. Placing the same window in the back can be a screen saver.

G.12 Class Window Handling

manlib handles windows as a set of two files, the input and output. However, these don't need to allways be a set. It is possible to reuse an already open input file as the input side of any new window. This is possible because the user id supplied when the window is open is passed with all messages to the event procedure.

Allowing a single program thread to handle multiple open windows allows the creation of multiple window programs without the need to create a multitask program. For each message, the id is examined, and the action performed on the appropriate output file.

Because the output file is unique, it is allways the handle used to refer to the window in management calls.

G.13 Parallel Windows

Parallel tasking is a natural match to a multiwindowing system. With a task created to operate each window, the program is simplified, and user response is generally greater.

To increase the responsiveness of user interfaces, the recommended ideal program design for windowing programs is to create two tasks for each window, the "foreground" and the "background" tasks. The foreground task performs the event loop and the redraw, resize, move and other user interface tasks. The background task would handle computation tasks and other tasks related to performing actions or changing the drawing surface. The foreground task determines if an action that

requires the background task is required, then sends a command to the background task via a monitor queue.

The reason for this construction is that the things the user sees, such as keeping the client area redrawn, or obeying a move, resize, minimize or similar command allways have a task waiting to perform them. This means that elementary user desktop management appears responsive to the user, while data manipulation and comutation tasks simply delay client area updates. Users will be much more willing to tolerate client area slowdowns than having a window apparently lock stubbornly in position.. Ideally, the client area should also have an indication that computation is taking place. The most modern method for this is a progress indicator that gives estimates as to when a task will complete, if the task will take longer than about 1 second.

A parallel foreground task is automatically provided for a buffered mode window. It needs to be created in the case of an unbuffered window.

If the work performed in a client area is trivial, only a foreground task need be provided. But be aware that it is very easy to slip into a mode where essential updates are held off. Calling a file procedure, waiting on a timer, or other simple task compromises the response time for window management. Better to leave the window in buffered mode, or structure with foreground/background from program creation than to have to add it later.

G.14 Menus

The menu is a series of buttons labeled with text for user action. The buttons can either have a direct effect on the program, or they can activate a series of submenus in a tree structure.

The exact location of a menu is system dependent. It may or may not affect the client area. A menu is described to **manlib** by constructing a data structure:

```
menuptr = ^menurec;
menurec = record

    next:    menuptr; { next menu item in list }
    branch:  menuptr; { menu branch }
    onoff:   boolean; { on/off highlight }
    oneof:   boolean; { "one of" highlight }
    bar:     boolean; { place bar under }
    id:      integer; { id of menu item }
    face:    pstring  { text to place in button }

end;
```

Menu buttons are highlighted when pointed to. Additionally, a button can have "on/off" highlighting. In this mode, a state is kept for the button that is flipped on or off as the button is pressed. The highlighting for on and off states is different. The data structure that defines a menu is not used or updated after it is used to create a menu.

The **onoff** field true selects on/off highlighting. After the menu is activated, the state of the button can be changed with **menusel**. The button state is not automatically changed by a user menu select. The program must specifically change the state of the button.

Another highlight mode is "one of" or "checklist" highlighting. In this case, a group of related buttons are joined by setting the **oneof** flag on each item in the list but the last. The highlighting for a button that is selected is different from one that is not, and only one item will be active in the list.

Like on/off buttons, user selection does not automatically change the state of the buttons in the list. It must be specifically changed by a **menusel** call.

Typically, neither on/off nor one/of switching is available for top level (horizontal) menus, and the program should not count on them.

Menu items can be grouped in a vertical list by setting the **bar** field active. This causes a horizontal bar to be drawn under the menu item. Vertical lists are described next in "menu sublisting".

A button can be enabled or disabled. A disabled button is highlighted specially, such as greyed out. It indicates a button that is entirely inactive, usually because that function is not available. This is done to allow the same menu to be used regardless of the availability of the functions on the menu. Also, some functions come and go. For example, a "save" button (for "save file") might only be active if the file has changed, and needs to be saved.

The enable status can be changed after the menu is activated by **menusel**.

Each button in a menu can have a numeric **id**. This is used by several functions to change the state of buttons. It is an error to have two buttons with the same id.

The **face** text string indicates what text will appear on the face of the button. The system will automatically size the buttons to fit the largest text of a button, so care should be taken not to have a single button's text so long as to cause a lot of white space in the menu.

G.15 Setting Menu Active

A menu is constructed by the program as a list using the menu data structure, then set active by calling **menu**, which can also turn the menu back off.

When a menu data structure is activated, all of the fields are copied and placed into an internal form. After the activation, the menu data has no function, and changing its fields will have no effect. The menu data structure can be recycled immediately after the **menu** call.

G.16 Setting Menu States

For an on/off button, the procedure **menuena** is used to enable or disable the button after it has been placed in a menu. For oneof highlighting, the procedure **menusel** is used. This removes any other select active, and either sets the given button active, or no button active.

G.17 Standard Menus

Besides presenting a menu in the method standard for the implementation, it is also likely that there exists a standard arrangement of commonly used buttons in the menu. manlib defines a series of such standard buttons.

```

{ standardized menu entries }

smnew      = 1; { new file }
smopen     = 2; { open file }
smclose    = 3; { close file }
smsave     = 4; { save file }
smsaveas   = 5; { save file as name }
smpageset  = 6; { page setup }
smprint    = 7; { print }
smexit     = 8; { exit program }
smundo     = 9; { undo edit }
smcut      = 10; { cut selection }
smpaste    = 11; { paste selection }
smdelete   = 12; { delete selection }
smfind     = 13; { find text }
smfindnext = 14; { find next }
smreplace  = 15; { replace text }
smgoto     = 16; { goto line }
smselectall = 17; { select all text }
smnewwindow = 18; { new window }
smtilehoriz = 19; { tile child windows horizontally }
smtilevert  = 20; { tile child windows vertically }
smcascade  = 21; { cascade windows }
smcloseall  = 21; { close all windows }
smhelptopic = 22; { help topics }
smabout    = 23; { about this program }
smmax      = 23; { maximum defined standard menu entries }

{ standard menu selector }

stdmenusel = set of smnew..smmax;

```

Standard menu button definitions should be used whenever possible. To create a menu using standard menu buttons, the procedure **stdmenu** is used. The arrangement of the menu is chosen to match the implementation, and the non-standard buttons provided are integrated into the menu in the normal place for the implementation. When standard buttons are used, the button ids are set to the values shown above, so these values should not be used by the program.

Note that **stdmenu** simply constructs a menu, it does not set it active.

G.18 Menu Sublisting

When there is not enough room to represent the entire menu on a window, or anytime the size of a menu must be compressed, the tree structure of menus can be used with "pulldown" menus. A pulldown menu is a vertical menu list that appears under the selected button. This is done by placing a submenu in the **branch** pointer of the menu structure. The branch defines the pulldown menu items. The branch menu item is specially indicated to show that it is a branch, and not an immediate button, usually with an arrow pointing towards where the branch pulldown will appear. Any number of levels of pulldown menus can be created. The levels under the topmost branch are generally placed to the right of the branch button.

If a branch pulldown cannot be placed where it should, due to the proximity of the button to the edge of the display, it instead goes back in the other direction, up or down, or both, until space is found for it. If the entire pulldown cannot be displayed, it is truncated. This would only happen if the pulldown was too large for the display.

When a branch button is pressed, no event is generated for it. The button is strictly used to activate the pulldown.

G.19 Advanced Windowing

The features of a window besides the client area are designed to be the minimum features implemented across platforms. To implement more advanced windows appearances with custom menus, controls and status lines, the standard method is to turn off frames and menus for child windows, and use them as building blocks for more advanced client areas with multiple subwindows and features.

It is recommended that at least one menu appear for a program, even if the functions duplicate some of the advanced controls you provide. The reason for this is that the menu has different presentation methods in different systems, so providing the standard "top" menu will help your programs' portability.

G.20 Events

New event types are added for the management mode. As usual, events beyond the definition here should be ignored.

type

```
{ events }
```

```
evtcod = (etchar,      { ANSI character returned }
          etup,        { cursor up one line }
          etdown,      { down one line }
          etleft,      { left one character }
          etright,     { right one character }
          etleftw,     { left one word }
          etrightw,    { right one word }
          ethome,      { home of document }
          ethomes,     { home of screen }
          ethomel,     { home of line }
          etend,       { end of document }
          etends,      { end of screen }
          etendl,      { end of line }
          etscrl,      { scroll left one character }
          etscrr,      { scroll right one character }
          etscru,      { scroll up one line }
          etscrd,      { scroll down one line }
          etpagd,      { page down }
          etpagu,      { page up }
          ettab,       { tab }
          etenter,     { enter line }
          etinsert,    { insert block }
          etinsertl,   { insert line }
          etinsertt,   { insert toggle }
          etdel,       { delete block }
          etdell,      { delete line }
          etdelcf,     { delete character forward }
          etdelcb,     { delete character backward }
          etcopy,      { copy block }
          etcopyl,     { copy line }
          etcan,       { cancel current operation }
          etstop,      { stop current operation }
          etcont,      { continue current operation }
          etprint,     { print document }
          etprintb,    { print block }
          etprints,    { print screen }
          etfun,       { function key }
          etmenu,      { display menu }
          etmouba,     { mouse button assertion }
          etmoubd,     { mouse button deassertion }
          etmoumov,    { mouse move }
          ettim,       { timer matures }
          etjoyba,     { joystick button assertion }
          etjoybd,     { joystick button deassertion }
          etjoymov,    { joystick move }
          etterm,      { terminate program }
          etmoumovg,   { mouse move graphical }
          etframe,     { frame sync }
          etresize,    { window was resized }
          etredraw,    { window redraw }
          etmin,       { window was minimized }
          etmax,       { window was maximized }
          etnorm,      { window set normal }
```

```

        etmenus);    { menu item selected }

{ event record }

evtrec = record

    winid: ss_filhdl; { identifier of window for event }
    case etype: evtcod of { event type }

        { ANSI character returned }
        etchar:    (char:          char);
        { timer handle that matured }
        ettim:     (timnum:        timhan);
        etmoumov:  (mmoun:         mouhan;    { mouse number }
                    moupdx, moupdy: integer); { mouse movement }
        etmouba:   (amoun:         mouhan;    { mouse handle }
                    amoubn:        moubut);   { button number }
        etmoubd:   (dmoun:         mouhan;    { mouse handle }
                    dmoubn:        moubut);   { button number }
        etjoyba:   (ajoynd:        joyhan;    { joystick number }
                    ajoybn:        joybut);   { button number }
        etjoybd:   (djoynd:        joyhan;    { joystick number }
                    djoybn:        joybut);   { button number }
        etjoymov:  (mjoynd:        joyhan;    { joystick number }
                    joypdx, joypdy, joypdz: integer); { joystick coords }
        etfun:     (fkey:          funky);    { function key }
        etmoumovg: (mmoung:        mouhan;    { mouse number }
                    moupdxg, moupdyg: integer); { mouse movement }
        etredraw:  (rsx, rsy, rex, rey: integer); { redraw screen }
        etmenus:   (menuid:        integer); { menu item
                                                selected }

    etup, etdown, etleft, etright, etleftw, etrightw, ethome,
    ethomes, ethomel, etend, etends, etendl, etscrl, etscrr,
    etscru, etscrd, etpagd, etpagu, ettab, etenter, etinsert,
    etinsertl, etinsertt, etdel, etdell, etdelcf, etdelcb, etcopy,
    etcopyl, etcan, etstop, etcont, etprint, etprintb, etprints,
    etmenu, etterm, etframe, etresize, etmin, etmax,
    etnorm: (); { normal events }

{ end }

end;

```

G.21 Window Objects

All of the procedures, functions and other declarations in **manlib** are also available in a graphical terminal object. of the form:

```

class window;

var input, output: text; { input and output files for this screen }
    er: evtrec;           { event record }

{ procedures and functions from manlib }

```

The complete catalog of procedures and functions from `gralib` are available as methods in the `graphicalterminal` class. There are minor differences, which are detailed with the procedure and function descriptions in E.16.

When a **graphicalterminal** object is instantiated, it's input and output files are automatically linked to the standard **input** and **output** files of the program. Thus, where specifying the input and output files in **manlib** procedures and functions is optional, in methods of the **window** object these files *must not* be specified.

The input and output files used by the **window** object are made available as class members `input` and `output`. This allows them to be used in non-method procedures (such as `read` and `write`).

A **window** object also contains an event record, so it is not necessary to specify an external event record in the **event** procedure.

A **window** object can be created as follows:

```

program p;

joins manlib;

var ti: instance of gralib.graphicalterminal;

begin

    ti.home; { send cursor to home position }
    writeln(ti.output, 'hello, terminal world');
    repeat { event loop }

        ti.event; { get next event }
        { process events }

    until ti.event = etterm { loop until program cancelled }

end.

```

Where **terminal** is the terminal object. Terminal objects can be instantiated statically or dynamically.

Terminal objects contain their own state for the following:

1. Location of cursor.
2. Current attributes and colors.
3. Current font and size.
4. Buffer working/display status.
5. Tab stops.
6. Timer numbers.
7. Cached pictures.
8. Buffer/non-buffer mode.
9. Window attributes, size and location in parent.
10. Z-order.
11. Menus.

Window objects are inherently separate drawing surfaces (unlike **terminal** and **GraphicalTerminal** objects). The **OpenWin** procedure must be invoked to activate a window object. Unlike the **manlib** module procedure, **OpenWin** does not specify an output file, this is inherent to the object. The **input** file is optional, and is used to bond multiple windows to a single input handler.

When multiple terminal objects exist, and all have the cursor visibility on, the cursor will remain at the last place it was specifically directed to. This can mean that the cursor will rapidly appear to swap back and forth between the active drawing terminal objects. This can be improved by only selecting one of the terminal objects as having a visible cursor. Typically, a the cursor should be placed to attract user attention to where text is being entered, or if none is being entered, it should be turned off.

Note that **winlib** also exposes a terminal object that is identical to the **trmlib terminal** object and the **gralib GraphicalTerminal** object. It features the same calls as **window**, but does not allow access to the advanced features in **window**.

G.22 Procedure and Functions in manlib

For all of the following calls, If the screen file **f** is not present, the default is the standard output file. If the procedure or function is a method, neither the input nor output screen file should be specified, since it is inherent in the object.

The **OpenWin** procedure is an exception, and both the module procedure and the **window** method have special formats.


```

procedure openwin(infile, outfile [, parent]: text; id: integer); { module procedure }
procedure openwin(infile [,parent]: text; id: integer); { window object method }

```

Opens a new window. The input file **infile** will get messages pertaining to the window, and the output file **outfile** will be used to draw to it. The input file may already be open elsewhere, in which case it will get all messages for all open windows opened with it. The window will be placed at 1,1 within the parent and held out of display. The output file is always used as the window handle, since it unique to the window.

The window identifier **id** is a number from 1 to n that is returned in messages concerning the window.

Only the output side of a window pair is used in procedures or functions that operate on the window once opened with openwin.

A window is closed with the standard Pascaline close procedure, used on the output file for the window. The input side of the window is automatically closed when there are no longer any windows that reference it.

If the optional parent window is specified, the new window is created as a child of the given parent. This means that it will always be displayed within the parent, and be clipped to it.

When used as a method in the **window** object, the output file is not specified.

```

procedure buffer([var f: text;] b: boolean);

```

Engages or removes window **f** from buffered mode, according to the boolean **b**. In buffered mode, all of the drawing for a window is performed on a memory buffer, then copied to the screen. The screen view of the buffer can be all or part of the buffer, and multiple buffers can be managed.

```

procedure sizbuf[g]([var f: text;] x, y: integer);

```

Sets the size of the buffer used to draw into. **x** and **y** indicate the width and height, respectively, of the buffer surface in window **f**. It is an error if buffering is not enabled. Sizbuf sets the buffer size in characters. Sizbufg sets the buffer size in pixels.

```

procedure title([var f: text;] view s: string): integer;

```

Sets the title of the window **f** to the string **s**. If the title is too long for the current window size, an implementation defined method will be used to make it fit, for example, it is clipped.

```
procedure frame([var f: text;] e: boolean);
```

Enables or disables the appearance of the frame in window **f**, according to boolean **e**, which includes the minimize, maximize, title, size, move and close controls. If the frame is removed, the user will be unable to operate the frame controls.

It is possible that the implementation may not allow removal of the frame. In this case, all of the frame controls will still be active, and the program must accept these.

```
procedure sysbar([var f: text;] e: boolean);
```

Enables or disables the system control bar for a window **f**. If **e** is true, the system bar is enabled, otherwise disabled. The system bar normally includes the title, minimum, maximum, and other control buttons. If the frame is not enabled, then the system bar will not appear regardless of the sysbar status, but it will be recorded.

It is possible that the implementation may not allow removal of the system bar. In this case, all of the system controls will still be active, and the program must accept these.

```
procedure sizable([var f: text;] e: boolean);
```

Enables or disables the sizing bars for a window **f**. If **e** is true, the sizing bar is enabled, otherwise disabled. If the frame is not enabled, then the size bars will not appear regardless of the sizable status, but it will be recorded. If the size bars are removed, the user will be unable to resize the window.

It is possible that the implementation may not allow removal of the sizing bars. In this case, all of the size controls will still be active, and the program must accept these.

```
procedure setpos[g]([var f: text;] x, y: integer);
```

Sets the position, within the parent, of a child window **f**, using position **x** and **y**. If the window is on the desktop, then the window position is relative to the desktop. The **setpos** procedure sets the position in terms of characters, and the **setposg** procedure set the position in terms of pixels.

The position is set in terms of the parent's coordinates. The mode of the parent's coordinates may not be known. For example, the desktop may not even have a character mode, so the idea of a character size may be arbitrary. What matters in this case is the relative position and size in the desktop as determined by **scnsiz**.

```
procedure scnsiz[g]([var f: text;] var x, y: integer);
```

Finds the size of the user screen or desktop for window **f** to the size **x** and **y**. **scnsiz** returns the size in character terms, and **scnsizg** returns the size in pixel terms. If the desktop does not have a character mode, an arbitrary scale is created. What is important is the relative location within the desktop.

```
procedure winclient[g]([var f: text;] cx, cy: integer; var wx, wy: integer; ms: modset);
```

Determines the window size needed for a given client size within window **f**. Given a desired client size of **cx** and **cy**, in width and height, the necessary window size to achieve that will be returned in **wx** and **xy**. **winclient** determines these measurements in character demensions, and **winclientg** determines them in pixel terms.

The set of modes **ms** is used to determine the needed window size.

If the parent of the window has no character mode, then one is created that will be acceptable to **setpos**.

```
procedure getsiz[g](var f: text; var x, y: integer);
```

Finds the size of a window in parent coordinate terms for window **f**, to size **x** and **y**. **getsiz** returns the character size, and **getsizg** returns the pixel size.

If the parent has no character mode, then one is created that is compatible with other **manlib** functions and procedures.

```
procedure setsiz[g]([var f: text;] var x, y: integer);
```

Sets the size of window **f** in parent coordinate terms, to size **x** and **y**. **setsiz** sets the character size, and **setsizg** sets the pixel size.

If the parent has no character mode, then one is created that is compatible with other **manlib** functions and procedures.

```
procedure back[(var f: text)];
```

Sends the window **f** to the back of the parent Z order.

```
procedure front[(var f: text)];
```

Sends the window **f** to the front of the parent Z order.

```
procedure menu([var f: text;] m: menptr);
```

Sets up the menu bar for window **f** from the given list, which contains a menu bar definition structure.

If the menu pointer is nil, then the menu is removed.

The menu data structure is copied during the call, so the menu structure can be reused or freed.

```
procedure stdmenu(sms: stdmenusel; var sm: menuptr; pm: menuptr);
```

Constructs a standard menu and returns that in **sm**. **sms** contains the set of desired standard buttons. **pm** contains a menu containing non-standard buttons to be added to the menu. The menu is constructed using the desired standard buttons, and the non-standard buttons placed into the menu at a standard location.

```
procedure menuena([var f: text;] id: integer; e: boolean);
```

Enables or disables a on/off menu button for window **f**. **id** refers to a button **id** that was specified in the menu data structure. If **e** is true, the button is enabled, otherwise disabled. The highlighting of the button will change to match.

```
procedure menusel([var f: text;] id: integer; e: boolean);
```

Selects a button from a oneof list to be active in window **f**. **id** refers to a button id that was specified in the menu data structure. If **e** is true, the button is selected, otherwise deselected. All other buttons in the oneof list are deactivated. The highlighting of the buttons will change to match.

H [Annex H: Widget Library](#)

trmlib and **gralib** define terminal and graphical operations on a fixed screen. **manlib** defines its division into "virtual windows". The last part of the puzzle are the elements placed within those windows to allow user control. These include buttons, sliders, scroll bars, checkboxes, and similar user interface elements.

These are sometimes referred to as controls or widgets. Dialogs are predefined windows with widgets in them. What these user elements have in common is they all use **manlib** elements to define the window they appear in, and **gralib** or **trmlib** routines to draw their appearance.

wiglib can be performed entirely in terms of **manlib** with **gralib** or **trmlib** calls. However, it is still dependent on a particular operating system because of its appearance. **wiglib** is your key to attaining the "look and feel" of a particular operating system. But it is still possible to write your own widgets.

H.1 [Tiles, Layers and Looks](#)

Each widget is implemented in its own window. A layout of widgets occurs when several widgets are tiled side by side, or place on top of each other (layered). For example, a window surface with text and scrollbars to control the positioning of that text is done by constructing a series of windows.

The first window is the window that holds the presented text. The second and third are the windows that hold the horizontal and vertical scrollbars.

Layering is done by defacto transparency. A series of widgets are placed one atop the other. For example, a text window can be laid ontop of a background window.

The key to interface design is to think of a window as simply a building block for construction of the user interface.

Widgets are fundamental to the "look" of an interface. Because **wiglib** uses the native widgets on the operating system it serves, you will pick up quite a bit of that look from just the use of the widgets. Certainly its true that there is more to an application than just the look of the widgets. There are layout conventions, actions, and other intangibles. The rule that applies to Pascaline portability is:

- Pascaline should be able to target a high percentage of simple applications just by use of its normal **wiglib** components.
- Pascaline should be able to finish a high percentage of the work to create complex applications.

Think of interface design as performed by a collection of wooden blocks. Certainly any shape of building design can be performed, but the result won't be exactly the same as the equivalent building. What makes a building look "custom" are the final touches of plaster and paint that give it a finished look.

The typical cycle in designing an Pascaline application is to design an inital version that uses just wiglib components, then finish the design with special layouts, actions and colors for a particular operating system that give it the "look and feel" of a native application.

H.2 [Background colors and placement](#)

Widgets are placed by specifying their "bounding box" or rectangle. This is the rectangle that contains the widget. A widget can occupy all of the specified bounding box, or just a part of it. Typically, the operating system will try its best to format the widget to fit within the space provided.

The color scheme for widgets can vary. However, widgets are designed to be placed against a background color that is system dependent. This is available as a new system defined color, `backcolor`. It can be selected by the standard color selection routines.

```
type color = (black, white, red, green, blue, cyan,  
              yellow, magenta, backcolor);
```

```
fcolor(backcolor);  
bcolor(backcolor);
```

There are many ways to group widgets so that they blend into your background. The surface they appear on can be specifically colored using the background color, or you can create a child window with that color. Also, there is a background widget that can color the background for widgets automatically.

H.3 Sizes

Using widgets can go a long way towards getting the look and feel of a system in an independent way. The other important factor for achieving system independence is to control sizing of widgets. For each widget, with the exception of dialogs, there is a size routine. The size routine takes the particular features of that widget, such as face text or borders, and gives the best size for the widget, given the desired client area, contents, etc..

The size information must be considered against the particular widget to be created. Sizing is key to establishing the layout of widgets in a window, and key to producing a truly portable application.

H.4 Logical Widget Identifiers

A widget is created with a logical number, from 1 to *n*, where *n* is a positive integer. You specify the number you wish to create the widget under. The id of a widget cannot be the same as any other active widget, but widget logical identifiers can be reused by killing the widget first.

H.5 Killing, Selecting, Enabling and Getting Text to and from Widgets

A widget is killed by **killwidget**.

Some widgets can be selected, which changes their appearance to the select state. A widget is selected by **selectwidget**.

A selected widget can be a checkbox that is checked, a radio button that is pushed, or similar effect. The processing of selects, and keeping track of the state of widgets is up to you. However, this is a very flexible system. You can implement widgets that flip their state when clicked on, a series of widgets that are mutually exclusive, and many other combinations.

Similar to selection, widgets can be enabled or disabled by **enablewidget**. Widgets are disabled by default. When a widget is disabled, it has the disabled appearance, such as greyed out. The widget will not give click events when it is disabled. Disabling a widget is typically used when it is not needed or

not available in the current context. For example, a "next" button could be disabled when there is no next item to process.

Some widgets have text that can be set, read, or in some cases, both. The text in a widget is set by **putwidgettext**. The text in a widget can be read by **getwidgettext**.

Setting and getting the widget text is used with widgets that allow the user to modify the text, such as an edit box.

H.6 Resizing and repositioning a widget

To prevent the need to remove and replace widgets each time a window is resized, **sizwidget[g]**. To reposition the widget in the parent window, **poswidget[g]** exists.

H.7 Types of widgets

Widgets come in three different types, controls, components and dialogs. Controls are widgets that the user can manipulate, and these widgets issue events to the program that owns them.

Components are display widgets whose only job is to form part of a display to the user. A group box, and a background are examples of components. Some components have active displays, such as the progress bar. However, components never issue events.

Dialogs are fully autonomous windows that exist apart from the applications windows. They can be very complex inside, having a whole system of layout, widgets and other features. They resemble entirely separate programs.

Dialogs take a series of parameters when they are called, and deliver those same parameters back to the caller, with any modifications the user performs on the data.

H.8 Z ordering

widlib uses "implicit Z ordering" for layered widgets. In order for widgets to properly layer, the drawing Z order must be controlled. For example, a background must be drawn first, followed by whatever is on top of it, or the drawing of the background will wipe out what is on the surface of the background.

To enable this, **widlib** makes sure that widgets which are designed to be layered appear first in the drawing order.

H.9 Controls

A button can be created with **button[g]**. The button is drawn in the specified rectangle with a label text string, and logical widget number **id**. The text string will be a single line of text with no control characters, and will be presented on the face of the button. The font style and size will be the same as other buttons in the operating systems user interface.

When a button is pressed, it will typically change its appearance to indicate that. The button will send an event **etbutton**. This event carries the id of the button that was asserted. Similarly, when a button is released, it changes appearance back from the pressed state.

Buttons can be any size, any height and width. The button face text does not get larger with the button, but remains centered. However, if the button too small, some of the face text will be clipped off. The size of the button can be determined before creating it by **buttonsize[g]** and height, respectively, of the button.

Buttons cannot be selected, but they can be disabled. The text in a button can be neither read nor written.

A checkbox is created with **checkbox[g]**. when hit, give a single event that indicates activation, **etchkbox**. This event contains the id of the widget.

Checkbox sizing is found with **checkboxsize[g]**. Checkboxes are sized to minimum, but since they have no edges (like a button), there is typically no need to add space to them.

Checkboxes can be selected (checked). They can be enabled or disabled, and default to enabled. They cannot have their face text changed or read.

Radio buttons work identically to checkboxes, but have a different appearance. A radio button is created by **radiobutton[g]**. Radio buttons, when hit, give a single event that indicates activation, **etradbut**. This event contains the id of the widget.

Radio button sizing is found with **radiobuttonsize[g]**. Radio buttons are sized to minimum, but since they have no edges (like a button), there is typically no need to add space to them.

Checkboxes can be selected (checked). They can be enabled or disabled, and default to enabled. They cannot have their face text changed or read.

Scrollbars are placed vertically by **scrollvert[g]**. Scrollbars are placed horizontally by **scrollhoriz[g]**.

Scroll bars can be placed using any dimensions, but the width of a vertical scroll bar, and the height of a horizontal scroll bar usually has a standard size. This can be determined by **scrollvertsiz[g]**.

A user movement of a scrollbar is given by the event **etsclpos**. This event does not move the scrollbar slider. This must be done by the program via **scrollpos**.

When a user positions the scroll bar directly, it will follow the users mouse movements. However, it will return to the original position unless the **etsclpos** event is responded to and a **scrollpos** call is made.

Besides position events, scrollbars issue two types of button pushes, referred to as line and page button events. The line buttons are the arrow buttons at either side of the scroll bar. The page buttons are the space between the scrollbar slider and the line arrows. A press anywhere in this area generates a page button event.

The page button area may not appear at all if the slider is fully to one side of the scrollbar.

The page and line terminology for these buttons occurs because their most common use is to scroll documents. In this case, the line buttons would be used to move the document one line up or down. In the case of horizontal scrollbars, this would be one character left or right (despite the term "line" in the

buttons name). The page refers to one screenful of text, in any direction. If the page up button is hit, for example, the document would move one screenful up.

It's up to the program to implement the actions for line up/down and page up/down. In fact, these events can be used for any purpose you want in your programs.

Besides the position of the slider, its size can also be controlled by **scrollsiz**.

The standard use of the scrollbar size is to indicate what proportion of the document or display is contained within the onscreen display, vs. the total size of the document. For example, if the document has 50% of its total displayed, then the size of the scrollbar is set to %50, which is done by:

```
scrollsiz(f, n, maxint div 2);
```

Scroll bars cannot be selected, enabled or disabled, or have face text read or written.

A number can be selected in an edit box by **numselbox[g]**.

The first number that appears in the number select box is by default the lower bound.

Number select boxes are an easy way to enter numbers from the user, and automatically restrict the input to numbers only. The numbers are entered in decimal, and cannot be negative. The edit box allows the number to be directly edited. Also, there are usually up and down buttons that allow the user to count the number up or down by one.

The size of a number select box is found by **numselboxsiz[g]**.

Number select boxes cannot be selected, enabled or disabled, or have face text read or written.

A general string can be edited with **editbox[g]**.

And empty edit box is placed, and the user has the ability to edit text into the box, with cursor movements, character delete, etc.

An edit box can be presented blank, or default text can be placed into the edit box. If the user presses enter to the box, it sends a **eteditbox** event. However, the program can use any method to signal done, such as a button next to the edit control. The resulting text can then be retrieved from the edit box.

The size of an edit box is found by **editboxsiz[g]**.

Edit boxes cannot be selected, enabled or disabled, or have face text read or written.

A list box is a series of items that can be selected. It is placed with **listbox[g]**.

The string list definition appears as:

```
{ string set for list box }

strptr = ^strrec;
strrec = record

    next: strptr; { next entry in list }
    str:  pstring { string }

end;
```

The string pointer is a list of strings, each string of which describes an entry in the list box.

When the user selects an item from the list box, the **etlstbox** event is returned. This event gives the id of the widget, and the number of the select, from the top. The first item in the list will be 1, the second 2, etc.

The size of a list box is found by **listboxsiz[g]**.

List boxes cannot be selected, enabled or disabled, or have face text read or written.

The same multiple string selection can be done in a different way by **dropbox[g]**.

A drop box only shows the whole list if the user selects it, otherwise only the currently selected entry is shown. The full list "drops down" from the selection box when the user selects it. A drop box takes less space than a list box when it is not selected. A drop box selection is signaled by the **etdrpbox** event, which gives the widget id, and the number of the selection, from 1 to n.

The size of a drop box is found by **dropboxsiz[g]**. Because drop boxes have two appearances, one when open, and one when closed, both sizes are returned. The closed appearance gives the basic size of the widget, but the open size makes is possible to determine if the list will go beyond the edge of the window when open.

Drop boxes cannot be selected, enabled or disabled, or have face text read or written.

Very similar to a drop box, a drop/edit box allows selection from a list, but also allows the current selection string to be edited.

A drop/edit box is placed with **dropeditbox[g]**. When a selection is made from the drop/edit box, the **etdrebox** event is sent, which includes the widget identifier. The selection data itself is a string, and must be retrived with **getwidgettext**.

The size of a drop/edit box is found by **dropeditboxsiz[g]**. Because drop/edit boxes have two appearances, one when open, and one when closed, both sizes are returned. The closed appearance gives the basic size of the widget, but the open size makes is possible to determine if the list will go beyond the edge of the window when open.

Drop/edit boxes cannot be selected, enabled or disabled, or have face text read or written.

Sliders are linear controls that can be placed either horizontally or vertically.

A vertical slider is placed with **slidevert[g]**. A horizontal slider can be placed by **slidehoriz[g]**.

Sliders indicate changes in their position with the event **etsldpos**. This gives the widget id, and a **maxint** ratioed position of the slider, from 0 to **maxint**. 0 is the top or leftmost position of the slider, and **maxint** is the bottom or rightmost position of the slider.

The size of a slider can be determined by **scrollvertsiz[g]**.

Sliders cannot be selected, enabled or disabled, or have face text read or written.

Tab bars allow the user to select from a series of labeled tabs, usually to specify locations in document. A tabbar is a group box with tabs on one, two, three or four edges. The tabs can be placed on the top, bottom, left or right side of an included client area.

A tabbar is placed by **tabbar[g]**.

Tabbar selections are indicated by the event **ettabbar**, which gives the widget id, the side which generated the event and the tab number selection, from 1 to n, counting from the first string entry in the list.

The orientation of a tabbar is given by:

```
{ orientation for tab bars }  
type tabori = (totop, toright, tobottom, toleft);
```

The size of a tabbar is found by **tabbarsiz[g]**. A tabbar acts like a group box, and has a client area to place child windows or widgets. The required client size can be specified, and the sizing call returns the offset required to find the client location within the tabbar.

If the tabbar must fit into a fixed window size, the size of the resulting client for a tabbar can be found with **tabbarclient[g]**.

Tab bars cannot be selected, enabled or disabled, or have face text read or written.

H.10 Components

A background box is placed by **background[g]**. A background box is designed to serve as the background to a series of controls, and it has the standard color for such backgrounds.

Group boxes have no sizing, because there are no borders or other content. They are just a colored rectangle. Tab bars cannot be selected, enabled or disabled, or have face text read or written.

A group box is similar to a background box, but it has a label for the "group" of controls contained within it. It is placed by **group[g]**.

The size of a group box found by **groupsiz[g]**. A group box has a client area to place child windows or widgets. The required client size can be specified, and the sizing call returns the offset required to find the client location within the group box.

Group boxes cannot be selected, enabled or disabled, or have face text read or written.

A progress bar is used to indicate the progress of a job completion, like installing software, saving a file, etc.

It is placed by **progbar[g]**. The initial progress indication is zero when placed. The size of the progress bar is set by **progbarpos**.

The size of a progress bars can be determined by **progbarsiz[g]**.

Progress bars cannot be selected, enabled or disabled, or have face text read or written.

H.11 Dialogs

A dialog is a completely separate window which is preformatted with widgets. Dialogs introduce complex queries into a program, using the look of the native operating system,.

Dialogs display a property known as modality. Since the dialog is a separate window, it can be independent of the other windows created by the calling task, or the dialog can be forced to appear at the top of the applications stacking order.

widlib uses two modes of modality. If the task that created the dialog also created other windows, the dialog is forced to the front of the other windows, and selection of the other task windows is disabled. This indicates to the user that only the dialog is currently being managed by the task.

If windows are created by different threads, then the dialog will not be modal vs. the other thread's windows. This reflects the fact that the windows outside the dialog can run while the dialog does.

An alert dialog is used to send errors or other important messages to the user. It has a window title, a message that constitutes the alert, and typically has an "ok" or "close" button for the user to indicate they have seen it.

An alert is created by **alert**. The alert call will not return until the user has clicked the ok button for the alert.

The query dialogs allow the user to select important information such as a file, a search string, or color or a font. They use a model called "flow through". The query may select several types of information, and will accept a default setting, allow the user to select a new setting, and return that. The flow through model means that several parameters are set up before the call, may or may not be modified by the query, then are returned to the caller. The program stops until the dialog is completed by the user.

All of the parameters of a dialog may or may not be implemented in the actual system dialog. The flowthrough system allows for differences in systems. Since the variables are preinitialized with the defaults, if the dialog does not implement a particular parameter, the value will be left unchanged.

A color can be choosen by **querycolor**. The default color is set before the call, and the possibly changed color is returned by the call.

A file to open name is selected by **queryopen**.

The default filename is passed in as a dynamic string, and a different string is returned as the result of the dialog. The string returned is different than the input string, and the program is responsible for

disposing of both strings. If the dialog is canceled instead of completed by the user, the string returned is zero length. This means to not proceed with the open operation.

A file to save name is selected by **querysave**.

The default filename is passed in as a dynamic string, and a different string is returned as the result of the dialog. The string returned is different than the input string, and the program is responsible for disposing of both strings. If the dialog is canceled instead of completed by the user, the string returned is zero length. This means to not proceed with the save operation.

A string to search for is selected by **queryfind**.

The option flags are given by a set of flags:

```
{ settable items in find query }
qfnopt = (qfncase, { Case sensitive }
          qfnup}, { Search up/Search down }
          qfbre);
qfnopts = set of qfnopt;
```

The default search string is passed in as a dynamic string. The string returned is different than the input string, and the program is responsible for disposing of both strings. If the dialog is canceled instead of completed by the user, the string returned is zero length. This means to not proceed with the search operation.

A string to search for and replace is selected by **queryfindrep**.

The option flags are given by a set of flags:

```
{ settable items in replace query }
qfropt = (qfrcase, { case sensitive }
          qfrup,   { search up/search down }
          qfrfind,
          qfrallfil,
          qfralllin);
qfropts = set of qfropt;
```

The default search string and replacement strings are passed in as a dynamic strings. The strings returned are different than the input string, and the program is responsible for disposing of all strings. If the dialog is canceled instead of completed by the user, the strings returned are zero length. This means to not proceed with the search/replace operation.

Fonts are selected by **queryfont**.

The font effects are declared as:

```
{ effects in font query }
qfteffect = (qfteblink, qftereverse, qfteunderline,
             qftesuperscript, qftesubscript, qfteitalic,
             qftebold, qftestrikeout, qftestandout,
             qftecondensed, qfteextended, qftexlight,
```

```
          qftelight, qftexbold, qftehollow, qfteraised);  
qfteffects = set of qfteffect;
```

H.12 Events

The definition of an event record is upward compatible with previous event record declarations from **trmlib** and **gralib**.

```

{ events }
evtcod = (etchar,      { ANSI character returned }
          etup,        { cursor up one line }
          etdown,      { down one line }
          etleft,      { left one character }
          etright,     { right one character }
          etleftw,     { left one word }
          etrightw,    { right one word }
          ethome,      { home of document }
          ethomes,     { home of screen }
          ethomel,     { home of line }
          etend,       { end of document }
          etends,      { end of screen }
          etendl,      { end of line }
          etscr1,      { scroll left one character }
          etscrr,      { scroll right one character }
          etscru,      { scroll up one line }
          etscrd,      { scroll down one line }
          etpagd,      { page down }
          etpagu,      { page up }
          ettab,       { tab }
          etenter,     { enter line }
          etinsert,    { insert block }
          etinsertl,   { insert line }
          etinsertt,   { insert toggle }
          etdel,       { delete block }
          etdell,      { delete line }
          etdelcf,     { delete character forward }
          etdelcb,     { delete character backward }
          etcopy,      { copy block }
          etcopyl,     { copy line }
          etcan,       { cancel current operation }
          etstop,      { stop current operation }
          etcont,      { continue current operation }
          etprint,     { print document }
          etprintb,    { print block }
          etprints,    { print screen }
          etfun,       { function key }
          etmenu,      { display menu }
          etmouba,     { mouse button assertion }
          etmoubd,     { mouse button deassertion }
          etmoumov,    { mouse move }
          ettim,       { timer matures }
          etjoyba,     { joystick button assertion }
          etjoybd,     { joystick button deassertion }
          etjoymov,    { joystick move }
          etterm,      { terminate program }
          etmoumovg,   { mouse move graphical }
          etframe,     { frame sync }
          etresize,    { window was resized }
          etredraw,    { window redraw }

```

```

    etmin,      { window minimized }
    etmax,      { window maximized }
    etnorm,     { window normalized }
    etmenus,    { menu item selected }
    etbutton,   { button assert }
    etcheckbox,   { checkbox click }
    etradbut,   { radio button click }
    etsclull,   { scroll up/left line }
    etsclldr,   { scroll down/right line }
    etsclulp,   { scroll up/left page }
    etscldrp,   { scroll down/right page }
    etsclpos,   { scroll bar position }
    etedtbox,   { edit box signals done }
    etnumbox,   { number select box signals done }
    etlstbox,   { list box selection }
    etdrpbox,   { drop box selection }
    etdrebox,   { drop edit box selection }
    etsldpos,   { slider position }
    ettabbar,   { tab bar select }
    { the following is for internal use only }
    et_fndtrm,  { find dialog has messaged }
    et_wigstr,  { widget started }
    et_winstr,  { window started }
    et_wincls,  { window closed }
    et_im);     { intratask message }
{ event record }
evtrec = record

    winid: ss_filhdl; { identifier of window for event }
    case etype: evtcod of { event type }

        { ANSI character returned }
        etchar: (char: char);
        { timer handle that matured }
        ettim: (timnum: timhan);
        etmoumov: (mmoun: mouhan; { mouse number }
                  moupx,
                  moupy: integer); { mouse movement }
        etmouba: (amoun: mouhan; { mouse handle }
                  amoubn: moubut); { button number }
        etmoubd: (dmoun: mouhan; { mouse handle }
                  dmoubn: moubut); { button number }
        etjoyba: (ajoyn: joyhan; { joystick number }
                  ajoybn: joybut); { button number }
        etjoybd: (djoyn: joyhan; { joystick number }
                  djoybn: joybut); { button number }
        etjoymov: (mjoyn: joyhan; { joystick number }
                  joypx,
                  joypy,
                  joypz: integer); { joystick coordinates }
        etfun: (fkey: funky); { function key }

```



```

etmoumovg: (mmoung: mouhan; { mouse number }
            moupzg,
            moupzg: integer); { mouse movement }
etredraw: (rsx,
           rsy,
           rex,
           rey: integer); { redraw screen }
etmenus: (menuid: integer); { menu item selected }
etbutton: (butid: integer); { button id }
etchkbox: (ckbxid: integer); { checkbox }
etradbut: (radbid: integer); { radio button }
etsclull: (sclulid: integer); { scroll up/left line }
etsclldr: (sclldid: integer); { scroll down/right line }
etsclulp: (sclupid: integer); { scroll up/left page }
etscldrp: (scldpid: integer); { scroll dwn/rgt page }
etsclpos: (sclpid: integer; { scroll bar }
           sclpos: integer); { scroll bar position }
etedtbox: (edtbid: integer); { edit box complete }
etnumbox: (numbid: integer; { num sel box select }
           numbsl: integer); { num select value }
etlstbox: (lstbid: integer; { list box select }
           lstbsl: integer); { list box select num }
etdrpbox: (drpbid: integer; { drop box select }
           drpbsl: integer); { drop box select }
etdrebox: (drebid: integer); { drop edit box select }
etsldpos: (sldpid: integer; { slider position }
           sldpos: integer); { slider position }
ettabbar: (tabid: integer; { tab bar }
           tabor: tabori; { tab side }
           tabsel: integer); { tab select }
etup, etdown, etleft, etright, etleftw, etrightw,
ethome, ethomes, ethomel, etend, etends, etendl, etscrl,
etscrr, etscru, etscrd, etpagd, etpagu, ettab, etenter,
etinsert, etinsertl, etinsertt, etdel, etdell, etdelcf,
etdelcb, etcopy, etcopyl, etcan, etstop, etcont,
etprint, etprintb, etprints, etmenu, etterm, etframe,
etresize, etmin, etmax, etnorm, et_fndtrm, et_wigstr,
et_winstr, et_wincls: (); { normal events }

```

```
{ end }
```

```
end;
```

H.13 Widget enabled Objects

All of the procedures, functions and other declarations in **widlib** are also available in a object. of the form:

```
class widget;  
  
extends manlib.window;  
  
{ procedures and functions from widlib }  
  
.
```

A **widget** class can be based on any of **terminal**, **graphicalterminal**, or **window** classes, since widgets are not specific to any one display type.

The complete catalog of procedures and functions from **widlib** are available as methods in the widget class. There are minor differences, which are detailed with the procedure and function descriptions in H.14.

When a **widget** object is instantiated, it uses whatever output file is associated with it's base class. That is, if it is based on the window class, the object will need to have an OpenWin method executed to set the file associations. When the widget object is used, an output file must not be specified.

A **widget** object can be created as follows:

```
program p;  
  
joins widlib;  
  
var ti: instance of widlib.widget;  
  
begin  
  
    alert('Important Message!', 'Operation Failed!');  
    repeat { event loop }  
  
        ti.event; { get next event }  
        { process events }  
  
    until ti.event = etterm { loop until program cancelled }  
  
end.
```

Where **widget** is the widget object. Widget objects can be instantiated statically or dynamically.

H.14 Procedures and functions in widlib

For all of the following calls, If the screen file **f** is not present, the default is the standard output file. If the procedure or function is a method, the output screen file should not be specified, since it is inherent in the object.

```
procedure killwidget([var f: text;] id: integer);
```

The widget within window **f** with the logical identifier **id** is removed from the system. It will be erased from the screen, and contents under it will be restored as required.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure selectwidget([var f: text; id: integer;] e: boolean);
```

The widget within window **f** by the logical identifier **id** will enter the select state if **e** is true, otherwise the select state is removed. The exact effect of the select state depends on the widget. See the individual widget to be selected for more information. It is an error to select a widget that has no selectability.

If the window file **f** does not exist, the "output" file will be used by default.

Selection is typically used to indicate that the widget is "on", by changing its face appearance. For example, it can be checked, pressed or similar.

```
procedure enablewidget([var f: text;] id: integer; e: boolean);
```

The widget within window **f** by the logical identifier **id** will enter the enable state if **e** is true, otherwise the disable state is entered. The exact effect of the enable or disable state depends on the widget. See the individual widget to be selected for more information. It is an error to enable or disable a widget that has no such capability. The default state for all widgets is to be enabled. A widget that is disabled will stop sending events.

If the window file **f** does not exist, the "output" file will be used by default.

Disabling a widget is typically used to mark it as unusable or invalid in the current context. An example would be a "next" button where no next page or item exists. The standard method used to indicate disabled widgets is to give them a "greyed out" appearance, but the actual effect may depend on the operating system.

```
procedure getwidgettext([var f: text;] id: integer; var s: pstring);
```

Retrives the text contained by the widget with the logical identifier **id** within window **f**, and returns the text as a dynamic string **s**. It depends on the widget as to if it has text that can be read. It is an error to get text from a widget that has no such capability.

If the window file **f** does not exist, the "output" file will be used by default.

Widgets can typically have their text read if the widget provides the user with the ability to modify or edit text. In this case, retrieving the text is required to obtain the new text.

```
procedure putwidgettext([var f: text;] id: integer; view s: string);
```

Places text in the widget with the logical identifier **id** within window **f** from the string **s**. It depends on the widget as to if it can accept text placed in this manner. It is an error to place text in a widget that has no such capability.

If the window file **f** does not exist, the "output" file will be used by default.

A widget will have the ability to place text if it can edit text by the user. Placing text in the widget can be used to initialize the contents of such a widget, or as part of the overall interaction with the user.

```
procedure sizewidget[g]([var f: text;] id: integer; x, y: integer);
```

Resize an existing widget. The widget with the logical identifier **id** is resized to be the size in **x** and **y**, in the window **f**.

If the graphical version is used, the size is in pixels, otherwise, the size is in characters.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure poswidget[g]([var f: text;] id: integer; x, y: integer);
```

Reposition an existing widget. The widget with the logical identifier **id** is repositioned to be at the position **x** and **y**, in the parent window of the window **f**.

If the graphical version is used, the size is in pixels, otherwise, the size is in characters.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure buttonsiz[g]([var f: text;] view s: string; var w, h: integer);
```

Finds the minimum size of a button within window **f**, with face text string **s**. The width to use is returned in **w**, and the height in **h**. Button sizing returns the minimum size in terms of the minimum amount of space needed to contain the face text, in the labeling font, and the border of the button itself. You will want to use the size as a guide to button sizing, and not for the actual size of the button. A good rule of thumb is to add %25 of the minimum height of the button to the actual height and width of the button to give the user sufficient area to click on the button. In addition, buttons should be "justified" when they appear in groups to appear as the same width. This is done by calculating a maximum size for a group of related buttons, then using the same width and height for all of them.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure button[g]([var f: text;] x1, y1, x2, y2: integer; view s: string; id: integer);
```

Places a button within window **f** in the bounding box formed by **x1**, **y1**, **x2**, **y2**, with the face text from the string **s**. The logical identifier is specified in **id**, which must be an integer from 1 to n, which is not currently in use in any other widget. The button drawn is not guaranteed to completely fill the bounding box. The button should be placed against a background that is colored with the standard background color. The button will be placed at the front of the window stacking order, and should be placed after any other widgets or child windows that the button is to appear in front of.

When the button is pressed, it will send an **etbutton** event, which contains the logical id of the button that was pressed. It is up to the system whether the event occurs when the button is depressed or released.

Buttons cannot be selected, or have their face text changed or read. Buttons can be enabled and disabled. If the button is disabled, it will not sent **etbutton** events when pressed.

```
procedure checkboxsiz[g]([var f: text;] view s: string; var w, h: integer);
```

Finds the minimum size of a checkbox within window **f**, with face text string **s**. The width to use is returned in **w**, and the height in **h**. Checkbox sizing returns the minimum size in terms of the minimum amount of space needed to contain the face text, in the labeling font, and the border of the checkbox itself, if it exists. You will want to use the size as a guide to checkbox sizing, and not for the actual size of the checkbox. A good rule of thumb is to add %25 of the minimum height of the checkbox to the actual height and width of the checkbox to give the user sufficient area to click on the checkbox.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure checkbox[g]([var f: text;] x1, y1, x2, y2: integer; view s: string; id: integer);
```

Places a checkbox within window **f** in the bounding box formed by **x1**, **y1**, **x2**, **y2**, with the face text from the string **s**. The logical identifier is specified in **id**, which must be an integer from 1 to **n**, which is not currently in use in any other widget. The checkbox drawn is not guaranteed to completely fill the bounding box. The checkbox should be placed against a background that is colored with the standard background color. The checkbox will be placed at the front of the window stacking order, and should be placed after any other widgets or child windows that the checkbox is to appear in front of.

If the window file **f** does not exist, the "output" file will be used by default.

When the checkbox is clicked, it will send an **etchkbox** event, which contains the logical id of the checkbox that was pressed. It is up to the system whether the event occurs when the checkbox is depressed or released.

Checkboxes cannot have their face text changed or read. A checkbox can be selected or deselected, and can be enabled and disabled. If the checkbox is selected, it will appear with a selected face, which is typically a checkmark in a box. If the checkbox is disabled, it will not send etchkbox events when pressed.

A checkbox only changes its appearance in response to a select, and does not keep a state that can be read by the program. Its up to the program to keep track of the state of the checkbox, and how to handle it. In particular, if the checkbox is pressed, it is up to the program to change its select status, otherwise the press will have no effect. The program can implement many different effects for checkboxes. The checkbox can toggle, or it can be one of a series of mutually exclusive selections.

```
procedure radiobuttonsiz[g]([var f: text;] view s: string; var w, h: integer);
```

Finds the minimum size of a radio button within window **f**, with face text string **s**. The width to use is returned in **w**, and the height in **h**. Radio button sizing returns the minimum size in terms of the minimum amount of space needed to contain the face text, in the labeling font, and the border of the radio button, if it exists. You will want to use the size as a guide to radio button sizing, and not for the actual size of the checkbox. A good rule of thumb is to add %25 of the minimum height of the radio button to the actual height and width of the radio button to give the user sufficient area to click on the radio button.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure radiobutton[g]([var f: text;] x1, y1, x2, y2: integer; view s: string; id: integer);
```

Places a radio button within window **f** in the bounding box formed by **x1**, **y1**, **x2**, **y2**, with the face text from the string **s**. The logical identifier is specified in **id**, which must be an integer from 1 to **n**, which is not currently in use in any other widget. The radio button drawn is not guaranteed to completely fill the bounding box. The radio button should be placed against a background that is colored with the standard background color. The radio button will be placed at the front of the window stacking order, and should be placed after any other widgets or child windows that the radio button is to appear in front of.

If the window file **f** does not exist, the "output" file will be used by default.

When the radio button is pressed, it will send an **etradbut** event, which contains the logical id of the radio button that was pressed. It is up to the system whether the event occurs when the radio button is depressed or released.

Radio buttons cannot have their face text changed or read. A radio button can be selected or deselected, and can be enabled and disabled. If the radio button is selected, it will appear with a selected face, which is typically a blacked out radio button. If the radio button is disabled, it will not sent **etradbut** events when pressed.

A radio button only changes its appearance in response to a select, and does not keep a state that can be read by the program. Its up to the program to keep track of the state of the radio button, and how to handle it. In particular, if the checkbox is pressed, it is up to the program to change its select status, otherwise the press will have no effect. The program can implement many different effects for radio buttons. The radio button can toggle, or it can be one of a series of mutually exclusive selections.

```
procedure groupsiz[g]([var f: text;] view s: string; cw, ch: integer; var w, h, ox, oy: integer);
```

Finds the required size of a group box in window **f**, with the face text given in string **s**, and the client area width **cw**, and client area height **ch**. The required width is returned in **w**, the height in **h**, and the offset to the client area in **x** and **y**. A group box consists of a border area, a label, and an internal client area. Group boxes are designed to be layered components. They contain other widgets, and provide a background for them. When a group size is found, the minimum size is found as what will contain all of the border, face text and the requested client area. The program will know where to place its widgets in the client area by the client offset, which is given as a difference between the bounding box origin, and the origin of the client rectangle.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure group[g]([var f: text;] x1, y1, x2, y2: integer; view s: string; id: integer);
```

Creates a group box in window **f** within the bounding rectangle **x1, y1, x2, y2**, the face text given in string **s**, and with the logical identifier **id**. Group boxes are containers for other widgets, and consist of a border area, the face text, and an internal client area where other widgets are to be placed.

The entire client area of the group will have the standard background color, and widgets can be placed into the client in any arrangement or number. The program should be sure to create the client area widgets after the group is created, so that they will appear in front of the group in stacking order.

The location of the client area within a group box can be found with the group sizing call.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure background[g]([var f: text;] x1, y1, x2, y2: integer; id: integer);
```

Creates a background box in the window **f**, with the bounding rectangle **x1, y1, x2, y2**, and the logical widget identifier **id**. A background is simply a rectangle with the standard background color. It is more convenient than simply painting a rectangle on the window with the background color because it handles its own redraws. Because a background box has no borders, widgets can be placed within it anywhere, and in any number. Any widgets to be placed within the group should be created after the group box is created, so that they are on top of the group box in stacking order.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure scrollvertsiz[g]([var f: text;] var w, h: integer);
```

Finds the size for a vertical scroll bar in window **f**. Returns the width in **w**, and the height in **h**. Scrollbars typically can be sized to any size, and the width of a vertical scroll bar is a suggested width designed to match others used in the same system. The height of a vertical scrollbar is simply a suggestion, and can be ignored.

If a scrollbar cannot be arbitrarily sized, then the width and height will reflect the dimensions of a fixed scrollbar.

If the window file **f** does not exist, the "output" file will be used by default.


```
procedure scrollvert[g]([var f: text;] x1, y1, x2, y2: integer; id: integer);
```

Creates a vertical scrollbar in window **f**, with bounding rectangle **x1**, **y1**, **x2**, **y2**, and logical widget identifier **id**. If possible, the scrollbar is made to fill the width and height requested. If this is not possible, then the largest scrollbar is created that fits within the bounding rectangle. There is no guarantee that the scrollbar will completely fill the rectangle. The area under the scrollbar should be drawn with the standard background color.

The scrollbar will generate several events when clicked. The **etsclull** event indicates the line up button of the scrollbar was pressed. The **etsclldr** event indicates the line down button of the scrollbar was pressed. The **etsclulp** event indicates the page up section of the scrollbar was pressed. The **etscldrp** event indicates the page down section of the scrollbar was pressed. It is system dependent as to whether the buttons generate their events on a button press or a button release.

The **etsclpos** event gives the position of the top of the slider after the user moves it. The position is returned as a ratioed **maxint** number, where 0 means the slider is at the top, and **maxint** means the slider is at the bottom. The number is affected by the size of the slider. If, for example, the slider occupies %50 of the scrollbar, then only the positions 0 to **maxint div 2** will be generated. It is undefined as to exactly when etsclpos events occur. They may only be generated when the slider is moved and then released, or they may be generated continuously as the slider is moved. If the generation is continuous, then the movements are usually subject to "rate limiting" to keep them from generating events too fast to handle.

The scrollbar will not change position on its own. The scrollbar will generate events, and it is up to the program to use those to set the scrollbar slider position, and to take action on them, such as move the screen data up or down. The page up/down and line up/down terminology is suggestive of the use of these events, but it is up to the program exactly how to use or implement these functions. Typically, these are used to move the displayed area of a document one line up or down, and one page or screenful up or down.

The size of the scrollbar slider is set by default to small, but convenient for the user to press and manipulate. This can be left alone for programs that don't require sized scrollbar sliders. The size of the slider is set by **scrollsiz[g]**. Typically, it is used to set the ratio of onscreen data shown to the entire document or other data available. For example, if %50 of the document is being displayed, then the slider should occupy %50 of the scrollbar.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure scrollhorizsiz[g]([var f: text;] var w, h: integer);
```

Finds the size for a horizontal scroll bar in window **f**. Returns the width in **w**, and the height in **h**. Scrollbars typically can be sized to any size, and the height of a horizontal scroll bar is a suggested width designed to match others used in the same system. The width of a horizontal scrollbar is simply a suggestion, and can be ignored.

If a scrollbar cannot be arbitrarily sized, then the width and height will reflect the dimensions of a fixed scrollbar.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure scrollhoriz[g]([var f: text;] x1, y1, x2, y2: integer; id: integer);
```

Creates a horizontal scrollbar in window **f**, with bounding rectangle **x1, y1, x2, y2**, and logical widget identifier **id**. If possible, the scrollbar is made to fill the width and height requested. If this is not possible, then the largest scrollbar is created that fits within the bounding rectangle. There is no guarantee that the scrollbar will completely fill the rectangle. The area under the scrollbar should be drawn with the standard background color.

The scrollbar will generate several events when clicked. The **etsclull** event indicates the line left button of the scrollbar was pressed. The **etsclldr** event indicates the line right button of the scrollbar was pressed. The **etsclulp** event indicates the page left section of the scrollbar was pressed. The **etsclrdp** event indicates the page right section of the scrollbar was pressed. It is system dependent as to whether the buttons generate their events on a button press or a button release.

The **etsclpos** event gives the position of the left of the slider after the user moves it. The position is returned as a ratioed **maxint** number, where 0 means the slider is at the left, and **maxint** means the slider is at the right. The number is affected by the size of the slider. If, for example, the slider occupies %50 of the scrollbar, then only the positions 0 to **maxint div 2** will be generated. It is undefined as to exactly when etsclpos events occur. They may only be generated when the slider is moved and then released, or they may be generated continuously as the slider is moved. If the generation is continuous, then the movements are usually subject to "rate limiting" to keep them from generating events too fast to handle.

The scrollbar will not change position on its own. The scrollbar will generate events, and it is up to the program to use those to set the scrollbar slider position, and to take action on them, such as move the screen data left or right. The page left/right and line left/right terminology is suggestive of the use of these events, but it is up to the program exactly how to use or implement these functions. Typically, these are used to move the displayed area of a document one character left or right, and one page or screenful left or right.

The size of the scrollbar slider is set by default to small, but convenient for the user to press and manipulate. This can be left alone for programs that don't require sized scrollbar sliders. The size of the slider is set by **scrollsiz[g]**. Typically, it is used to set the ratio of onscreen data shown to the entire document or other data available. For example, if %50 of the document is being displayed, then the slider should occupy %50 of the scrollbar.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure scrollpos([var f: text;] id: integer; p: integer);
```

Sets the scrollbar slider position for window **f**, scrollbar identifier **id**, to position **p**. The position is in ratioed **maxint** format. That is, 0 means to set the position to the top or left, and **maxint** means bottom or right. The position is affected by the size of the scrollbar slider. For example, if the slider occupies %50 of the scrollbar, then the range of positions would only be from 0 to **maxint** div 2. If the position given is beyond the maximum position possible, then the slider is set to the maximum travel position, and no error occurs. It is an error if the position is negative.

The program must specifically set the position of the scrollbar. The user moving the scrollbar slider may temporarily move the slider while it is being moved, but this will not remain in position after the user releases it. The program must specifically set the scrollbar position in response to the event.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure scrollsiz([var f: text;] id: integer; s: integer);
```

Sets the size of the scrollbar slider in window **f**, logical identifier **id**, to the size **s**. The size of the scrollbar slider is a **maxint** ratio, with 0 meaning infinitely small, and **maxint** meaning that it occupies the entire scrollbar. In practice, there is a practical limit to how small the slider can be. If the slider is set too small, it will be set to the minimum size, and no error will occur. If the size set is negative, then an error will result.

If the window file **f** does not exist, the "output" file will be used by default.

The size of the scrollbar is set to a reasonable default if it is never specifically set. This is typically a fairly small size that is still easy to press and manipulate by the user. This allows the scrollbar to be used when slider sizing is not supported by the program.

The meaning of the scrollbar slider size is up to the program. However, it is typically used to indicate how much of the data is onscreen. For example, if a document has %50 of its content currently displayed, then the slider would be set to %50 of the scrollbar.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure numselboxsiz[g]([var f: text;] l, u: integer; var w, h: integer);
```

Finds the width and height of a number select box for window **f**, with lower number limit **l** and upper number limit **u**. The width required is returned in **w**, and the height in **h**. The minimum width and height is determined by the maximum length of the number to be displayed, with borders and up/down arrows considered. This can be used without adding extra space.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure numselbox[g]([var f: text;] x1, y1, x2, y2: integer; l, u: integer; id: integer);
```

Creates a number select box for window **f**, in the rectangle **x1, y1, x2, y2**, with lower number limit **l**, and upper number limit **u**. The default number appearing in the box is set to the lower limit. The number select box allows the user to edit the number, or use up/down arrow controls to select the number. Any digits typed into the edit section are limited to the digits 0-9, and negative numbers are not allowed. When the user presses enter to the number edit box, an event, **etnumbox** will be sent, which includes the number selected.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure editboxsiz[g]([var f: text;] view s: string; var w, h: integer);
```

Finds the size of an edit box for window **f**, with face text string **s**. The width is returned in **w**, and the height in **h**. The string passed is a dummy, and will not be used for any purpose other than as a reference to determine the width of the required edit box. The string should contain text that is representative of the string to be edited. This could be the string that you plan to place in the edit box as its default, or it could be the worst case contents of the edit box. For example, if 8 characters is the planned edit width, the string "WWWWWWWW" (8 "W" characters) would be the largest width of edit possible. The edit box is sized to be the minimum appropriate, and can be used without extra added space.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure editbox[g]([var f: text;] x1, y1, x2, y2: integer; id: integer);
```

Creates an edit box for window **f**, in rectangle **x1, y1, x2, y2**, with logical identifier **id**. Edit boxes can be used to allow the user to enter any text. The text within an edit box can be set by **putwidgettext**, and retrieved by **getwidgettext**. This can occur at any time. When the user presses enter in the edit box, it sends an **etedtbox** event. The program can then retrieve the text from the edit box.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure progbarsiz[g]([var f: text;] var w, h: integer);
```

Finds the size of a progress bar for window **f**. The width is returned in **w**, and the height in **h**. For systems that can size progress bars arbitrarily, the height is returned as the size that matches others used in the system. The width is a suggestion, and can be ignored.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure progbar[g]([var f: text;] x1, y1, x2, y2: integer; id: integer);
```

Creates a progress bar in window **f**, in rectangle **x1, y1, x2, y2**, with logical identifier **id**. The progress bar starts by default at 0, and is entirely operated by the program with **progbarpos** calls.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure progbarpos([var f: text;] id: integer; pos: integer);
```

Sets the progress bar in window **f**, with logical identifier **id**, to the position **pos**. The position is a ratioed **maxint** number, from 0 to **maxint**. 0 indicates "no progress", and **maxint** indicates "complete". Because of rounding, it is recommended that the program specifically set **maxint** at completion, instead of using a formula.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure listboxsiz[g]([var f: text;] sp: strptr; var w, h: integer);
```

Finds the required size of a listbox for window **f**, with string list **sp**. The required width is returned in **w**, and the required height is returned in **h**. A listbox is sized such that all of the strings in the string list can be presented in it, with borders added. No extra space is required.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure listbox[g]([var f: text;] x1, y1, x2, y2: integer; sp: strptr; id: integer);
```

Creates a listbox for window **f**, in rectangle **x1, y1, x2, y2**, with string list **sp**, and logical identifier **id**. A listbox contains a series of strings that can be selected by the user. When the user clicks a string, the event **etlistbox** will be sent, which contains the number of the selected string in list order. For example, the first string in the list would be 1, and second string in the list 2, etc. If there is not enough room in the height of the listbox for all strings in the list to be presented, then the widget will use a compression method to fit the available space. This is typically done by allowing the user to scroll through the selections. If there is not enough width for the strings in the list, they are typically clipped at the right.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure dropboxsiz[g](var f: text; sp: strptr; var cw, ch, ow, oh: integer);
```

Finds the size of a drop box for window **f**, with string list **sp**. The closed width is returned in **cw**, and the closed height in **h**. The open width is returned in **ow**, and the height in **oh**. Drop boxes are used to display a list of selections as in a listbox, but they occupy less space than a listbox. Dropboxes have two bounding rectangles, one is its demensions when closed, and another when the user drops it down, or opens it. Both sizes are returned. This allows layout planning for both modes of the widget. Generally, the closed size is used to plan placement, then the open size is used to check if the widget will extend past the edges of the window when open.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure dropbox[g]([var f: text;] x1, y1, x2, y2: integer; sp: strptr; id: integer);
```

Creates a dropbox in the window **f**, within rectangle **x1, y1, x2, y2**, for string list **sp**, with logical identifier **id**. The bounding rectangle for a drop box specifies its open mode, where the user has selected and dropped down the list of items. If the size specified is greater than or equal to the open size, as determined by **dropboxsiz**, then the entire dropbox will be presented. If the size is between the closed size and the open size, the system will attempt to work around the fact that the entire list cannot be dropped down. This is typically done by allowing the user to scroll though the list. If the size is less than the closed size, the dropbox will be clipped.

When a string within the drop box is selected, it will send an **etdrpbox** event. It contains the sequential number of the string that was selected. For example, the first string sends 1, the second in the list sends 2, etc.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure droppeditboxsiz[g]([var f: text;] sp: strptr; var cw, ch, ow, oh: integer);
```

Finds the size of a drop edit box for window **f**, with string list **sp**. The closed width is returned in **cw**, and the closed height in **h**. The open width is returned in **ow**, and the height in **oh**. Drop edit boxes are used to display a list of selections, and acts as a combination of a list and edit box, but they occupy less space than a listbox. Drop edit boxes have two bounding rectangles, one is its dimensions when closed, and another when the user drops it down, or opens it. Both sizes are returned. This allows layout planning for both modes of the widget. Generally, the closed size is used to plan placement, then the open size is used to check if the widget will extend past the edges of the window when open.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure dropeditbox[g]([var f: text;] x1, y1, x2, y2: integer; sp: strptr; id: integer);
```

Creates a drop edit box in the window **f**, within rectangle **x1, y1, x2, y2**, for string list **sp**, with logical identifier **id**. The bounding rectangle for a drop edit box specifies its open mode, where the user has selected and dropped down the list of items. If the size specified is greater than or equal to the open size, as determined by **dropboxsiz**, then the entire dropbox will be presented. If the size is between the closed size and the open size, the system will attempt to work around the fact that the entire list cannot be dropped down. This is typically done by allowing the user to scroll though the list. If the size is less than the closed size, the dropbox will be clipped.

When a drop box string is selected, or enter is hit while editing, it sends the event **etdrebox**. There is no other information associated with this event. Since the text is editable, it could be anything, and may not match one of the list entries. Instead, the program should use **getwidgettext** to retrieve the result of the edit.

Drop edit boxes default to a blank edit string. The idea of the drop edit box is that the user can simply use it as an edit box to enter the needed data, or drop down a list of preselected items. If you wish to make one of the string list items the default, or even a text that is not on the list, use the **putwidgettext** to initialize the edit field.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure slidehorizsiz[g]([var f: text;] var w, h: integer);
```

Finds the size of a horizontal scrollbar for window **f**. The required width is returned in **w**, and the required height in **h**. The height of a slider is chosen so that they match other slidebars used in the system. The width is a suggestion, and can be ignored.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure slidehoriz[g]([var f: text;] x1, y1, x2, y2: integer; mark: integer; id: integer);
```

Creates a horizontal slider in window **f**, in bounding rectangle **x1, y1, x2, y2**, with **mark** number of tick marks, and a logical identifier **id**. Sliders give a convenient way to select from a range of values. The system will either size the slider to fit the given rectangle, or choose the slider representation that is as large as possible, but still fits the given rectangle. It is not guaranteed that the slider will fill the entire rectangle. This means that it is important for the entire background under the rectangle to be set to the standard background color.

When the slider is moved by the user, it generates a **etsldpos** event. This event carries the new position of the slider, which is a ratioed **maxint** number, from 0 to **maxint**. 0 means the slider is at the extreme left, and **maxint** means the slider is at the extreme right.

There is no guarantee as to when a slider generates its events. It can generate them as the slider is moved, or it may wait until the user moves, and then releases the slider to generate events. Sliders automatically update the position of the slider, and do not need to be set by the program.

The number of tick marks given by **mark** are evenly distributed across the slider. If **mark** is zero, then no tick marks are placed at all. Tick marks have no other effect besides appearing on the slider.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure slidevertsiz[g]([var f: text;] var w, h: integer);
```

Finds the size of a vertical slider for window **f**. The required width is returned in **w**, and the required height in **h**. The width of a slider is chosen so that they match other slidebars used in the system. The height is a suggestion, and can be ignored.

If the window file **f** does not exist, the "output" file will be used by default.


```
procedure slidevert[g]([var f: text;] x1, y1, x2, y2: integer; mark: integer; id: integer);
```

Creates a vertical slider in window **f**, in bounding rectangle **x1, y1, x2, y2**, with mark number of tick marks, and a logical identifier **id**. Sliders give a convenient way to select from a range of values. The system will either size the slider to fit the given rectangle, or choose the slider representatin that is as large as possible, but still fits the given rectangle. It is not garanteed that the slider will fill the entire rectangle. This means that it is important for the entire background under the regtangle to be set to the standard background color.

When the slider is moved by the user, it generates a **etsldpos** event. This event carries the new position of the slider, which is a ratioed **maxint** number, from 0 to **maxint**. 0 means the slider is at the extreme top, and maxint means the slider is at the extreme bottom.

There is no garantee as to when a slider generates its events. It can generate them as the slider is moved, or it may wait until the user moves, and then releases the slider to generate events. Sliders automatically update the position of the slider, and do not need to be set by the program.

The number of tick marks given by **mark** are evenly distributed across the slider. If **mark** is zero, then no tick marks are placed at all. Tick marks have no other effect besides appearing on the slider.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure tabbarsiz[g]([var f: text;] tat, tar, tab, tal: boolean; cw, ch: integer; var w, h, ox, oy: integer);
```

Finds the size of a tabbar, in window **f**, with tab side active state **tat, tar, tab, tal**, client width **cw**, and client height **ch**. The required width is returned in **w**, the height in **h**, and the client offset in x and y. The size of a tabbar is enough to hold the height of the labeling font (in whatever orientation), plus border areas, any selection scrolling arrows, and the client area.

A tabbar can have tabs on any of its four sides. The enables for the tabs on a side are:

tat	Top
tar	Right
tab	Bottom
tal	Left

They are in clockwise order starting from the top.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure tabbarclient[g]([var f: text;] tat, tar, tab, tal: boolean; w, h: integer; var cw, ch, ox, oy:
integer);
```

Finds the size of a tabbar client area, in window **f**, with tab side active state **tat**, **tar**, **tab**, **tal**, tabbar width **cw**, and tab bar height **ch**. The client width is returned in **w**, the height in **h**, and the client offset in x and y. This procedure is used to find the client area for a specific size of tabbar.

A tabbar can have tabs on any of its four sides. The enables for the tabs on a side are:

tat	Top
tar	Right
tab	Bottom
tal	Left

They are in clockwise order starting from the top.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure tabbar[g]([var f: text;] x1, y1, x2, y2: integer; spt, spr, spb, spl: strptr; id: integer);
```

Creates a tab bar, in the window **f**, with string lists **spt**, **spr**, **spb**, **spl**, and logical identifier **id**. A tabbar gives the user a paradygm of a book with tabs on the side. The list of tabs, which are specified by the program, each generate events. The program establishes the view to be selected in the client area at the center of the tabbar, then it uses the tab events to switch the views in the client. If there is not enough area to display the full list of tabs, the system will allow the user to scroll through them with arrows.

To locate the tabbar and client, the **tabbarsiz** call is used to establish the size and client offset. Then, the client is offset into the tabbar. The client widgets or child windows must be created after the tabbar in order to be placed to the front of the stacking order.

When a tab is selected, it generates an **ettabbar** event, which contains both the logical tabbar **id**, and the number of the string list that was selected. This number will be the number in the string list. For example, the first string in the list will be 1, the second 2, etc.

A tabbar can have tabs on any of its four sides. The string lists that form the tabs on a side are:

spt	Top
spr	Right
spb	Bottom
spl	Left

They are in clockwise order starting from the top.

If tabs are not required on a given side, the string list for that side is passed as nil. When a tab list is not active on a side, the client area will be extended to cover the tabs on that side. If no tab string list is active, then a tabbar is equivalent to a group.

If the window file **f** does not exist, the "output" file will be used by default.

```
procedure alert(view title, msg: string);
```

Creates an alert dialog, with window title **title**, and client message **msg**. The alert dialog is a freestanding window that is placed at the front of the desktop stacking order. It is generally used to display an error, warning or other attention condition. The window title should tell the user what application is generating the alert, and the client message should give the error or warning. The user dismisses the dialog, and the program holds until the dialog completes.

```
procedure querycolor(var r, g, b: integer);
```

Creates a color select dialog. The dialog "flows through" to set its parameters. When called, **r** contains the default red, **g** the default green, and **b** the default blue colors. These defaults are used to set the dialog default selection. When the user chooses a color, the same parameters return the new selection. If the user cancels, or leaves the default selection alone, the input colors will simply be left as the default output colors. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

```
procedure queryopen(var s: pstring);
```

Creates an open file dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default filename to open (which could be null). This default is used to initialize the dialog default. When the user edits a filename, that is then returned in **s** as well. The input and out put strings are not the same even if the user chooses the default. The result string must be disposed of by the caller, and the default string supplied to the dialog is allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

If the user cancels, a null string is returned (a string with zero length, not a nil pointer).

```
procedure querysave(var s: pstring);
```

Creates an save file dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default filename to save (which could be null). This default is used to initialize the dialog default. When the user edits a filename, that is then returned in **s** as well. The input and out put strings are not the same even if the user chooses the default. The result string must be disposed of by the caller, and the default string supplied to the dialog is allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

If the user cancels, a null string is returned (a string with zero length, not a nil pointer).

```
procedure queryfind(var s: pstring; var opt: qfnopts);
```

Creates a find string dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default search string (which could be null). This default is used to initialize the dialog default. When the user edits a search, that is then returned in **s** as well. The input and out put strings are not the same even if the user chooses the default. The result string must be disposed of by the caller, and the default string supplied to the dialog is allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

The find dialog may set one or more of several options from the set provided in **opt**. These are set before the call, and they are used to initialize the defaults in the dialog. When the dialog terminates, the new state of the options are returned in **opt** as well. If the dialog does not implement a particular option, then the input value will simply be copied to the output value without change.

If the user cancels, a null string is returned (a string with zero length, not a nil pointer).

```
procedure queryfindrep(var s, r: pstring; var opt: qfropts);
```

Creates a find/replace string dialog. The dialog "flows through" to set its parameters. When called, **s** contains a default search string (which could be null), and **r** contains the default replacement string (which could be null). This default is used to initialize the dialog default. When the user edits a search, that is then returned in **s** and **r** as well. The input and out put strings are not the same even if the user chooses the default. The result strings must be disposed of by the caller, and the default strings supplied to the dialog are allocated and deallocated entirely by the caller as well. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

The find dialog may set one or more of several options from the set provided in **opt**. These are set before the call, and they are used to initialize the defaults in the dialog. When the dialog terminates, the new state of the options are returned in **opt** as well. If the dialog does not implement a particular option, then the input value will simply be copied to the output value without change.

If the user cancels, a null string is returned (a string with zero length, not a nil pointer).

```
procedure queryfont([var f: text;] var fc, s, fr, fg, fb, br, bg, bb: integer; var effect: qfteffects);
procedure queryfont([var f: text;] var fc, s: integer; var fcl, bcl: color; var effect: qfteffects);
```

Creates a query font dialog. The dialog "flows through" to set its parameters. When called, **fc** contains the font code, **s** contains the size, **fr**, **fg** and **fb** contain the foreground red, green and blue colors, **br**, **bg**, **bb** contains the background red, green and blue colors, and effect contains a set of font effects. These values are used to initialize the dialog defaults. The user then sets any or all of the parameters, and the results are copied back to the same output parameters. The dialog is presented to the front of the desktop stacking order, and the program holds until the dialog is complete.

If the dialog does not have a particular feature such as color set ability or one or more effects, the input for that parameter is simply copied to the output.

I Annex I: Sound Library

sndlib adds both a synthesizer interface via the MIDI standard, and the ability to play wave files. It implements or takes advantage of a sequencer that programs the exact time at which each of the events to make a complex combination of sounds occur.

The MIDI interface is defined as a serial interface, but the target of a MIDI port can be a standard serial MIDI daisy chain, another type of interface that carries MIDI commands (such as USB), or simply terminate in an internal sound card or even a software synthesiser.

I.1 Ports

A port is the basic MIDI output device. Typically, a computer has two of them, the sound card internal to the computer, and the external MIDI jack. These ports are labeled **synth_out** and **synth_ext**, respectively. A synthesizer output is opened with **opensynthout**. It can be closed with **closesynthout**. All synthesizer ports are automatically closed when the program closes.

```
synth_out = 1; { the default output synth for host }  
synth_ext = 2; { the default output to external synth }
```

The total number of synthesizer output ports is found by **synthout**. Notes

```
type note = 1..128; { note number for midi }
```

The basic work of making music is playing notes. MIDI can play 128 notes, numbered from 1 to 128. This ranges in frequency from 8 Hertz, or cycles per second, to 12 Kilohertz. This is approximately the range of human hearing. MIDI can also change each note in frequency enough to move it to the note next to it (and then some), so MIDI is able to reach any frequency desired.

Humans perceive a frequency that is 4 times higher as being only twice as high. If a musical note is doubled in frequency, it will be perceived as the same note one octave higher. There are twelve notes in an octave. In the lowest octave, they are:

```
{ the notes in the lowest octave }
```

```
note_c      = 1;  
note_c_sharp = 2;  
note_d_flat = 2;  
note_d      = 3;  
note_d_sharp = 4;  
note_e_flat = 4;  
note_e      = 5;  
note_f      = 6;  
note_f_sharp = 7;  
note_g_flat = 7;  
note_g      = 8;  
note_g_sharp = 9;  
note_a_flat = 9;  
note_a      = 10;  
note_a_sharp = 11;  
note_b_flat = 11;  
note_b      = 12;
```

The bases of the octaves are:

```
{ the octaves of midi, add to note to place in that octave }
```

```
octave_1 = 0;  
octave_2 = 12;  
octave_3 = 24;  
octave_4 = 36;  
octave_5 = 48;  
octave_6 = 60;  
octave_7 = 72;  
octave_8 = 84;  
octave_9 = 96;  
octave_10 = 108;  
octave_11 = 120;
```

So any note in any octave can be found by:

`note+octave`

For example, C in the 6th Octave:

`note_c+octave_6`

Notes are activated in MIDI by the **noteon** procedure, and deactivated by **noteoff**. Each of these calls may take:

- A Port
- A time
- A channel
- A note
- A volume

Each We will go over each of these parameters separately. The time to play will be discussed below. For now, it can be zero, which means "now". The channel is the instrument type to play it to, for instance, a piano, or an organ, or a tuba. The note is the logical note number we saw above, one of the 128 MIDI notes. The volume gives the volume the particular note is to be played at. This might seem odd that we can control each note, but this is like a real instrument. A piano note can be louder if hit harder.

A note can either last forever, until we turn it off with `noteoff`, or it can stop on its own. For example, an organ plays as long as you hold the key down, but a string instrument plays a note when you pluck the string, then dies away. `noteoff` need not be used for these instruments, but can still be used to cause the note to be "clipped" off early, much as if the player put a hand on the string to stop it. Similarly, a `noteon` can be used to restart the note, even while it is playing.

1.2 Channels and Instruments

```
type channel      = 1..16; { channel number }
    instrument = 1..128; { instrument number }
```

MIDI has from 1 to 16 logical channels, indexed by a logical channel number. Although there are 128 instruments, you can only play 16 of them at any one time. To play an instrument, it must be assigned to a channel. This is done with **instchange**. The instruments we can choose from are:

```
{ Standard GM instruments }

{ Piano }

inst_acoustic_grand      = 1;
inst_bright_acoustic     = 2;
inst_electric_grand      = 3;
inst_honky_tonk          = 4;
inst_electric_piano_1    = 5;
inst_electric_piano_2    = 6;
inst_harpsichord         = 7;
inst_clavinet            = 8;

{ Chromatic percussion }

inst_celesta             = 9;
inst_glockenspiel        = 10;
inst_music_box           = 11;
inst_vibraphone          = 12;
inst_marimba             = 13;
inst_xylophone           = 14;
inst_tubular_bells       = 15;
inst_dulcimer            = 16;
```

```
{ Organ }

inst_drawbar_organ      = 17;
inst_percussive_organ   = 18;
inst_rock_organ         = 19;
inst_church_organ       = 20;
inst_reed_organ         = 21;
inst_accoridan          = 22;
inst_harmonica          = 23;
inst_tango_accordian    = 24;

{ Guitar }

inst_nylon_string_guitar = 25;
inst_steel_string_guitar = 26;
inst_electric_jazz_guitar = 27;
inst_electric_clean_guitar = 28;
inst_electric_muted_guitar = 29;
inst_overdriven_guitar    = 30;
inst_distortion_guitar    = 31;
inst_guitar_harmonics     = 32;

{ Bass }

inst_acoustic_bass      = 33;
inst_electric_bass_finger = 34;
inst_electric_bass_pick  = 35;
inst_fretless_bass      = 36;
inst_slap_bass_1        = 37;
inst_slap_bass_2        = 38;
inst_synth_bass_1       = 39;
inst_synth_bass_2       = 40;

{ Solo strings }

inst_violin             = 41;
inst_viola              = 42;
inst_cello              = 43;
inst_contrabass         = 44;
inst_tremolo_strings    = 45;
inst_pizzicato_strings  = 46;
inst_orchestral_strings = 47;
inst_timpani            = 48;
```



```
{ Ensemble }

inst_string_ensemble_1    = 49;
inst_string_ensemble_2    = 50;
inst_synthstrings_1       = 51;
inst_synthstrings_2       = 52;
inst_choir_aahs           = 53;
inst_voice_oohs           = 54;
inst_synth_voice          = 55;
inst_orchestra_hit        = 56;

{ Brass }

inst_trumpet              = 57;
inst_trombone             = 58;
inst_tuba                 = 59;
inst_muted_trumpet        = 60;
inst_french_horn          = 61;
inst_brass_section        = 62;
inst_synthbrass_1         = 63;
inst_synthbrass_2         = 64;

{ Reed }

inst_soprano_sax          = 65;
inst_alto_sax             = 66;
inst_tenor_sax            = 67;
inst_baritone_sax         = 68;
inst_oboe                 = 69;
inst_english_horn         = 70;
inst_bassoon              = 71;
inst_clarinet             = 72;

{ Pipe }

inst_piccolo              = 73;
inst_flute                = 74;
inst_recorder             = 75;
inst_pan_flute            = 76;
inst_blownd_bottle        = 77;
inst_skakuhachi           = 78;
inst_whistle              = 79;
inst_ocarina              = 80;
```

```
{ Synth lead }

inst_lead_1_square           = 81;
inst_lead_2_sawtooth         = 82;
inst_lead_3_calliope         = 83;
inst_lead_4_chiff            = 84;
inst_lead_5_charang          = 85;
inst_lead_6_voice            = 86;
inst_lead_7_fifths           = 87;
inst_lead_8_bass_lead        = 88;

{ Synth pad }

inst_pad_1_new_age           = 89;
inst_pad_2_warm              = 90;
inst_pad_3_polysynth         = 91;
inst_pad_4_choir            = 92;
inst_pad_5_bowed            = 93;
inst_pad_6_metallic          = 94;
inst_pad_7_halo              = 95;
inst_pad_8_sweep             = 96;

{ Synth effects }

inst_fx_1_rain               = 97;
inst_fx_2_soundtrack         = 98;
inst_fx_3_crystal            = 99;
inst_fx_4_atmosphere         = 100;
inst_fx_5_brightness         = 101;
inst_fx_6_goblins            = 102;
inst_fx_7_echoes             = 103;
inst_fx_8_sci-fi            = 104;

{ Ethnic }

inst_sitar                   = 105;
inst_banjo                   = 106;
inst_shamisen                = 107;
inst_koto                    = 108;
inst_kalimba                 = 109;
inst_bagpipe                 = 110;
inst_fiddle                  = 111;
inst_shanai                  = 112;
```

```
{ Percussive }

inst_tinkle_bell      = 113;
inst_agogo            = 114;
inst_steel_drums      = 115;
inst_woodblock        = 116;
inst_taiko_drum        = 117;
inst_melodic_tom       = 118;
inst_synth_drum        = 119;
inst_reverse_cymbal    = 120;

{ Sound effects }

inst_guitar_fret_noise = 121;
inst_breath_noise      = 122;
inst_seashore          = 123;
inst_bird_tweet        = 124;
inst_telephone_ring    = 125;
inst_helicopter        = 126;
inst_applause          = 127;
inst_gunshot           = 128;
```

When MIDI starts up, all channels are assigned logical instrument number 1, an acoustical grand piano, with the exception of channel 10. However, remember that you might be driving MIDI outward through an external chain of MIDI cables and synthesizers, so the best practice is to specifically set instruments you are going to use.

The MIDI channel system is like a band. You make an "arrangement" based on the different instruments you are going to need, so you configure each channel with an instrument, then you can start playing. Unlike a real band, the computer can quickly change instruments during the music, and start an entirely different kind of music without skipping a beat. However, it is good practice not to count on an instrument being able to complete a note that is playing if it is swapped out of its channel for another instrument. Think of it like the Saxophone player being kicked out of the band mid note. You wouldn't expect that last note to turn out right!

We mentioned that channel 10 was an exception. This channel is always reserved for percussion (or drum) sounds. Just like a band must always have a drummer. In this channel, the notes sent have a special meaning. In fact, each note selects a different instrument:

```
chan_drum = 10; { the GM drum channel }

{ Drum sounds, activated as notes to drum instruments }
```

```
note_acoustic_bass_drum    = 35;
note_bass_drum_1           = 36;
note_side_stick            = 37;
note_acoustic_snare        = 38;
note_hand_clap             = 39;
note_electric_snare        = 40;
note_low_floor_tom         = 41;
note_closed_hi_hat         = 42;
note_high_floor_tom        = 43;
note_pedal_hi_hat          = 44;
note_low_tom               = 45;
note_open_hi_hat           = 46;
note_low_mid_tom           = 47;
note_hi_mid_tom            = 48;
note_crash_cymbal_1        = 49;
note_high_tom              = 50;
note_ride_cymbal_1         = 51;
note_chinese_cymbal        = 52;
note_ride_bell             = 53;
note_tambourine            = 54;
note_splash_cymbal         = 55;
note_cowbell               = 56;
note_crash_cymbal_2        = 57;
note_vibraslap             = 58;
note_ride_cymbal_2         = 59;
note_hi_bongo              = 60;
note_low_bongo             = 61;
note_mute_hi_conga         = 62;
note_open_hi_conga         = 63;
note_low_conga             = 64;
note_high_timbale          = 65;
note_low_timbale           = 66;
note_high_agogo            = 67;
note_low_agogo             = 68;
note_cabasa                = 69;
note_maracas               = 70;
note_short_whistle         = 71;
note_long_whistle          = 72;
note_short_guiro           = 73;
note_long_guiro            = 74;
note_claves                = 75;
note_hi_wood_block         = 76;
note_low_wood_block        = 77;
note_mute_cuica            = 78;
note_open_cuica            = 79;
note_mute_triangle         = 80;
note_open_triangle         = 81;
```

Percussion instruments always stop themselves.

The same instrument can be assigned to multiple channels. This allows an instrument harmonize with itself, playing overlapping notes.

I.3 Volume

The volume can be set for each individual note. It can also be set for the entire synthesizer port by **volsynth**. Volume can even be set for each channel by **volsynthchan**.

The volume is "maxint ratioed". It exists as a value from 0 to maxint, where 0 off (no volume) and maxint is full on. It is not decibel compensated, meaning that maxint div 2 is not half volume.

Balance between left and right can be set for each channel with **balance**. Its still maxint ratioed, except that 0 means middle, maxint means full right, and -maxint means full left.

I.4 Time and the Sequencer

MIDI does not have a concept of time built into the protocol. All notes or events sent to the MIDI port are assumed to happen "Now".

sndlib has sequencer support that is used by setting a time on each event call. If the time is 0, it means to sent the note or event to the MIDI port immediately, otherwise the sequencer schedules the event to occur at the indicated time.

To start **sndlib**'s sequencer, the **starttime**, which starts a 100us counter running (it ticks every 10,000th of a second). Then, each time is specified relative to that running timer. The current time on the sequencer can be found with **curtime**, so the required time can be specified as an offset from that:

`curtime+10000`

means a time that is one second in the future.

This example dumps a "fanfare" into the MIDI port using the sequencer. It will be played using the time specified in the **noteon** and **noteoff** calls.

```

program p;

uses sndlib;

const second = 10000;          { one second }
      osec   = second div 8;    { 1/8 second }

begin

    starttime; { start sequencer }

    noteon(synth_out, 0,          1, note_c+octave_6, maxint);
    noteoff(synth_out, curtime+osec*2, 1, note_c+octave_6, maxint);
    noteon(synth_out, curtime+osec*3, 1, note_d+octave_6, maxint);
    noteoff(synth_out, curtime+osec*4, 1, note_d+octave_6, maxint);
    noteon(synth_out, curtime+osec*5, 1, note_e+octave_6, maxint);
    noteoff(synth_out, curtime+osec*6, 1, note_e+octave_6, maxint);
    noteon(synth_out, curtime+osec*7, 1, note_f+octave_6, maxint);
    noteoff(synth_out, curtime+osec*8, 1, note_f+octave_6, maxint);
    noteon(synth_out, curtime+osec*9, 1, note_e+octave_6, maxint);
    noteoff(synth_out, curtime+osec*10, 1, note_e+octave_6, maxint);
    noteon(synth_out, curtime+osec*11, 1, note_d+octave_6, maxint);
    noteoff(synth_out, curtime+osec*13, 1, note_d+octave_6, maxint);

end.

```

The fanfare plays, and your program goes on to other work, or you can wait for the sequenced time to pass by setting a timer to the time required to finish it.

If you output a note (or other action) with a 0 time while the sequencer is running, it will still occur immediately. Time 0 always means "now". The sequencer is a "flow through" model. You can time actions and notes with the sequencer, or do it yourself, or any combination thereof.

When the sequencer is no longer required, stoptime stops it. Doing that can save processor time, and probably free up system timers.

1.5 Effects

There are many effects in MIDI that can be applied to output notes. However, there is no requirement for the system to implement them. In fact, few of the effects are implemented on most computer sound cards or software synthesizers.

attack adjusts the "attack time" of each note.

release adjusts the release or "decay" time of the note.

reverb sets the amount of reverberation.

vibrato sets the tremulo.

chorus sets the tremulo.

phaser sets the tremulo.

brightness sets the tremulo.

timbre sets the tremulo.

aftertouch sets the tremulo

pressure sets the tremulo.

legato sets the tremulo.

portamento sets the tremulo.

Some missing effects can be simulated by other means. As an example, **release** control can be emulated by putting the instrument to control in its own channel, sounding the note, then lowering the volume in steps until 0, then turning the note off.

I.6 Pitch Changes

If we need a frequency that is not exactly on a note, we can "bend" the pitch with **pitch**. The pitch change is none for 0, and by default, one note up or down. In other words, the default pitch change range is one note up or down. A D note can be bent downwards to C, or upwards to E. The term "bend" comes from bending a string to change the note.

The default range of pitch changes can also be changed, by **pitchrange**. The range is a ratioed 0..maxint. 0 means no pitch range at all (disabled), and **maxint** means the full 128 notes worth of pitch range. What you pick up with total range, you lose in fine control. If the pitch range is **maxint**, each step of pitch change is going to be very coarse.

I.7 Prerecorded MIDI

We don't have to make all our MIDI commands on the fly. In fact, we can forget doing any MIDI, and just play back prerecorded MIDI files with **playsynth**. The format is system defined. Note that even though the prerecorded MIDI file has its own timing, it is played relative to the clock start position that is indicated for it.

I.8 Waveform Files

Waveform files are how we get arbitrary sounds into the computer. Anything, for any length, can be played via the waveform files. Waveform outputs go out via their own ports separate from MIDI ports. Like MIDI, however, they are selected via logical numbers from 1 to n, where n is the maximum number of waveform output devices on the computer. The total number of waveform devices present in the system can be found via waveout. By convention, the normal wave output device is 1.

Waveform devices must be opened and closed individually. They are opened with **openwaveout**, and closed with **closewaveout**.

A waveform file is played with playwave. Waveform files are usually very system dependent, so the exact format of the file will be different for different systems.

As with MIDI files, waveforms have their own timing, and are simply played relative to the indicated start time.

The playback volume for waveform files is adjusted separately from MIDI with the **volwave**.

It is possible that the implementation will only be able to play one waveform file at a time. In this case, the behavior will be to stop any currently playing waveform file and start the new one, if a new waveform play is ordered before the previous one has finished. High quality implementation will be capable of playing multiple waveform files at once, typically via mixing of the files.

I.9 Functions and Procedures in sndlib

procedure starttime;

Start time for sequencer. Starts the sequencer running. If the sequencer is already running, it will be restarted at 0.

procedure stoptime;

Stop sequencer. Halts the sequencer timer, and releases it.

function curtime: integer;

Get current sequencer time. Returns the current sequencer time, in 100 Microsecond counts. The count is guaranteed not to wrap for 24 hours.

function synthout: integer;

Find number of output synthesizers. Returns the total output synthesizers in the system.

procedure opensynthout(p: integer);

Open output synthesizer. Opens the output synthesizer by the logical number **p**, where p is 1..synthout.

procedure closesynthout(p: integer);

Close output synthesizer. Closes the output synthesizer by the logical number **p**.

procedure noteon(p, t: integer; c: channel; n: note; v: integer);

Start note. Starts a note for synthesizer **p**, in channel **c**, with note **n**, and 0..maxint ratioed volume **v**.

procedure noteoff(p, t: integer; c: channel; n: note; v: integer);

Stop note. Stops a note for synthesizer **p**, in channel **c**, with note, and 0..maxint ratioed volume **v**. **v** is usually ignored on a noteoff.

procedure instchange(p, t: integer; c: channel; i: instrument);

Change instrument. Changes the instrument assigned to a channel, for output port **p**, at time **t**, for channel **c**, to instrument **i**.

procedure attack(p, t: integer; c: channel; at: integer);

Set attack time. Sets the attack time for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed time **at**.

procedure release(p, t: integer; c: channel; rt: integer);

Set release time. Sets the release time for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed time **at**.

procedure legato(p, t: integer; c: channel; b: boolean);

Set legato. Sets legato mode on or off, for synthesizer output port **p**, at time **t**, for channel **c**, to on/off value **b**.

procedure portamento(p, t: integer; c: channel; b: boolean);

Set portamento. Sets portamento mode on or off, for synthesizer output port **p**, at time **t**, for channel **c**, to on/off value **b**.

procedure vibrato(p, t: integer; c: channel; v: integer);

Set vibrato. Sets vibrato amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **v**.

procedure volsynthchan(p, t: integer; c: channel; v: integer);

Set volume for channel. Sets volume for channel, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **v**.

procedure porttime(p, t: integer; c: channel; v: integer);

Set portamento time. Sets portamento time, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **v**.

procedure balance(p, t: integer; c: channel; b: integer);

Set channel balance. Sets the right left balance for synthesizer output port **p**, at time **t**, for channel **c**, to -maxint..maxint ratioed value. -maxint is full left, maxint is full right, and 0 is centered.

procedure pan(p, t: integer; c: channel; b: integer);

Set channel pan. Sets the right left pan for synthesizer output port **p**, at time **t**, for channel **c**, to -maxint..maxint ratioed value. -maxint is full left, maxint is full right, and 0 is centered.

procedure timbre(p, t: integer; c: channel; tb: integer);

Set timbre. Sets timbre amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **tb**.

procedure brightness(p, t: integer; c: channel; b: integer);

Set brightness. Sets brightness amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **b**.

procedure reverb(p, t: integer; c: channel; r: integer);

Set reverb. Sets reverb amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **r**.

procedure tremulo(p, t: integer; c: channel; tr: integer);

Set tremulo. Sets tremulo amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **tr**.

procedure chorus(p, t: integer; c: channel; cr: integer);

Set chorus. Sets chorus amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **cr**.

procedure celeste(p, t: integer; c: channel; ce: integer);

Set celeste. Sets celeste amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **ce**.

procedure phaser(p, t: integer; c: channel; ph: integer);

Set phaser. Sets phaser amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **ph**.

procedure aftertouch(p, t: integer; c: channel; n: note; at: integer);

Set aftertouch. Sets aftertouch amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **at**.

procedure pressure(p, t: integer; c: channel; n: note; pr: integer);

Set pressure. Sets pressure amount, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **pr**.

procedure pitch(p, t: integer; c: channel; pt: integer);

Set pitch bend. Sets the pitch "bend", or change amount, for synthesizer output port **p**, at time **t**, for channel **c**, to -maxint..maxint ratioed value **pt**. **pt** value is -maxint for full down range, maxint for full up range, and 0 for neutral (on note) pitch. The amount of pitch range is set by the ptchrange procedure, and defaults to one note down and one note up.

procedure pitchrange(p, t: integer; c: channel; v: integer);

Set pitch bend range. Sets the total amount of pitch change that can be reached by the pitch command, for synthesizer output port **p**, at time **t**, for channel **c**, to 0..maxint ratioed value **v**. 0 disables the pitch command, and maxint allows it to reach all 128 notes of MIDI. Note that increasing the range of the pitch command decreases its resolution.

procedure mono(p, t: integer; c: channel; ch: integer);

Set mono mode. Sets mono mode for synthesizer output port **p**, at time **t**, for channel **c**, for the number of channels **ch**. See MIDI specification for details.

procedure poly(p, t: integer; c: channel);

Set polyphonic mode. Sets polyphonic mode for synthesizer output port **p**, at time **t**, for channel **c**. Reverses the effect of a mono operation.

procedure playsynth(p, t: integer; sf: string);

Play MIDI synthesizer file. Plays the MIDI instructions from the file by the name in **sf**, for output synthesizer **p**, at time **t**.

function waveout: integer;

Find number of waveform output files. Returns the total number of waveform files in the system.

procedure openwaveout(p: integer);

Open waveform device. Opens the logical waveform device **p**, where **p** is 1..**waveout**.

procedure closewaveout(p: integer);

Close waveform device. Closes the logical waveform device **p**.

procedure playwave(p, t: integer; sf: string);

Play waveform file. Plays the waveform file by the name **sf**, for output waveform device **p**, at time **t**.

procedure volwave(p, t, v: integer);

Set waveform volume. Sets the output waveform device volume for logical device **p**, at time **t**, to 0..maxint ratioed value **v**.

J Annex J: Networking Library

netlib gives Pascaline the ability to transfer data over a network such as the internet. It does this by connecting standard Pascal files to network resources. Because of the use of standard file mechanisms, few added calls are needed.

To open a new network connection, **opennet** is used. To close network connections, the standard Pascaline **close** is used. **opennet** uses an address/port pair to indicate the network address of the server, and the port within the server. The address of a server, as determined from its name in characters, is found with **addrnet**.

When a remote network port is opened, netlib treats the connection as a pair of communications channels, one going to, and one coming from, the remote resource. This makes it easier to use the standard idea in Pascal of a file having a read or write mode.

J.1 Functions and Procedures in netlib

```
procedure opennet(var infile, outfile: file; addr: lcardinal; port: lcardinal);
```

The server is indicated by a logical address number **addr**, whose exact meaning and format is dictated by the network itself. A logical port number **port** selects which resource within the server is being accessed. For the internet, this is a series of fixed constants that give the exact service being asked of the far server.

When a network link is opened, the input coming from the far server is connected to the **infile** of the **opennet** call, and the output to be sent to the far side is connected to the **outfile**. These files establish two way communication with the far server.

```
procedure addrnet(name: string; var addr: integer); forward;
```

addrnet takes the logical name of a server in string **name** and finds the address number **addr** for it. Such names are formatted according to the needs of the network. In the internet, such names consist of the characters:

Network connections are ended by the end of the program, or by using the standard file **close()** procedure on either of the in or out files of the connection.

K Annex K: Character sets

Like Pascal, Pascaline has no standard character set representation. Individual characters can be any size, and the mapping of character numerical values to character glyphs or figures is entirely system dependent.

The character set used must provide all of the characters in the word-symbols and special symbols sections listed above, with the addition of a space.

This annex describes two different character sets that are commonly used with Pascaline, the ISO 8859-1 and ISO 10646 sets. For the purposes of this annex, it is assumed that users of alternative ISO 8859 code pages will be migrating to the ISO 10646 standard (see below).

K.1 ISO 8859-1 Character Set Encodings

Pascaline itself uses only the lower page ASCII codes, which are common to all ISO 8859 pages, and only uses a subset of those

The ISO 8859-1 characters require a set of 256 elements to be able to represent the declaration:

```
set of char;
```

K.2 ISO 10646 Character Set Encodings

The ISO 10646 or Unicode character standard specifies up to 32 bits of character values. The two variations in common use are UCS-2 and UCS-4.

Variant	Bits	set of char requirement
UCS-2	16	8192 bits
UCS-4	32	536870912 bits

The exact format of a character under the Unicode system is system dependent. It commonly varies according to if the machine implemented on arranges its bytes from most significant to least (so called "big endian"), or from least significant to most (so called "little endian").

Unicode uses the first 0 to 127 character values to represent the ASCII (or ISO 646) character set. It uses the first 0 to 255 values to represent the 8859-1 or latin-1 character set. Unicode also includes all of the other parts of the ISO 8859 code pages in higher value codes. Unicode is designed to be a functional replacement for the ISO 8859 family.

K.3 Unicode text file I/O

When considering Unicode formats, it is necessary to consider both the internal memory format, and the form as written and read from Pascal text files. In the case of Unicode, they are not necessarily the same.

There are several in-file possible text representations for Unicode. The most common are:

Name	Bits	Endian mode

UTF-8	8	None
UTF-16	16	Big or little endian
UTF-32	32	Big or little endian

Although Unicode represents up to 32 bits of character code information, it is possible to represent all of these codes in either the UTF-8 or UTF-16 formats due to their use of escapes.

When big and little endian modes are considered, there are a total of possible combinations of 5 different unique formats.

There is no runtime method defined in standard Pascaline to set or switch between input and output text formats. Such selection is usually performed by a compile time change that may include special libraries, compiler options, or some combination.

The "pascaline normal" or suggested default format for text file I/O, is UTF-8. UTF-8 has the following special advantages:

1. It is compact. It represents the most common characters in the fewest number of bits.
2. It is downward compatible with ASCII or ISO 646. When a text file consists entirely of ASCII characters, the file will be identical to an ASCII file.
3. It does not have more than one endian mode.

To be fair, UTF-8 also has disadvantages:

1. When representing files whose contents are mainly outside the 0-255 code range, it may actually increase the net size of the text file.
2. It moves the ISO 8859-1 128-255 code range characters outside of the 8 bit range, making text files containing such characters possibly larger.

Use of UTF-8 makes it possible for Pascaline compilers that allow both Unicode and ISO 8859 support to switch character set modes and still keep the same text file format.

K.4 Use of different character sets

Pascaline is designed to use only one character set at a time. There is no built in system to represent more than one character set type at once, nor to switch between character set types at runtime. The configuration for character set is done as a compile time option.

L Annex L: Character escapes

Pascaline, like Pascal, does not specify a particular character set. However, the most common implementations of character sets, are ISO 8859-1, and Unicode with representations from ISO 10646. All of these character codes have ASCII as a subset. Not required, but suggested for standard Pascaline implementations are character escape sequences. The purpose of a character escape sequence is to allow access to non-printable characters, also known as "control characters" within these character sets. Additionally, there may be characters that, while they are printable, cannot be represented in program source without a particular font or character set loaded into the host operating system. In this case, character escapes allow access to these character by character code number.

The appearance of an escape within a character string is indicated by the ‘\’ character. Following the ‘\’ character is one of:

<number>

<control-identifier>

<char>

The last construct simply means that the meaning of any character, including backslash itself, can be "forced" by prefacing it with a backslash. The backslash itself must appear via a "backslash image" of ‘\’.

The <number> can be any valid character sequence code. High quality Pascaline implementations will allow '\$', '&' or '%' radix marked numbers to appear after the backslash character. Examples are:

The digits that constitute the character are limited by radix and number of digits. That is, any digit outside of the radix of the number will terminate the escape sequence. For example:

Gives an octal character escape of "01", followed by the character ‘8’, because 8 is beyond the octal radix.

Similarly, only as many digits as are needed to form a character in the current character set of the compiler are used:

Base	ISO 8859-1	UNICODE 16 bit
Decimal	3 digits	5 digits
Octal	3 digits	6 digits
Hexadecimal	2 digits	4 digits
Binary	8 digits	16 digits

So for example:

is not the character for \$45, but the character value \$00 followed by the characters "45". Leading zeros are not discarded in numbers.

It is possible for the escape sequence to be ambiguous. In this case, the backslash character must also be used as a terminator:

Is ambiguous if the desired string is 'The number is ' followed by '12', so use instead:

Control identifiers give easy access to control codes in the ISO 8859-1 character set. For ISO 8859-1, they are:

NUL	Null character
SOH	Start of Header
STX	Start of Text
ETX	End of Text
EOT	End of Transmission
ENQ	Enquiry
ACK	Acknowledgment
BEL	Bell
BS	Backspace
HT	Horizontal Tab
LF	Line feed
VT	Vertical Tab
FF	Form feed
CR	Carriage return[h]
SO	Shift Out
SI	Shift In
DLE	Data Link Escape
DC1,XON	Device Control 1
DC2	Device Control 2
DC3,XOFF	Device Control 3
DC4	Device Control 4
NAK	Negative Acknowledgement
SYN	Synchronous Idle
ETB	End of Trans. Block
CAN	Cancel
EM	End of Medium
SUB	Substitute
ESC	Escape
FS	File Separator
GS	Group Separator
RS	Record Separator
US	Unit Separator
DEL	Delete
PAD	Padding Character
HOP	High Octet Preset
BPH	Break Permitted Here
NBH	No Break Here
IND	Index
NEL	Next Line
SSA	Start of Selected Area
ESA	End of Selected Area
HTS	Character Tabulation Set
HTJ	Character Tabulation with Justification
VTs	Line Tabulation Set
PLD	Partial Line Forward
PLU	Partial Line Backward
RI	Reverse Line Feed
SS2	Single Shift 2
SS3	Single Shift 3
DCS	Device Control String
PU1	Private Use 1

PU2	Private Use 2
STS	Set Transmit State
CCH	Cancel Character
MW	Message Waiting
SPA	Start of Guarded Area
EPA	End of Guarded Area
SOS	Start of String
SGCI	Single Graphic Character Introducer
SCI	Single Character Introducer
CSI	Control Sequence Introducer
ST	String Terminator
OSC	Operating System Command
PM	Privacy Message
APC	Application Program Command
NBSP	No Break Space

So the sequence can appear:

If the underlying implementation of Pascaline is not ISO 8859, then the control mnemonics would be different. In addition, there may be other control identifiers defined for a particular implementation or character set.

```
'Hi there\del, george'
```

The use of character escapes causes a compatibility issue with standard Pascal. In standard Pascal, the character string:

```
'Hi there\george'
```

Would include the backslash, whereas a Pascaline implementation would remove the backslash. For this reason, standard complying Pascaline implementations must have a flag that enables the use of character escapes that can be turned on or off to enable normal treatment of programs. Since standard Pascal requires that any Pascaline implementation have an option that will restrict the input language to strict ISO 7185, the character escape option must also be automatically set to off by the ISO 7185 compatibility flag. The reason that the character escape option should be a separate option, and not simply tied to the ISO 7185 flag, is that it would then be impossible to have ISO 7185 complying programs work with Pascaline full language mode enabled. This would be against the rule that Pascaline is, or can be, a completely upward compatible superset of standard ISO 7185 Pascal.

M Annex M: Undefined program parameter binding

To further the idea of program parameter binding to external entities, this annex gives a suggested method of such bindings.

Program parameters are divided into three classes:

1. Predefined parameters, such as input, output, error, list and command.
2. Undefined text files.
3. Other than text type parameters.

Pascaline defines bindings for the first category, and this annex details a standard methodology for binding the second. The third category is left to the implementation, but the requirement exists for there to be an error if these parameters cannot be bound correctly (see ISO 7185 6.10 "Programs").

The suggested binding method for text file program parameters with names other than the predefined names is called "command parameter binding". For each such program parameter, reading from left to right in the program header, each file is connected to a parameter of the command used to activate the program as follows:

1. A filename is parsed from the command file.
2. The filename parsed is "assigned" to the parameter file.

The equivalent code to do this is:

```
procedure get_command_filename(var f: text);
var fn: packed array maxlin of char;
begin
    for i := 1 to 100 do fn := ' '; { clear filename }
    { skip leading spaces }
    while (command^ = ' ') and not eoln(command) do get(command);
    { get file characters }
    i := 1; { set first character position }
    while (command^ <> ' ') and not eoln(command) do begin
        fn[i] := command^;
        get(command);
        i := i+1
    end;
    assign(f, fn) { set filename }
end;
```

Where "maxlin" is the maximum string length, and assuming that "command" appears as a header parameter.

The result is that the next **reset** or **rewrite** applied to the file will cause the file, with the name specified in the command file, to be activated. Note that this method does not allow the file to be closed, then reopened, since the **close** procedure is defined as erasing the name of the file.

The side effect of parsing the actual file name from the command file is that the filename is removed from the command file, since the current read position of the file is skipped.

The point at which the above binding of the program parameters to command file filenames is before the module that contains the program parameters is activated. This means that further reads from the command file (if it appears in the header) read data that appears after any undefined text file names have been parsed and skipped.

N Standard exceptions

Normally, exceptions are either declared by the program or imported from external modules. However, several exceptions exist that are generated by the support libraries used to implement the base Pascaline language. These exceptions are all declared at the system level in Pascaline, and any program can handle such exceptions.

The following are a list of standard exceptions. If an implementation does not use a particular exception, it is recommended that the exception name be provided, even if the exception will never be thrown. This will insure software compatibility with applications that reference the exceptions here.

Name	Meaning
ValueOutOfRange	Value out of range
ArrayLengthMatch	Array lengths do not match
CaseValueNotFound	Case value not found
ZeroDivide	Zero divide
InvalidOperand	Invalid Operand
NilPointerDereference	Nil pointer dereference
RealOverflow	Real overflow.
RealUnderflow	Real underflow.
RealProcessingFault	Real processing fault.
TagValueNotActive	Tag value for enclosing variant not active.
TooManyFiles	Too many files active.
FileIsOpen	File is open, should be closed.
FileAlreadyNamed	Attempt to name a file already named.
FileNotOpen	File is closed, should be open.
FileModeIncorrect	File in incorrect mode.
InvalidFieldSpecification	Invalid field specification.
InvalidRealNumber	Invalid real number.
InvalidFractionSpecification	Invalid fraction specification;
InvalidIntegerFormat	Invalid integer format
IntegerValueOverflow	Integer value overflow.
InvalidRealFormat	Invalid real format.
EndOfFile	End of file encountered.
InvalidFilePosition	Invalid file position specified.
FilenameTooLong	The filename passed to the system was too long.
FileOpenFail	Unable to open a file.
FileSizeFail	Unable to find size of file.
FileNotOpen	An operation was attempted on a closed file.
FileCloseFail	Unable to close a file.
FileReadFail	Unable to read a file.
FileWriteFail	Unable to write a file.
FilePositionFail	File position fails.
FileDeleteFail	File delete fails.
FileNameChangeFail	File name change fails.
SpaceAllocateFail	Dynamic space allocation fails.
SpaceReleaseFail	Dynamic space release fails.
SpaceAllocateNegative	Attempt to allocate negative space.
CannotPerformSpecial	Cannot perform operation on special file.
CommandLineTooLong	Command line too long.

ReadPastEOF	Read attempted past file EOF.
FileTransferLengthZero	Attempted file transfer of zero length.
FileSizeTooLarge	File size is too large to return.
FilenameEmpty	Filename is empty.
CannotOpenStandard	Cannot open standard I/O file.
TooManyTemporaryFiles	Too many temporary files in use.
InvalidFilePosition	Invalid file position specified.
InputBufferOverflow	Input buffer overflows.
TooManyThreads	Maximum number of threads exceeded.
CannotStartThread	Cannot start new thread
InvalidThreadHandle	Invalid thread handle
CannotStopThread	Cannot stop thread
TooManyIntertaskLocks	Maximum number of interthread locks exceeded.
InvalidLockHandle	Invalid interthread lock handle
LockSequenceFail	Invalid intertask lock sequence detected
TooManySignals	Too many intertask signals active at once
CannotCreateSignal	Cannot create new intertask signal
InvalidSignalHandle	Invalid intertask signal handle
CannotDeleteSignal	Cannot delete intertask signal
CannotSendSignal	Cannot send intertask signal
WaitForSignalFail	Failure waiting for intertask signal

If an exception occurs that is not one of the above, it goes to the general exception.

If a program handles a system exception, but does not correct the problem, the program may simply fail later in the run, perhaps with a different exception. It is not advised to simply capture and then ignore exceptions.

The implementation typically connects the list of standard exceptions to equivalent exceptions in its implementation libraries. In this case, the complete set of implementation specific exceptions can be utilized by directly referencing the exceptions in the implementation libraries.